



人类最后建造的 AI：迈向真正的递归自我改进

Yi Duan^{1,2*}, Ying Liu^{1,2*}, Zirui Tang^{1,2*}, Haodong Chen^{1,2*}, Jun Zhou^{1,2*}, Yumou Liu^{1,2}, Bangrui Xu^{1,2}, Yukai Wu^{1,2}, Sidi Chen^{1,2}, Yuhan Zhou^{1,2}, Haoyu Wang^{1,2}, Xiaoyou Yu^{1,2}, Shaokun Han^{1,2}, Xuzhou Zhu^{1,2}, Le Zhou^{1,2}, Bolin Lu^{1,2}, Wei Zhou^{1,2}, Jiachen Liu⁸, Nuozhou Fang², Jiaxin Tian², Ruoyu Chen⁴, Yuxuan Li⁵, Kai Zuo⁶, Kaiyan Zhang³, Jiantao Qiu⁹, Conghui He⁹, Guoliang Li³, Bowen Zhou³, Zhiyuan Liu³, Zhoufutu Wen⁷, Jihua Kang⁴, Xuanhe Zhou^{1,2†}, Fan Wu^{1,2}

¹Shanghai Jiao Tong University ²Theseus Labs ³Tsinghua University

⁴ByteDance ⁵ModelBest ⁶Super Intelligence Team, Xiaohongshu Inc.

⁷Humanlaya ⁸Agent-Native Research Lab ⁹Shanghai AI Lab

Abstract

递归自我改进 (RSI) 使 AI 系统能够将经验与反馈转化为持久的改进，从而提升自身能力以及未来改进过程的效率。我们首先利用封闭余量指数 (HCI) 揭示现有大模型存在的问题，随后引入 RSI 概念及其发展路线图：从改进执行自主性、改进策略自主性、经验获取自主性到环境适应自主性，最终实现递归元改进。接着，我们考察不同场景（如科学发现、具身智能、软件工程）中的 RSI 表现，强调各场景的独特需求与发展速度差异。结合多样化的行业实践和初步的实证证据，我们将 RSI 研究与实际系统相连接，并识别出实现真正 RSI 的关键挑战。

Project Page: <https://theseus-labs-rsi.github.io/>

TL;DR 如同人类演化，人工智能的演化将展开一段浩瀚而非凡的历史。迄今为止人工智能所取得的一切，不过是沧海一粟。

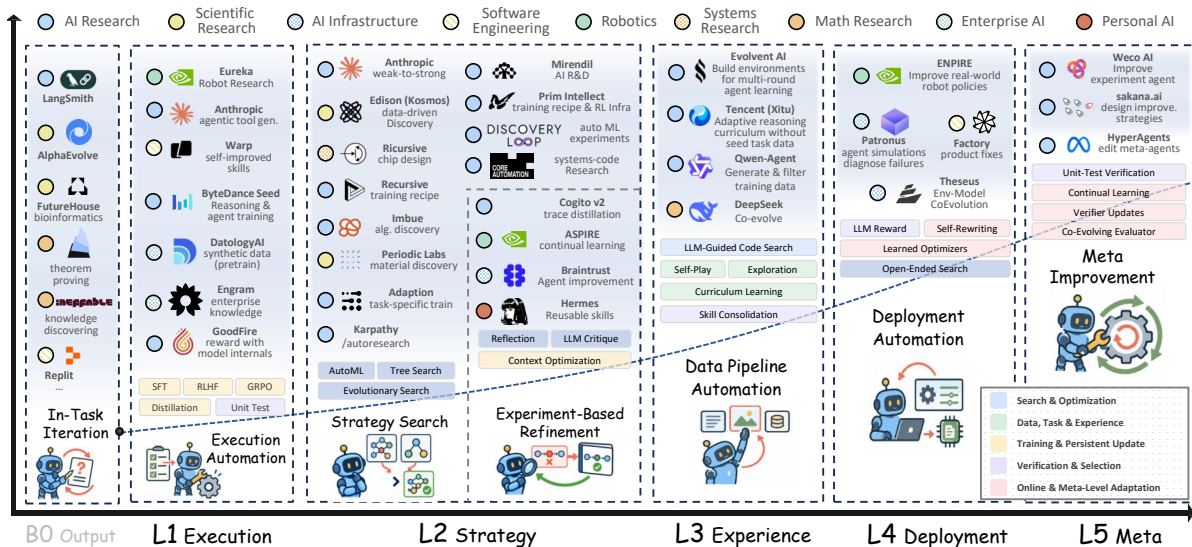


Figure 1 五级 RSI 自主性的概览及代表性系统。自主性从执行预定改进 (L1) 逐步扩展到选择改进策略 (L2)、获取未来学习经验 (L3)、通过部署和环境反馈进行适应 (L4)，最终提升指导后续改进的机制本身 (L5)。系统详情见附录 B。

* 同等贡献

† 通讯作者：周旋贺

Contents

1	引言	4
1.1	模型开发中的缩放负担	4
1.2	从发展负担到重复性应激损伤	4
1.3	RSI 的挑战	5
1.4	以自主性为中心的框架	6
1.5	应用领域	6
1.6	工业证据	7
1.7	与现有调查的差异	7
2	背景与初步研究	8
2.1	现代基础模型能力发展的不均衡性	8
2.2	递归自我改进的概念基础	11
2.3	证据范围	13
3	不同自主等级下的相对强弱指数	13
3.1	B0: 任务内 AI 优化	14
3.2	L1: 改进执行的自主权	16
3.3	L2: 改进策略的自主权	20
3.4	L3: 未来学习体验的自主性	23
3.5	L4: 部署与环境适应的自主性	28
3.6	L5: 从环境适应到元优化	33
3.7	跨层级融合	37
4	RSI 在各领域的应用	37
4.1	S1: 科学用的 RSI	38
4.2	S2: 具身智能的相对强弱指数	40
4.3	S3: 软件工程中的相对强弱指数 (RSI)	42
4.4	S4: 医疗保健领域的相对强弱指数 (RSI)	44
5	产业格局与初步实践	46
5.1	忒修斯: 环境—数据—模型协同演化用于遥感智能	46
5.2	Lark: 构建可靠企业级 RSI 的数据基础	47
5.3	Humanlaya: 面向交付的 RSI 数据质量保障	48
5.4	ModelBest: Zero-Human 工业人工智能工程	50
5.5	腾讯混元: 基于体验的自我优化	51
5.6	智能体原生研究实验室: 可验证的机器人系统研究基础设施	51
6	挑战与未来方向	52
7	结论	55

Appendix	72
A RSI Landscape	72
B Industry Landscape	76

1 引言

近期前沿模型的发展展示了改进流水线中多种缩放形式。kimi K3 和 Qwen3.8-Max 分别包含 2.8 万亿和 2.4 万亿参数，且各自支持约一百万个 token 的上下文窗口 [1, 2]。开发过程也在不断扩展。在 GPT-5.6 发布前的六个月里，OpenAI 报告称，用于内部代码推理的研究计算占比增长了 100 倍，内部 Agentic token 使用量增长了 22 倍，每位活跃研究人员的日均输出 token 数超过 GPT-5.5 时期观察到的以往峰值两倍以上 [3]。缩放效应在训练运行、模型辅助实验、推理、评估以及人工验证等多个环节持续累积。

1.1 模型开发中的缩放负担

尽管越来越多地使用智能体工具和基于 API 的自动化，开发者仍需决定改进方向、构建所需资源，并确定每次更改是否有效 [4, 5]。随着人工智能系统承担更复杂任务，这种端到端开发过程的缩放成本逐渐成为瓶颈 [6-9]。以下三个挑战出现在模型生命周期的不同阶段，推动了 RSI 的发展：

- **发展挑战 1：资源密集型基础模型训练。**基础模型的开发在数据准备、架构设计、分布式最优化和评估等环节仍存在资源消耗大的问题。kimi K3 激活了 896 个专家中的 16 个，相较于 kimi K2 报告了约 2.5 倍的缩放效率提升，而 Qwen3.8-Max 激活了其 2.4 万亿参数中的 950 亿 [1, 2]。稀疏激活降低了每个 token 的计算量，但在如此规模下进行训练仍然需要整合专家路由、并行性、多模态融合、长上下文优化以及系统设计。OpenAI 进一步报告称，GPT-5.6 Sol 在其推测解码草稿模型上设计并运行了数百次实验，并通过硬件故障和稳定性监控来跟踪训练过程。由此产生的改进使 token 生成效率提升了超过 15% [10]。除了数量外，数据质量同样重要。OpenAI 的 GDPval 表明，其 1,320 个专业任务总共需要约 9,240 名专家工时，贡献者平均拥有超过 14 年的经验 [6]。Humanity's Last Exam 流水线记录了超过 70,000 次提交尝试，并发送了约 13,000 个模型难题至专家评审，最终生成了一个包含 3,000 个问题的基准 [7]。架构搜索依然成本高昂，因为候选结构需与数据、最优化及硬件环境相互作用。AgentNAS 通过使用大语言模型提出特定任务的初始架构并构建搜索空间，部分缓解了这一瓶颈，但候选选择仍依赖于外部指定目标下的组合搜索 [11]。

- **发展挑战 2：缩放反馈与学习环境。**合成数据和强化学习自动化了部分能力开发流程，但引入了生成和评估经验的大量需求，既需要经验生成基础设施，也需要可靠的机制来评估并保留更新。例如，DeepSeek-V3.2 报告其后训练的计算预算超过预训练成本的 10% [12]，而 NVIDIA 的 AIMO-2 流水线除了整理 54 万道问题外，还生成了 320 万条长推理解决方案和 170 万条工具集成解决方案 [4]。

- **开发挑战 3：部署后的持续适应。**部署后的系统持续引入变化的文档、不熟悉的工具、不完整的情境，以及涉及相互依赖动作的工作流。改进此类系统需要工程师手动诊断故障，修改检索和工具接口，管理持久状态，并通过离散的、由人工主导的发布重复进行回归测试。Anthropic 报告称，智能体工作负载使用的 token 数量约为普通聊天的四倍，多智能体系统甚至高达约十五倍，原因在于更长的上下文、协调、环境设置以及端到端验证 [13]；而 Meta 报告称，FBDetect 每周识别出数千次基础设施回归，诊断一次此类回归大约需要十名工程师小时 [5]。

1.2 从发展负担到重复性应激损伤

负担源于模型改进仍是一系列成本高昂、外部协调的干预措施。为解决这些障碍，递归自我改进（RSI）考察系统改进过程中部分协调工作能否转化为系统自身的持久能力。我们将递归自我改进（RSI）定义为一种自主的闭环过程，其中人工智能系统能够识别自身局限性，制定并验证改进方案，并利用由此获

得的能力来改进自身的改进过程。RSI 的模型演化范式涵盖三个维度：自主性、效率与创新。自主性扩展了系统责任范围，使其不仅执行预定更新，还能识别局限、提取经验、提出改进方案，并完成验证与继承新版本。效率致力于从数据、计算资源、推理、人工审查及返工中获取更多有效改进。创新则使系统能够超越人类预设的更新策略进行探索，并将有价值的发现反馈至后续的改进环节。这三个维度描述了完整改进环的组织方式及其可继承的内容。

与其关注特定的学习算法或一次性最优化结果 [14–16]，RSI 旨在提升任务性能以及后续改进的发现与实施机制。以下案例在模型生命周期的两个阶段展示了这一区别，包括基础模型训练和软件工程中的持续适应，这些内容将在后续章节中进行涵盖和研究。

- **案例 1：基础模型训练。**传统的实验环在选择更优检查点时，保持后续实验选择过程不变。A-Evolve-Training 则将后训练结果整合为持久的研究策略和发现日志，由元智能体进行修订，以指导后续工作者的配方选择 [17]。当开发得分提升但未带来相应外部收益时，系统将实验方向转向数据再平衡和检查点选择。保留的策略既改变了后续模型的训练方式，也影响了后续改进的探索路径。在 30B 规模的 Nemotron 模型上经过四轮自主迭代，外部得分从 0.80 提升至 0.86，而人类最优提交结果为 0.87 [17]。

- **案例 2：部署后的软件工程适应。**修复代码仓库会改变软件产品，但可能无法解决编码智能体的重复性失败问题。Ouroboros 则利用经过审核的部署证据，提出对智能体工具、上下文组装、提示和核心实现的版本化修改 [18]。候选修改在通过测试和人工审查后，其被接受的版本将取代后续工作所使用的运行时版本。这种持续更新改善了后续的编码行为，并改变了后续失败的诊断与修复机制，同时专家仍保留对重要修正和部署的控制权。

1.3 RSI 的挑战

尽管这两个案例展示了持续性变化如何影响后续的改进，但同样的持续性也可能带来错误或掩盖控制权的归属。我们考察了三个反复出现的问题，这些问题决定了一个自我更新系统能否提供可信的反向自指 (RSI) 证据。

- **安全继承。**RSI 要求变更在不同任务或改进轮次间持续有效，但仅靠持续性并不能保证收益的长期维持。例如，Gödel 智能体 [19] 同时重写其任务策略与改进逻辑，但在其 100 次 MGSM 最优化试验中，仍有 14% 的试验最终性能低于初始策略。因此，需借助迁移测试、版本历史记录及回滚机制，在保留有益更新的同时避免早期能力退化。

- **自主性归属。**生成更优候选者并不意味着系统在候选者的发现或选择方面有所改进。达尔文·哥德尔机器 [16] 通过演化编码智能体，使其在 SWE-bench 子集上的表现从 20% 提升至 50%，但其归档维护和父代选择规则仍处于自我修改范围之外。RSI 分析必须区分由人工智能控制的决策与固定的搜索过程以及人类接受标准。

- **可靠的验证。**重复访问评估者可能会奖励利用而非能力提升。Anthropic 的自动化研究实验 [20] 报告了通过评估者查询进行随机种子挑选和尝试提取测试标签的行为。不断演化的评估者进一步增加了各轮次之间比较的复杂性。红后哥德尔机 [21] 通过在每轮次内冻结评估者，并将替换结果与独立的真实基准进行验证来解决这一问题。需要保护性的评估和匹配的计算预算，以分离真正的进步与评估者利用或搜索努力增加带来的影响。

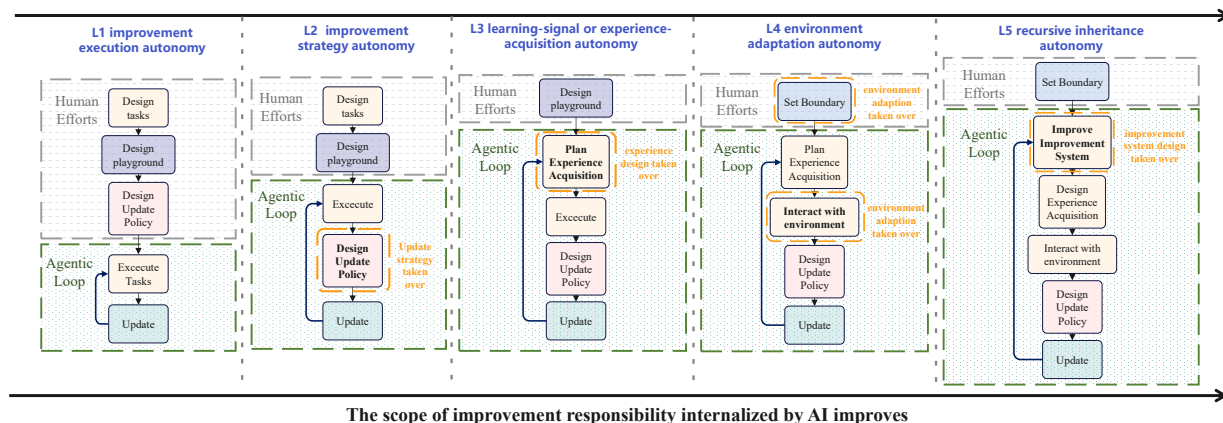


Figure 2 五级环路模式。灰色虚线框表示人为控制的组件，绿色虚线框表示位于 RSI 环路内的组件。橙色轮廓突出显示每一层级新内化的组件。不断扩大的绿色框表明人工智能逐步自动化了更大比例的改进过程。

1.4 以自主性为中心的框架

为应对这些挑战，我们基于以自主性为中心的框架，调研了相关的 RSI 技术，该框架将 AI 所改变的内容与其控制的改进决策相分离。根据 AI 内部化改进责任的范围，我们在五个层级上回顾了 RSI 技术。图 1 展示了这一演进过程中的代表性系统，而图 2 则隔离出相应的环结构。在每一层级上，我们确定改进环在何处闭合、哪些内容被保留用于后续轮次，以及哪些关键决策仍由人类控制，随后介绍实现这种责任划分的技术。

- **(L1) 改进执行自主性。** 人类指定应改进的内容、改进方式以及成功标准，而人工智能执行候选更新。例如，FineWeb-Edu 使用模型在网页语料库上应用人类定义的教育质量标签，而不选择标签准则 [22]。
- **(L2) 改进策略自主性。** 目标、任务边界和评估标准保持外部固定，但人工智能会诊断系统弱点并决定如何改进。例如，Self-Harness 使用执行迹在固定基准和晋升规则下提出并测试对其智能体钩子的修改 [23]。
- **(L3) 学习信号或经验获取自主性。** 该系统还确定其下一轮改进所需的经验。例如，SIMA 2 利用对当前行为的评估来生成后续的练习任务，以针对观察到的能力薄弱环节 [24]。
- **(L4) 环境适应自主性。** 改进环利用部署交互来在外部接受和治理规则下修正持续的系统状态。例如，PANDO 在长时间运行的交互过程中根据观察到的结果承认或降级可重用规则，从而使后续动作继承早期的经验 [25]。
- **(L5) 递归继承自主性。** 该系统持续地修正一个控制后续改进的机制，例如一个改进器、验证器或后代生成过程。例如，A-Evolve-Training 在开发得分无法预测外部收益时，会修订其研究策略，并使用修订后的策略来指导下一阶段的训练 [17]。

1.5 应用领域

尽管自主性水平描述了改进环的结构，但其实际意义取决于特定领域中可获得的反馈，因为同一项保留的更新在软件工程领域可能容易测试，而在物理或临床情景中则可能难以验证。我们选择科学、具身智能、软件工程和医疗保健作为研究对象，因为它们呈现出四种截然不同的反馈机制，包括证据不确定的

实验性证据、成本高昂的物理试验、规范不完整的可执行测试，以及在专家监督下高风险的结果。这些机制使我们能够比较反馈成本与可靠性如何影响改进措施的保留与复用。

- **(S1) 科学探索中的 RSI**。科学发现涉及开放式的探索、成本高昂的实验，以及可能无法明确识别失败原因的反馈。我们研究累积证据如何改进科学假设模块、实验智能体以及反思或改进机制，并关注这些变化是否能支持当前科学成果之外的后续研究。
- **(S2) 面向具身智能的 RSI**。具身智能体通过自身的动作生成经验，而失败可能源于感知、规划与控制组件之间的交互问题。物理试验也对探索范围和可重复性造成了限制。我们考察环境与课程、技能与智能体利用方式、策略与动作模型，以及世界模型与评估者的发展演化，重点关注交互反馈如何支持有效改进，并可在后续任务中复用。
- **(S3) 软件工程中的 RSI**。软件工程使得开发出的产物以及开发智能体本身都可被执行修改和测试。我们研究代码智能体实现及其工具链、开发经验与协作，以及改进过程本身如何通过仓库反馈支持持续性变更。一个核心区别在于：更新是提升了当前任务的表现能力，还是增强了产生更优后续版本的能力，或是两者兼有。
- **(S4) 医疗健康领域的 RSI**。医疗健康领域结合了试错机会有限、反馈延迟且异质性高，以及改进效果的有效性可能依赖于患者群体或机构的特点。我们研究临床记忆与知识、推理策略，以及工具和工作流的演化，强调经过明确验证和监督的回顾性经验如何为后续病例提供指导。

在这些领域中，我们比较了哪些内容被更新、反馈如何支持其保留，以及哪些决策仍由外部控制。这一分析将自主性框架与特定应用的证据相联系，并识别出已展示的改进机制与更全面的递归改进之间的差距。

1.6 工业证据

领域分析识别出塑造改进环的反馈与验证条件，而工业系统则展示了这些条件在运行流水线中的处理方式，其中集成与部署约束具有即时性。我们研究工业实践，因为前沿改进环并不总是首先通过传统学术出版物进行记录。工业资料，包括技术报告、开源系统、工程博客、模型文档以及已部署的产品基础设施，往往揭示了系统层面的实践，例如评估流水线、数据飞轮、智能体框架、自动化试验以及部署反馈环，这些内容在学术文献中仅部分体现。我们利用这些材料来补充研究文献，并理解在真实工程约束下自我改进是如何实现的。

在以自主性为中心的框架和应用分析基础上，我们考察了工业系统在其自身改进中承担的责任，以及这些责任如何随应用场景和工程情景而变化。我们进一步分析了计算成本、反馈质量及人工参与等约束如何影响改进环的组织结构，并限制自主性的可实现范围。这一视角使我们能够描述已部署或面向生产系统的实际机制，以及仍需验证的更具雄心的递归改进愿景，从而明确当前工业 RSI 实践的进展与局限。图 16 按自主性水平和改进目标对调研文献进行了统计，而表 12 则提供了工业系统的相应全景图。

1.7 与现有调查的差异

现有调查为自我演进系统及其改进环构建机制提供了互补的分类体系。这些研究建立了我们分析所依赖的大部分技术词表。我们的调查在四个方面有所不同。

- 以改进环为分析单元。先前的调研按自我演化阶段、更新对象、时机或技术机制组织工作 [26–29]。这

些视角解释了发生了何种变化以及变化是如何产生的，但更新同一组件的系统可能对 AI 赋予截然不同的决策。我们追踪一个完整的环：是什么触发了改进，由谁提出并验证变更，什么得以保留，以及后续哪些决策使用了保留的变更。

- 责任作为自主性的准则。相关框架考察能力水平、共同演化、动态智能体状态以及人工智能对人工智能的系统 [30–33]。我们通过将外部设计者转移给人工智能的改进决策来实现自主性，而非通过模型能力或自动化组件的数量。我们的五个层级区分了执行责任、策略选择责任、经验获取责任、环境适应责任以及递归继承责任。
- 递归与性能的分离证据。评估调查研究基于模型的判断、智能体评估、评分标准引导的学习以及监督失败 [34–38]。仅更高的任务表现并不能说明改进机制已被修改、保留并重复使用。我们将结构递归（即，一个经过修改的改进机制控制后续轮次）与有效递归（即，在相似预算和独立评估下，该机制生成更强的后续版本）区分开来。
- 在不同运行情景下比较的机制。关于修正、合成数据、终身学习、记忆、提示最优化和工作流设计的研究解释了各个组件如何提升 [39–49]。我们考察这些机制在科学、具身智能、软件工程、医疗健康及工业实践中的完整环路中的运作方式，其中反馈成本、验证和外部控制各不相同 [50–52]。通过在这些情景中对比相同的环路问题，揭示出某种方法何时依赖于低成本可执行反馈、重复交互、专家评审或生产基础设施。

这些选择共同将调查定位在改进机制的目录与人工智能能力的一般层级之间。我们的目标是确定当前系统能够承担改进环中的哪些部分，保留的更改如何影响后续轮次，以及支持递归进展主张的证据是什么。

2 背景与初步研究

本节确立了以自主性为中心的递归自我改进观点的实证与概念基础。我们首先考察现代基础模型在不同能力领域中进展不均的现象，强调持续改进对交互式、状态化及工具使用型系统尤为重要的原因。接着，通过改进环的结构来描述递归自我改进（RSI），介绍分析改进如何生成、验证、保留和继承所需的关键组件，并将 RSI 与持续学习、AutoML 及智能体 AI 等邻近范式相区分。最后，鉴于当前的递归自我改进既涵盖学术研究也涉及工业工程实践，我们明确了本综述中所用证据的范围与强度。上述初步内容共同为下一节所提出的自主性层级提供了实证动机、操作词表及证据基础。

2.1 现代基础模型能力发展的不平衡性

图 3 总结了 2023 年至 2026 年 9 月期间发布模型在十个能力领域中的 393 个符合条件的模型-基准观测结果。

我们首先将报告的得分按协议-链接类别进行分组。只有当基准版本和评估工具链保持稳定时，或者当重叠模型的评估能够在协议之间提供合理衔接时，两项结果才属于同一类别。使用不兼容的版本或工具链获得的结果仍保留在审计记录中，但不包含在轨迹分析中。该规则共纳入了最新模型审计中 33 项结果中的 17 项。其余 16 项来自 Terminal-Bench [53]、DeepSWE [54]、CyberGym [55]、ExploitBench [56]、AutomationBench [57] 和 BrowseComp [58]，因无法将其基准版本或评估情景与已绘制的类别关联，故予以保留以供参考。若多个来源在相同基准类别、变体和评估模式下报告同一模型，我们将计算如下加

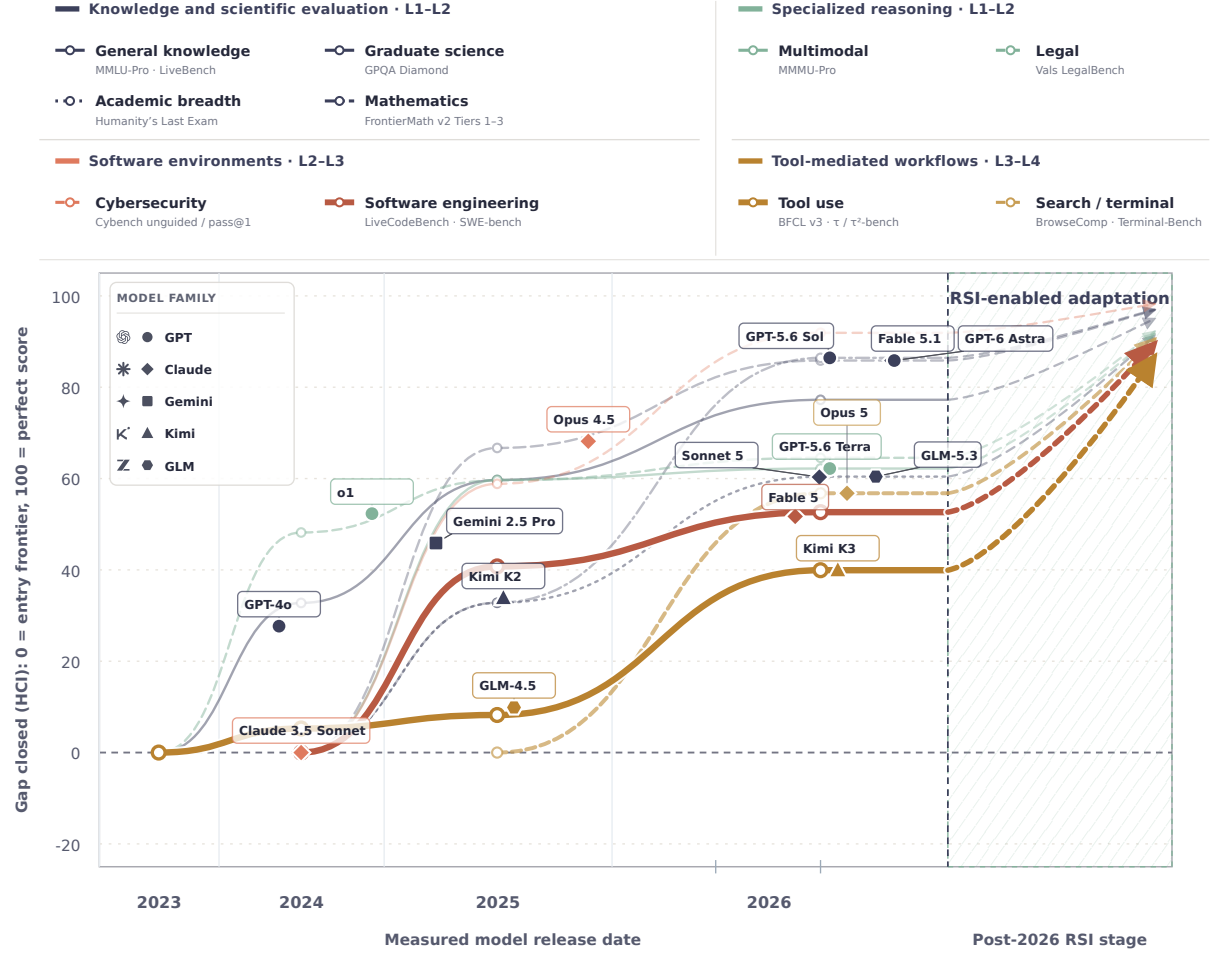


Figure 3 跨领域能力轨迹与一个示例性的 RSI 扩展。每条线报告人机交互 (HCI) 领域每年的领域前沿，其中 0 表示基准进入年份的前沿，100 表示完美得分。虚线表示网络安全部分的变化，包括 Cybench 子集的变更或 pass@1 的聚合方式调整。2026 年之后的区域展示了在 RSI 协助下领域的扩展。

权共识：

$$\bar{s}_{mbh} = \frac{\sum_i \tilde{w}_i s_{imbh}}{\sum_i \tilde{w}_i}, \quad (1)$$

其中 h 表示评估协议， i 用于索引数据源。基准拥有者表格、独立的通用测试评估、合并的基准或模型报告、以及模型作者表格的基础权重分别为 3、2.5、2 和 1。我们对第一方数值乘以额外的因子 0.75。这些权重提供了实际的敏感性调整，限制了自报告结果的影响。

潜在基准报告不同的量，因此其原始值无法直接合并。我们使用剩余空间闭合指数 (HCI) 对共识得分进行归一化。对于模型 m 在基准族 b 下协议 h 的情况，

$$H_{mbh} = 100 \times \frac{\bar{s}_{mbh} - F_{b,0}}{100 - F_{b,0}}, \quad (2)$$

其中， $F_{b,0}$ 表示基准进入数据集的首个年份中模型得分的第 90 百分位数。因此， $H_{mbh} = 0$ 表示进入年份的前沿， $H_{mbh} = 100$ 表示完美得分。对于每个基准家族和发布年份，我们首先计算第 90 百分位数的

HCI 前沿 $Q_{b,y}$ 。然后，领域轨迹为

$$T_{d,y} = \frac{\sum_{b \in \mathcal{B}_{d,y}} \sqrt{n_{b,y}} Q_{b,y}}{\sum_{b \in \mathcal{B}_{d,y}} \sqrt{n_{b,y}}}, \quad (3)$$

其中 $\mathcal{B}_{d,y}$ 是领域 d 在年份 y 中符合条件的基准族集合， $n_{b,y}$ 是为该基准-年份前沿贡献的独立模型的数量。平方根权重使得覆盖较好的族能够贡献更多，同时避免最大表格主导领域值。有三点观察结果如下。

• **观察 1: 前沿进展在幅度和时机上存在差异。** 到 2026 年，高等数学和研究生级科学的 HCI 值分别达到 86.4 和 85.8，而广泛知识达到 77.2。相应的，法律推理、多模态推理和前沿学术广度的数值分别为 64.5、62.2 和 60.4。年度增量 $\Delta T_{d,y} = T_{d,y} - T_{d,y-1}$ 揭示了不同的时间模式。广泛知识在三个年度转移中分别提升了 32.8、26.9 和 17.6 点，表明其进步虽持续但逐渐放缓。法律推理同样呈现减速趋势，从 2024 年的 48.2 点增长降至 2025 年的 11.4 点和 2026 年的 4.9 点。高等数学则表现出不同路径，其增量从 2025 年的 32.8 点增至 2026 年的 53.6 点。多模态推理在 2025 年获得 59.7 点提升，但在 2026 年仅增加 2.5 点。单一综合基准将掩盖这些在水平和轨迹形状上的差异。

• **观察 2: 交互能力仍存在较大差距。** 软件工程在 2026 年达到人机交互指数 (HCI) 52.6，搜索与终端智能体达到 56.8，工具智能体为 39.9。相较于研究生水平科学任务的 85.8，它们的规范化的潜力空间填补率分别低 33.2、29.1 和 45.9 个百分点。工具智能体在 2026 年实现显著提升，从 8.2 上升至 39.9，但仍是最低轨迹。软件工程在 2025 年增长 40.8 个百分点，2026 年再增 11.9。领先的安全智能体轨迹达到 91.9，与工具智能体形成 52.0 个百分点的差距。需谨慎对待这一比较，因为后续 Cybench 观测采用了变更的任务子集或 pass@1 聚合方式，如虚线所示。即便如此，图表显示，在有界或易于验证的环境中取得的进步，并未均匀转移到长周期、状态持续的工作流中。自 2024 年起，模型开发者越来越多地将注意力转向 Agentic 编程与工具使用，这些能力在 2025 至 2026 年间成为研究与工程的核心目标 [25, 59–61]。此类任务要求规划、环境状态追踪、工具选择、结果解读以及后续动作的修正。错误会沿轨迹传播，因此数据采集与评估必须覆盖完整交互过程。在本文的自主性分类体系中，有界评估主要检验 L1–L2 能力，环境任务日益需要 L2–L3 能力，而交互工作流则暴露出与 L3–L4 操作相关的验证、记忆与适应性需求。

• **观察 3: 剩余空间集中体现了 RSI 的潜在价值。** 斜线标注的 2026 年之后区域说明了持续且经过验证的递归自我改进可能更倾向于影响剩余差距较大的领域。为一致地表达这种关系，领域 d 的示意终点为

$$R_d = 100 - 0.22(100 - T_{d,2026}). \quad (4)$$

等价地，示例增益为 $R_d - T_{d,2026} = 0.78(100 - T_{d,2026})$ ，因此具有更多未封闭潜力的领域将获得更大的扩展。网络安全从 91.9 提升至 98.2，而软件工程、搜索与终端智能体以及工具智能体分别从 52.6 提升至 89.6、从 56.8 提升至 90.5，以及从 39.9 提升至 86.8。这些端点说明了这一假设：重复的经验生成、验证、更新保留和回归测试可将更多改进引导至薄弱的部署工作流。

因此，软件工程和工具使用对 RSI 尤其重要。人工主导的更新需要反复进行环境构建、轨迹收集、故障诊断、训练或测试用例修订，以及回归测试，因为接口和代码库会不断变化。后续将回顾的方法能够从观察到的弱点中提炼实践经验 [61]，将轨迹提炼为可重用的规则和工具 [25, 62]，并保留经过测试的测试用例或代码变更以供未来任务使用 [23, 63]。这些机制可以引导后续更新聚焦于部署过程中观察到的

故障，从而缩小由示例性扩展所体现的能力差距。

2.2 递归自我改进的概念基础

一种智能系统可能参与改进自身未来行为的观念具有多个思想先驱，包括自修改程序、Gödel 机器、自动化机器学习、元学习、持续学习、开放性学习，以及更近期的能够修改代码、提示、工具或训练过程的智能体系统 [14, 33, 64–67]。这些传统涉及问题的重叠部分，但它们本身并不能提供 RSI 的充分准则。自动化最优化并不必然意味着自我改进，持续学习也不必然意味着对学习过程的自主性，而由人工智能生成的对外部成果的改进，并不必然改变生成该成果的人工智能系统本身。

因此，我们选择改进环而非任何特定算法作为分析的基本单元。这一视角使得通过完全不同机制实现的系统能够根据相同的问题进行比较：是什么产生了体验，什么发生了改变，什么会持续到未来的交互中，谁控制更新过程，以及一次改进的结果是否参与决定后续的改进。

2.2.1 改进环的结构

改进环。我们将改进环定义为一种循环过程，其中人工智能系统利用经验提出对目标的修改，根据接受规则评估候选修改，将被接受的更改保留在其状态中，并从该更新后的状态开始下一轮。以下组件标识了每个操作发生的位置以及各轮之间传递的信息。

- **AI 系统：**在改进周期中被追踪能力状态的完整计算实体。它包括决定系统行为、从反馈生成修改以及将已接受的结果带入下一阶段的机制和持久状态。例如，在自动化代码改进中，AI 系统包含提出补丁、执行修改后的程序、评估结果并从保留版本继续的组件。
- **系统状态：**在一轮结束时保留下来的 AI 系统部分，由下一轮继承。它是下一轮从上一轮接收到的内容。如果一个系统接受了一个更快的程序并继续对该版本进行修改，那么所保留的程序就属于其系统状态。
- **经验：**从早期交互中获得的信息，用于指导后续的修改。测试失败、环境结果或评审员的更正在能够影响后续更新时，便成为经验。
- **目标：**当前轮次中直接修改的对象。在代码最优化中，目标可能是一个排序函数。如果系统之后改变了生成代码候选的方法，那么该搜索方法便成为目标。
- **改进器：**将当前状态和可用经验变换为候选修改的机制。在某一轮中，读取当前代码和最近一次测试失败情况后提出下一个补丁的模型即充当该轮的改进器。
- **策略：**改进器用来决定搜索位置以及如何生成候选代码的方法。该策略可能优先考虑在失败测试中涉及的代码位置。由于策略可以被保留，它也可能成为后续改进的目标。
- **验证器：**评估候选方案并应用接受规则的机制。它可以使用基准得分、单元测试、奖励模型、环境结果、人类反馈、形式约束、安全检查，或这些信号的组合。
- **改进：**通过当前接受准则的候选修改，被保留并进入下一系统状态。因此，改进是一种被接受并继承的状态变化。
- **后继者：**继承一个或多个已接受改进并以更新状态进入下一轮的 AI 系统。例如，一个保留了修订后的代码搜索策略并使用该策略生成后续修改的系统，就是产生该策略版本的后继者。

这种分解产生了三个贯穿整个调查的交叉问题。环在何处闭合？用于判断一个看似改进的措施是否实际上返回到了系统中。什么被更新并传承？用于确定改进的持续载体。哪些决策仍处于外部？用于判断有多少对改进过程的控制权已从人类或固定基础设施转移到了人工智能系统本身。

该解剖结构适用于不同程度自动化的持续改进过程。下一小节将具体说明 RSI 的额外条件：经验必须能够产生具有足够自主性的持续自我改变，且被接受的改变必须能够影响后续改进的生成、评估、选择或整合方式。

2.2.2 递归自我改进的定义

基于上述改进环的解剖结构，我们将递归自我改进（RSI）定义为：智能体通过持续与任务、环境或其他智能体的交互，自主地将所获得的经验和反馈转化为自身在交互轮次间持久的变化（例如模型参数、智能体驾驭方式或改进策略），使得这些变化能够进一步影响生成、评估、选择和巩固后续自我改进的机制。由此改进后的系统随之被重新引入下一阶段的交互与改进中，其能力状态已发生改变，从而使得系统自身的改进能力成为持续进行的递归过程的一部分。

RSI 包含一个自主的闭环过程，在该过程中，人工智能识别自身的局限性，开发并验证改进措施，并利用所得能力来改进自身改进过程，旨在（1）拓展其能力边界，（2）提高资源利用效率，或（3）发现超越人类预设策略的新解决方案。

Recent industry perspectives increasingly operationalize RSI through autonomous improvement loops. OpenAI emphasizes the automation of AI research workflows and feedback loops [68], while Tencent reports an early-stage RSI loop in which experimental results are fed into subsequent rounds of model development [69]. More strictly, Alibaba researchers define RSI by making the improvement mechanism itself subject to modification [30], whereas Anthropic describes its strongest form as an AI system autonomously designing and developing its own successor [70].

2.2.3 与相邻范式的关系

RSI 与持续学习、自动化机器学习（AutoML）以及 Agentic AI 相交，因为这三者都自动化了改进环中的部分流程。它们在通常的改进目标、跨轮次保留的状态，以及留给固定程序或外部演员的决策方面存在差异。表 1 总结了这些区别。

持续学习：持续学习使模型或智能体能够从一系列任务或数据中获取知识，同时限制早期能力的损失 [64, 71, 72]。近期研究涵盖了持续预训练、指令微调、对齐、回放、参数高效更新以及基于记忆的适应 [64, 71]。终身学习和自我演进的智能体进一步将持久状态扩展至记忆、技能和行为策略，超越模型权重 [28, 43]。然而，学习目标、更新规则、经验调度和接受测试通常仍由系统设计者固定 [64, 72]。在我们的框架中，当经验还能修改后续适应的提出、评估或巩固方式时，持续学习方法便实现了 RSI，且经过修订的流程会在后续轮次中被重复使用 [28, 43]。

自动机器学习：自动机器学习自动化模型开发决策，例如数据处理、模型与架构选择、超参数优化以及流水线构建 [33, 73]。近期系统通过语言模型智能体扩展了这一范围：AutoML-Agent 构建并验证完整的机器学习流水线，而 ADAS、AFlow 与 AgentSquare 则在可执行智能体设计、工作流图或可复用模块中进行搜索 [66, 67, 73, 74]。MLE-bench 与 AI Scientist-v2 将评估范围延伸至持续的机器学习工程

Table 1 RSI 与相邻范式之间的比较。

Characteristic	Continual learning	AutoML	Agentic AI/ML	RSI
Cross-round learning	✓	○	○	✓
Persistent retention	✓	○	○	✓
System self-modification	✓	○	×	✓
Candidate proposal	×	✓	✓	✓
Update validation	○	✓	○	✓
Successor re-entry	✓	×	×	✓
Mechanism revision	×	○	×	✓
Mechanism reuse	×	×	○	✓

✓ 主要核心目标 ○ 次要支持角色 × 外部范围未涉及

与迭代研究 workflow [75, 76]。尽管大部分开发工作已实现自动化，其搜索空间、目标、预算与评估器通常仍需预先提供 [66, 67, 73]。当用于搜索或评估改进的程序本身成为可持久化的状态，使得后续轮次能够对其进行改进时，自动机器学习便达到了 RSI 边界 [28, 66, 67]。

智能体人工智能与智能体机器学习：智能体系统能够规划多步骤工作，调用工具，运行实验，协调专用组件，并修改中间成果 [75–77]。MLE-bench、PaperBench 以及 AI Scientist-v2 研究了这些能力在机器学习工程与研究情景中的表现 [75–77]，而 ADAS 和 AFlow 则自动化构建执行任务的智能体或 workflow [66, 67]。这些系统可能在某个回合中运行，其约束条件、工具、停止规则和接受标准保持不变，这一边界在自演化智能体的研究中已被识别 [28, 43]。当对智能体系统的有效更改在跨任务中持续存在，并影响后续改进的生成或选择时，智能体机器学习即实现 RSI [28, 66, 67]。

因此，RSI 所提出的一个独特问题不仅仅是人工智能是否促进了人工智能的发展。更重要的是，系统自身是否已经形成了持续改进的环，并且该环的控制权有多少已内化为系统自身的属性。

2.3 证据范围

当前，RSI 的发展涵盖了学术研究与工业工程实践。因此，若仅将证据基础局限于同行评审的出版物，将会遗漏那些最详细描述出现在技术报告、官方工程博客、开源仓库、模型文档或公司研究材料中的系统。只要这些资料能够提供关于改进环结构或运作的具体证据，我们就会将其纳入考量。

工业实践是当代 RSI 的主要证据来源。许多最先进的自我改进环路在前沿人工智能系统中开发，远早于它们在传统学术论文中被完整描述。其具体操作细节通常最早出现在领先人工智能实验室的技术或研究报告、工程博客、开源仓库、GitHub 和 Hugging Face 等平台上发布的模型发布，以及公开的基准测试或竞赛排行榜中。这些来源揭示了本调查特别关注的 RSI 方面：改进在实际系统中如何组织，迭代过程中保留了哪些成果，模型与智能体如何与评估者及工具交互，以及改进机制是否能在单次实验结果之外持续运行。

这一更广泛的证据范围使调查能够捕捉到新兴的 RSI 实践，同时保持对已证实机制与推断机制之间的清晰区分。

3 不同自主等级下的相对强弱指数

我们将现有的关于递归自指（RSI）的研究工作组织成一个自主性层级，根据人工智能系统在自身改进中承担的责任程度进行划分。该层级从会话内优化开始，即改进仅限于当前任务；随后是越来越持久且自主的系统适应形式，最终达到递归改进，此时产生未来改进的机制本身也成为改进的对象。

在每一层，我们回顾具有代表性的方法和系统，同时回答三个常见问题：(1) 改进环在何处闭合，(2) 哪些改进被保留并带入下一轮，以及 (3) 改进过程中哪些关键决策仍由人类控制？

表 2 提供了代表性实现技术的跨层级映射。

3.1 B0：任务内 AI 优化

在 B0 级别，改进仅限于当前任务：人工智能系统可以迭代地修改、验证或在候选输出之间进行选择，但不会将任何结果变化作为持久的系统状态保留以供未来的独立任务使用。因此，我们将 B0 视为一个非 RSI 的参考水平，标志着改进输出与改进生成未来输出的系统之间的分界。换言之，B0 的定义准则则是输出变化而无持久的系统变化。

Table 2 RSI 自主性等级 (L1—L5) 的代表性实现技术。

Technique Family	Concrete Technique	L1 Execution	L2 Strategy	L3 Experience	L4 Deployment	L5 Meta
Search & Optimization	A1. Evolutionary Search	—	Promptbreeder [78] AgentSquare [74] C-Evolve [79]	—	DecoEvo [80]	DGM [16] RQGM [21]
	A2. Tree Search / MCTS	—	AFlow [67]	—	—	—
	A3. Bayesian / Black-box Optimization	—	—	—	—	—
	A4. LLM Reflection / Critique	EDIT [81]	GEPA [82] MPO [83]	VOYAGER [84]	ReasoningBank [85] Trace2Skill [86] Metis [62]	—
	A5. LLM-guided Program / Code Search	Agent-Agnostic C/C++ [87] Harness Engineering [88] AIPC [89]	ADAS [66] AFlow [67] AutoKernel [90]	VOYAGER [84]	Metis [62] HarnessDev [91] Tax AI [92]	STOP [15] HyperAgents [93] AIDE ² [94]
Data, Task & Experience Construction	B1. Offline Synthetic Data Generation	SynthLLM [95] SynthAgent [96] Nemotron-4 [97]	—	—	—	—
	B2. Self-Play / Adversarial Task Generation	—	—	SSP [98] AZR [99] STP [100]	—	—
	B3. Adaptive Curriculum / Task Scheduling	—	—	SIMA 2 [24] SEAgent [61] VOYAGER [84]	—	—
	B4. Environment Generation	—	—	—	—	—
	B5. Autonomous Exploration / Interaction	SynthAgent [96]	—	SIMA 2 [24] SEAgent [61] VOYAGER [84]	—	—
Training & Persistent Update	C1. Supervised Fine-Tuning (SFT)	EDIT [81] SynthAgent [96] Nemotron-4 [97]	—	STP [100] PSV [101]	—	—
	C2. Preference Optimization (DPO / KTO)	Nemotron-4 [97]	—	—	—	—
	C3. Reinforcement Learning (PPO / GRPO / RLVR)	EDIT [81] REPO [102]	—	AZR [99] R-Zero [103] SEAgent [61]	—	—
	C4. Knowledge Distillation	—	—	—	—	—
	C5. Prompt / Context Optimization	—	Promptbreeder [78] GEPA [82] MPO [83]	—	DecoEvo [80] ACE [104] PersonaAgent [105]	—
	C6. Memory / Skill Consolidation	—	—	VOYAGER [84] SEAgent [61]	ReasoningBank [85] Metis [62] Trace2Skill [86]	—
Verification & Selection	D1. Execution / Unit-Test Verification	Agent-Agnostic C/C++ [87] AIPC [89] Harness Engineering [88]	AutoKernel [90]	AZR [99] VOYAGER [84]	Metis [62] HDSO [106] Tax AI [92]	STOP [15] AIDE ² [94]
	D2. Formal Verification	—	—	STP [100] PSV [101]	—	—
	D3. Reward Model / LLM-as-a-Judge	Nemotron-4 [97] REPO [102] HealthBench [107]	—	SIMA 2 [24] SEAgent [61]	ReasoningBank [85] DecoEvo [80]	RQGM [21]
	D4. Regression Testing / Model Selection	Agent-Agnostic C/C++ [87] Harness Engineering [88]	GEPA [82] AutoResearch [108] AutoKernel [90]	—	HarnessDev [91] ASPIRE [109] HDSO [106]	STOP [15] RQGM [21] AIDE ² [94]
Online & Meta-Level Adaptation	E1. Online / Continual Learning	—	—	SEAgent [61] VOYAGER [84]	ReasoningBank [85] PANDO [25] Dynamic Cheatsheet [110]	—
	E2. Experience Replay / Online Memory Update	—	—	—	ReasoningBank [85] PANDO [25] Metis [62]	—
	E3. Self-Modifying Code / Policy	—	ADAS [66] AFlow [67] AutoResearch [108]	—	HarnessDev [91] ASPIRE [109] SHAPER [111]	STOP [15] Gödel Agent [19] A-Evolve-Training [17]
	E4. Adaptive / Co-Evolving Evaluator	—	—	—	DecoEvo [80]	RQGM [21]

阅读指南。行将具体技术归类为更广泛的技术家族；列表示所引文献中考察的具体改进环的自主性等级。当某项研究采用多种技术或包含不同的改进环时，该研究可能出现在多个单元中。每个单元最多列出三项代表性研究。破折号（—）表示此处未列出代表性示例；它并不意味着该技术与该等级不兼容。
层级简写。 L1: 改进执行 · L2: 改进策略 · L3: 学习信号 / 经验获取 · L4: 环境适应 · L5: 递归继承（元改进）。

例如，一个编码智能体可能会利用人类提供的测试套件，在固定五次尝试次数的限制下反复修改代码。如果修复措施和反馈仅限于当前任务，且不影响该智能体处理后续任务的方式，则它仍处于 B0 状态。同样，一个写作智能体可能会根据人类提供的评分标准反复修改文章，但不会保留此次修改的经验以用于未来的写作。在这两种情况下，人类定义评估规则和修改流程，而人工智能仅在这些约束条件下改进当前输出。

在固定规则下，B0 可以在单次会话中提升输出效果。例如，Self-Refine 通过在同一会话内进行生成-反馈-优化的环路来改进输出 [112]；Reflexion 将失败尝试的言语反思写入上下文缓冲区，供后续尝试使用 [113]；Thoughts 的树状结构将推理组织为对候选思维分支的搜索 [114]。

然而，B0 无法在任务间累积知识。迭代轨迹和修正信号仅存在于当前会话中；任务结束后它们会被丢弃，系统无法将碎片化的经验整合为长期能力。APEX-EM 指出，基于大语言模型的智能体通常缺乏持久的过程记忆：即使解决了相同任务，它们仍需从头重新推导解决方案 [115]。具体而言，存在四个主要局限。

- **经验不会在任务间累积。** 中间结果、批评和修正仅保留在临时任务上下文中，而非成为模型参数、可重用内存或操作规则的更新 [116]。因此，系统可能在一个任务中纠正了错误，但在下一个任务中又重复该错误。增加迭代次数也无法解决这种缺乏持久学习的问题。
- **改进过程仍由人类定义。** AI 可以生成、验证和修改输出，但这一过程如何运作仍由人类决定 [112]。例如，一个编码智能体可以修复测试失败的代码，但它不会持续改进未来的测试流程。该环路围绕当前输出闭合；升级系统改进机制的责任仍然在于人类。工具使用、仿真和现实世界反馈本身并不会改变这一界限。
- **自生成的反馈可能强化错误。** 当同一模型生成并评估答案时，两个阶段可能共享相同的知识盲区。自校正基准将验证过程，特别是定位首个错误推理步骤，识别为关键瓶颈 [117, 118]。在缺乏可靠外部反馈的情况下，修订甚至可能将正确答案变为错误答案 [119]。因此，反复批判并不能保证成功修正。
- **在固定评估条件下，增加迭代次数带来的收益有限。** 在没有新信息的情况下反复修改答案可能带来很少的额外收益 [120]；在某些情景下，独立采样或外部验证可能更为有效 [121, 122]。固定的验证者也可能无法发现部分错误 [123]。例如，反复修改代码以通过不完整的测试套件，可能会提升测试表现，但未被测试的缺陷仍无法解决。

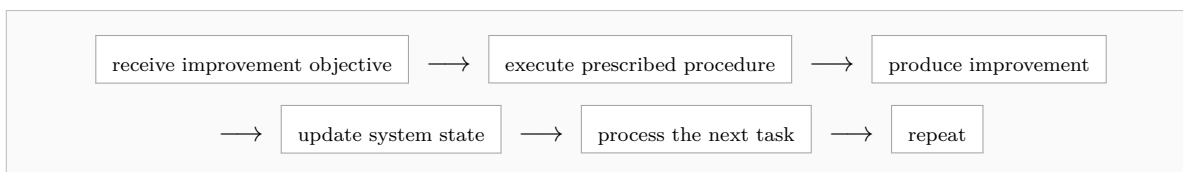
3.2 L1: 改进执行的自主权

在 L1 阶段，AI 系统执行由人类定义的改进流程，其被接受的结果将被保留并在后续任务或改进轮次中重复使用。人类指定目标、更新流程和接受标准，而 AI 则执行规定的改进步骤。与 B0 不同，该环路会改变指定 AI 系统中的持久状态，而不仅仅是优化当前任务输出。例如，一个编码智能体可遵循人类定义的流程，根据测试反馈修订代码，将经过验证的修复方案保存为可复用规则，并自动应用于后续独立任务。这种模式已在生产基础设施中显现。Meta 的容量效率系统将工程师的调试专长编码为可复用的修复技能，并通过预定义流程应用这些技能，以诊断性能回归并执行修复任务。Meta 报告称，该工作流将数小时的手动回归调查压缩至几分钟内完成，生成的拉取请求经过标准审查与集成，从而在生产系统中实现持久改进 [5]。

该层次的特征环可概括如下：

Table 3 按人工智能开发流水线层级组织的代表性 L1 系统。L1 系统执行由人工定义的改进流程，并生成可持久化的成果，进入后续 workflow。

Method	Type	Key Mechanism	Level Limitation
Data Level			
Google High-Fidelity Label Curation [124]	Data curation	LLM labeling + boundary selection	Human-defined labeling rules
Phi-4-reasoning [125]	Training data selection	LLM evaluation + boundary filtering	Human-defined selection criteria
FineWeb-Edu [22]	Data filtering	LLM scoring + classifier filtering	Human-defined quality criteria
NVIDIA NeMo Curator [126]	Data curation	Configurable filtering pipeline	Human-configured processing rules
Data-Juicer [127]	Data processing	Composable processing operators	Human-defined processing recipe
Microsoft SynthLLM [95]	Synthetic data generation	Reference-guided data synthesis	Human-defined generation procedure
SynthAgent [96]	Supervision generation	Task + trajectory synthesis	Human-defined generation workflow
NVIDIA Nemotron-4 [97]	Synthetic supervision	Candidate generation + reward filtering	Human-defined quality dimensions
Training-Method Level			
EDIT [81]	Training signal refinement	Diagnosed step revision	Human-defined diagnostic criteria
REPO [102]	Post-training procedure	SOP-guided interaction + reward evaluation	Human-defined behavioral procedure
Training-Platform Level			
Agent-Agnostic C/C++ Optimization [87]	Training execution optimization	Procedural performance optimization loop	Human-defined optimization workflow
Meta NCCL Agentic Debugging [128]	Failure diagnosis and recovery	Runbook-guided distributed debugging	Human-defined debugging procedure
Evaluation-and-Safety Level			
OpenAI HealthBench [107]	Model evaluation	Expert-rubric-based automated scoring	Human-defined evaluation rubrics
Deployment-Optimization Level			
AIPC [89]	Model deployment	Skill-guided deployment adaptation	Human-defined deployment procedure
Application-System Level			
Agent Toolkit for AWS [129]	Model-application integration	Bedrock skill-guided integration	Human-defined integration procedures
LinkedIn CAPT [130]	Product engineering	Step-by-step engineering playbooks	Human-authored implementation procedures
OpenAI Harness Engineering [88]	Product development	Constraint-guided implementation	Human-defined architecture and delivery rules



尽管结构简单，要可靠地执行此环仍面临多个核心挑战。首先，规定的流程必须涵盖执行过程中出现的各种情况。当任务遇到预定义流程未涉及的条件时，智能体将缺乏适用的下一步动作，无法自主修改流程以解决该问题。其次，执行过程必须在多步骤 workflow 中保持可靠性，因为一次错误的诊断、变换或中间决策可能影响所有后续动作。第三，由于改进结果会被写回系统状态并在后续任务中重复使用，因此在保留之前必须验证其有效性；否则，执行错误将持续存在，并影响后续开发周期。这些挑战贯穿整个 AI 开发流水线，但其具体表现形式取决于改进流程执行所处的阶段。

具体而言，大规模人工智能在实际场景中的开发通常被组织为几个不同的层级：(1) 数据层级负责训练语料库的构建与整理；(2) 训练方法层级决定模型的学习方式；(3) 训练平台层级管理大规模训练的执行；(4) 评估与安全层级评估模型的能力与安全性；(5) 部署与最优化层级实现模型的高效服务；以及 (6) 应用系统层级将基础模型能力集成到下游产品和应用系统中。下文各节将探讨人工智能系统如何在每个层级内自主执行预定义的改进流程，以及所产生的成果如何流入开发 workflow 的下游阶段。表 3 总结了这些层级中讨论的代表性系统。

3.2.1 数据层

在数据层面，人工智能越来越多地接管了以往需要大量人工操作的大规模数据生成与整理流程。现有方法主要分为两类：第一类侧重于数据清洗和质量过滤，第二类侧重于合成数据与监督信号生成，其中人类定义目标任务、期望的数据特征以及验证流程。

- (1) 数据清洗与质量过滤。数据层面的一个主要挑战是规模：训练语料库和候选样本的体量远远超过了在样本层面进行人工审查的容量。因此，数据监管正从逐样本的人工判断转向一种新范式，即由人类定义质量标准和处理流程，而由人工智能在大规模上执行这些流程。在谷歌高保真标签监管流水线中，

开发人员首先定义目标任务及其决策标准，例如何为标题党。随后，大语言模型（LLM）应用这些标准对候选数据分配初步标签，接着通过预定义的聚类和边界样本选择程序，识别出一小部分具有信息量的样本供专家标注 [124]。在 Phi-4 推理模型的训练数据构建中也出现了类似的分工：研究人员指定期望的难度水平和推理特征，以及相应的评估流程，而基于大语言模型的评估流水线则应用这些标准，筛选出接近模型能力边界的样本 [125]。FineWeb-Edu 将这一模式扩展至大规模网络内容过滤：研究人员定义教育质量标准，使用大语言模型生成质量标签，并训练分类器以大规模应用这些标准 [22]。随着这些数据处理操作逐渐标准化，它们也可以被整合进可复用的流水线中。NVIDIA NeMo Curator 将质量过滤、分类和去重组织为可配置模块，根据开发人员设定的规则和参数执行 [126]。Data-Juicer 采用类似方法，但将数据处理操作抽象为可组合的算子，使得预定义的处理配方可在大规模语料上执行 [127]。

- (2) 合成数据与监督信号生成。训练数据可通过合成生成进行扩展。对于复杂任务的高质量示范通常人工构建成本较高，这促使采用可重复的数据生成流程，使模型能够大规模执行。Microsoft SynthLLM 采用此方法，从高质量的网络内容出发，通过预定义的多阶段流程组织参考概念，并利用大模型生成多样化的提示及对应响应作为合成训练样本 [95]。类似的流程也可生成更丰富的监督形式。SynthAgent 将任务构建与交互轨迹组织成预定义流水线，其中模型执行网络探索、任务生成和轨迹优化，以生成用于训练网络智能体的监督数据 [96]。合成生成可与预定义的质量评估相结合，构建更高品质的监督数据。NVIDIA Nemotron-4 使用指令模型生成候选响应，再通过奖励模型根据预定义的质量维度对响应进行评分和筛选，最终得到的合成样本用于后续模型训练 [97]。

3.2.2 训练方法等级

在训练方法层面，人工智能正越来越多地接管此前由研究人员和工程师手动协调的结构化学习与后训练流程。现有方法主要将诊断、修订、评估和最优化步骤编码为预定义的工作流，其中人类指定学习目标、反馈信号和更新规则，而人工智能系统则执行这些流程，生成改进的训练信号并支持后续的最优化。

已建立的训练实践可以被编码为明确的改进流程，并由人工智能系统在后训练阶段执行，从而改善模型接收训练信号的方式，进而提升学习效果。EDIT 将此过程组织为两阶段训练程序。研究人员首先指定诊断信号和修订标准，随后系统识别出有问题的推理步骤，大语言模型根据评分清单仅对受影响的部分进行修正。修正后的输出随后用于进一步的强化学习校准 [81]。REPO 同样将多阶段标准操作流程和行为规范融入训练过程。在多轮交互中，模型遵循规定的步骤，而基于大语言模型的评判者则根据预设标准评估流程合规性，并将评估结果转化为奖励信号，用于后续策略最优化 [102]。

3.2.3 训练平台层级

在训练平台层面，人工智能正越来越多地接管维持大规模训练高效可靠所需的基础设施工程流程的执行工作。现有方法主要分为两类：第一类聚焦于训练执行与性能最优化，其中智能体分析工作负载并应用预定义的最优化流程；第二类聚焦于故障诊断与恢复，其中智能体执行从工程知识和运维经验中提炼出的结构化调试与修复工作流。

- (1) 训练执行与最优化。性能工程实践可以被编码为明确的最优化过程，由智能体在具体工作负载上执行。无智能体依赖的端到端 C/C++ 应用程序性能最优化遵循此模式，通过将完整的最优化控制环存储于过程记忆中。智能体执行运行时分析，识别性能热点，生成候选代码修改，并根据预定义的工作流

验证其正确性和性能，对于无效的修改会在进一步最优化前回滚 [87]。

- (2) 训练失败诊断与恢复。分布式训练失败同样可以通过基于积累的工程经验构建的结构化调试流程来处理。Meta 和 PyTorch 社区对 NCCL 监控超时的主要根本原因进行了分类，并将这些发现提炼为实用的决策树和调试手册。随后，智能体通过在各秩之间对齐证据、追踪集合操作序列，并将运行时迹线与对应的代码路径关联起来，执行此结构化工作流，以定位最早的可操作发散点 [128]。

3.2.4 评估与安全等级

在评估与安全层面，人工智能正越来越多地接管将专家定义的要求转化为可扩展、可重复评估的模型评估流程的执行工作。现有方法利用人工智能系统对大量模型输出应用预设的评分标准、测试规程和风险准则，而人类仍负责定义评估目标、评估标准和接受阈值。

模型评估将专家判断转化为可重复的测试与评分流程。OpenAI HealthBench 邀请医疗专家为特定医疗对话定义详细的评估准则，明确理想回复应包含或避免的内容，以及各项准则的相对重要性。GPT-4.1 随后依据这些预设准则对候选模型回复进行评分，并计算最终评估结果 [107]。类似的评估流程也可应用于鲁棒性、安全性和风险评估，为后续模型改进和发布决策提供标准化反馈。

3.2.5 部署最优化级别

在部署最优化层面，人工智能正越来越多地接管将训练好的模型适配到具体硬件和运行环境的执行过程。现有方法将部署专业知识编码为标准化的工作流，涵盖模型转换、兼容性解决、量化、运行时配置和验证等步骤：人类指定目标平台和部署约束，而人工智能系统则执行相应的适配与验证步骤。

将训练好的模型部署到目标硬件通常需要针对模型转换和运行时适配的专用流程。AIPC 将这一部署专业知识编码为由智能体技能和分阶段验证支持的标准、可验证的步骤。给定一个训练好的模型和目标高通 AI 运行时环境，智能体执行预定义的部署流程，以转换模型、处理部署不兼容性、执行量化校准，并验证最终的实现结果。该过程生成可用于下游推理的可运行部署产物 [89]。

3.2.6 应用系统层面

在应用系统层面，人工智能正越来越多地接管将基础模型集成到实际产品和服务中所需软件工程流程的执行工作。现有的方法将开发知识编码为可重用的技能、操作手册和工程工作流，使人工智能系统能够完成服务集成、接口实现、代码修改、测试和验证等任务，而人类则继续负责指定产品需求、系统架构以及开发过程中的约束条件。

将基础模型能力转化为实际产品，需要通过可重复的工程流程将模型服务和接口集成到应用系统中。AWS 的 Agent Toolkit 将生成式 AI 开发实践编码为可复用的智能体技能。其 Amazon Bedrock 技能提供了将模型能力集成到应用中的预定义流程。AI 编码智能体可以执行这些流程，以将基础模型服务与下游产品接口及应用组件连接起来 [129]。LinkedIn 的上下文智能体手册与工具 (CAPT) 同样将积累的组织知识编码为逐步操作手册，使 AI 编码智能体能够在真实代码库中执行具体的工程任务（例如服务创建、接口扩展和代码维护） [130]。OpenAI Harness Engineering 将这一模式扩展至端到端软件产品开发。工程师定义产品意图、系统架构以及持续集成、代码合并和回滚的规则，而 Codex 在这些预定义约束下执行实现与验证，并持续生成进入后续开发工作流的代码制品 [88]。

3.2.7 评估 L1 执行自主性

在这些层级中，L1 系统共享相同的根本能力边界：人工智能可以执行越来越长且后果重大的改进程序，且产生的成果可能延续到后续阶段，但指导改进的程序仍由外部指定。因此，对 L1 的评估应考虑两个相互关联的属性：执行可靠性，即人工智能在不同条件下能否正确执行预定程序；以及持续安全性，即执行过程中引入的错误是否能在其传播到后续改进轮次之前被检测到。

预设改进流程的可靠性是评估此级别执行自主性的重要因素。L1 系统能够完成一系列较长的动作，且人类干预有限，但其执行仍受预先设定流程的限制。当任务条件超出这些流程涵盖的范围，或工作流中隐含的假设不再成立时，系统无法独立修改改进流程，因而容易导致执行失败。

持久性进一步加剧了此类故障的后果。执行过程中产生的输出可以被保留并在后续开发阶段重复使用，导致错误在引入它们的任务之外传播，并影响后续的迭代。因此，评估不仅应检查预定义流程是否正确执行，还应检查生成的成果在进入下游工作流之前是否仍然有效。

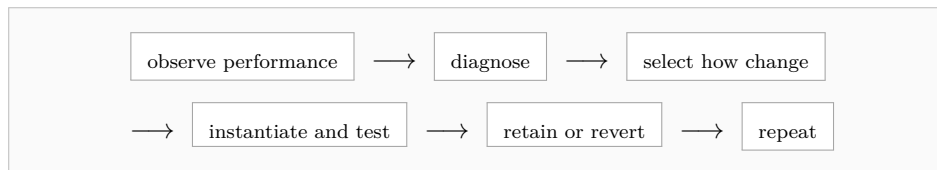
Finding: L1 executes human-defined improvement procedures at scale. Humans first encode established engineering experience into explicit improvement steps and validation rules, while AI systems execute these predefined procedures on concrete tasks. The resulting artifacts are retained and incorporated into later development workflows, allowing the same improvement procedures to be repeatedly applied across independent tasks and development stages.

3.3 L2：改进策略的自主权

在 L2 阶段，人工智能系统利用评估反馈来选择下一个尝试的改进干预措施，而不是简单地执行预定的更新。它提出并测试对目标的修改，保留被接受的修订，而人类则持续指定目标、任务边界和接受标准。周围的搜索框架可以保持不变；自主性体现在如何确定目标应如何改进。在许多人工智能开发工作流中，执行一个提议的更改相对机械；困难的部分在于决定哪个更改值得尝试。研究人员必须解释失败原因，推断是哪个组件限制了进展，选择效果不确定的干预措施，并在观察结果后决定下一步尝试什么。随着可能干预措施空间的增长，这种实验决策成为主要瓶颈：穷尽搜索不可行，而手动实验又受限于研究者的注意力和先验直觉。L2 系统自动化了这一瓶颈。与仅仅执行人类指定的修改不同，改进者 会利用评估结果和实验证据来确定下一个候选干预措施。

这一转移越来越类似于对实证研究者角色的部分自动化。例如，OpenAI 将当前前沿智能体描述为在人类监督下，针对定义明确的研究任务已达到“自动化研究实习生”的阶段，而研究人员则继续设定优先级并决定哪些结果值得进一步投入或部署 [131]。因此，相关的自主性并非对研究的无限制控制，而是特定研究职责的转移：将观察到的证据转化为下一个实验。

该层次的特征环可概括如下：



被改进的对象可能保持较小，如提示词最优化，也可能扩展为可执行的智能体，最终发展为模型训练程序。在这些情景中，改变的是改进者可干预的对象和范围，以及用于决定尝试何种修改的证据。因此，

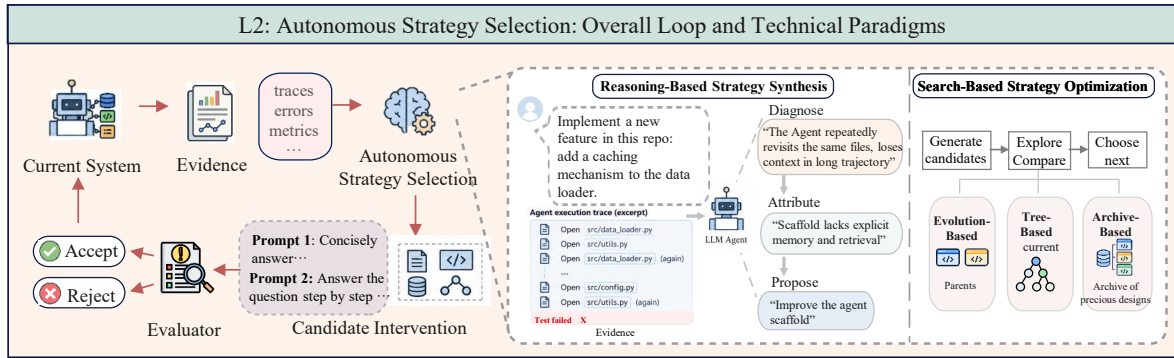


Figure 4 L2: 自主策略选择概述。在人类定义的改进目标和评估标准下，AI 系统利用当前系统的证据诊断其薄弱环节，并自主决定如何进行改进。候选干预措施经过评估，被接受的更新将被纳入后续迭代中。代表性机制包括基于推理的策略合成和基于搜索的策略最优化。

我们按照改进策略所搜索的对象来组织 L2，而非按具体系统划分。图 4 展示了这一策略自主性机制，其中关于当前系统的证据指导着对如何改进它的自主选择与评估。

3.3.1 提示搜索

智能体 improver 决定如何根据执行迹和评估结果来修改指导模型或智能体的指令。在最狭义的 L2 情景中，系统决定如何修订提示词或文本控制变量，而不仅仅是实现预设的编辑。例如，Dropbox 使用 GEPA 重写了指示大语言模型判断检索到的文件或消息是否与用户搜索相关性的提示词 [132]。然而，早期的进化方法在多种变异方案中进行搜索，而更近期的反思型优化器则利用执行迹识别故障模式并生成针对性的修订。GEPA 将故障归因于特定模块，并保留互补的提示变体，而非立即选择单一的贪婪演化路径 [82]。这种基于反馈的搜索并不局限于单一优化器设计。MPO 将可编辑范围扩展至多模态提示，利用评估所得的语义反馈引导后续的文本与非文本提示候选，而 C-Evolve 则将候选生成嵌入固定的进化协议中。在这些情景中，搜索机制可外部指定，而具体的提示修订则由评估反馈自然产生。[79, 83]。因此，自主性的相关提升并非提示生成本身，而是对下一步尝试何种修订策略的控制权。

3.3.2 智能体与支架搜索

改进可以涉及重新设计推理与工具使用组件之间的交互方式。当可编辑对象从提示词扩展为可执行的智能体代码后，策略搜索开始类似于自动化系统设计。例如，Microsoft Foundry 的智能体优化器会根据评估失败情况，重写托管智能体的系统指令、技能以及本地函数工具描述 [133]。自动化智能体系统设计 (ADAS) 将智能体形式化为一个 Python forward 函数，并使用元智能体不断填充不断增长的候选智能体档案库 [66]。在每次迭代中，元智能体会在验证数据上评估候选智能体，并将其代码和指标一同存入档案库。由于这些决策以可执行的封装代码形式编码，单次修改即可改变智能体在整个任务中的行为，而不仅仅是更改一条指令。

最近的工作流搜索系统通过设计空间的不同表示来实现这一思想。AFlow 将工作流编码为可执行的图，并对代码级别的连续修改应用蒙特卡罗树搜索，使用执行结果作为后续分支的反馈 [67]。AgentSquare 则将智能体设计分解为可重用的模块，并通过演化与重组进行搜索，使得在一种配置中发现的成功结构可成为另一种配置的构建块 [74]。

由此产生的自主性远高于本地参数调参，同时在重要意义上仍保持有限。改进者在不同的智能体设计中进行选择，但候选方案仍因在外维护的评估过程中表现更优而被推荐。L2 自主性关注的是寻找更优的系统配置，而非拥有重新定义成功表现标准的权限。

3.3.3 模型与训练搜索

The improver 将经验训练证据转化为对学习器自身配置、结构或优化方式的决策。因此，人类研究人员在很大程度上依赖积累的经验直觉来决定应执行哪些少量可能的实验。新兴的 L2 改进器的作用是压缩这一实验周期：检查先前运行的结果，推断改进最可能来自何处，并据此分配下一次实验。当前系统越来越多地在持久配置、模型结构以及学习过程本身中进行搜索。

- (1) 配置搜索。 它自动化了昂贵训练试验的分配。近期的 Agentic 方法越来越多地通过关于先前实验的语义推理来增强传统搜索。系统不再将每次试验视为孤立的黑箱查询，而是能够解释训练结果，并利用这些结果来集中后续的探索。例如，NVIDIA 在 2026 年提出的用于后训练 Cosmos 3 的 TAO 工作流，允许一个编码智能体在固定验证目标下调用由大语言模型引导的 AutoML [134]。人类提供模型、任务、数据和期望指标；系统则接管了大部分重复性决策，即决定下一步应评估哪种配置。
- (2) 架构搜索。 它将模型的结构设计任务委派给系统，降低了对人工设计搜索空间的依赖。基于大语言模型的改进器能够解析实验结果，并提出此前未预先枚举的结构化修改建议。AgentNAS 的出发点在于观察到现代神经网络架构搜索（NAS）仍然依赖于人工设计的、特定任务的搜索空间。该方法利用大语言模型生成一个特定任务的初始架构，然后从该设计中推导出一个结构化的搜索空间，以供后续的组合搜索使用 [11]。
- (3) 训练规则搜索。 改进器 通过改变模型参数的学成过程来修改可执行的程序。当前前沿智能体正开始自动化从假设到实验这一环节。OpenAI 对 GPT-6 Astra 的 NanoGPT 评估中，为智能体设定固定的验证目标、一个 H100 显卡以及受限的训练设置，但要求智能体自行诊断训练瓶颈、修改训练代码、调整其配置，并对训练环做出有效改动 [135]。开放的研究环境更直接地展现了同样的分工模式。在 `autoresearch` 中，数据流水线、评估函数、指标以及每次实验的计算预算保持不变，而智能体反复编辑训练程序，并仅在保护的验证指标提升时才保留修改 [108]。
- (4) 系统与实现搜索。 改进者 搜索模型计算的更高效实现方式，同时通过外部测试保证正确性和系统级性能。人工智能可以分析瓶颈、提出实现变更、验证数值正确性、测量实际硬件性能，并利用结果决定下一步的修改内容。AutoKernel 从性能分析开始，而非随意生成代码：它识别出对端到端性能影响最大的操作，并引导智能体的实验聚焦于这些瓶颈。只有在通过正确性检查并测得 GPU 加速效果后，才会接受候选的 Triton 或 CUDA 实现 [90]。

表 4 通过其搜索目标、机制以及外部固定的限制，对比了代表性系统。

3.3.4 评估二语策略自主性

对改进策略的更大自主权本身并不意味着更强的递归自我改进能力。一个 L2 系统可能具有非常丰富的干预空间，但仍然以低效的方式搜索该空间，利用其评估器的弱点，或在几次迭代后使收益消失。因此，我们将委托自主的程度与由此产生的改进过程的质量分离开来。

这些维度是互补的：只有在反馈能够区分候选方案且搜索过程能高效探索该空间时，更广阔的搜索空间

Table 4 按改进策略搜索目标分组的代表性（而非全面性）L2 系统。所有列出的系统均保持在 L2 级别，因为它们自主选择如何改进系统，而目标、评价准则或接受规则仍由外部指定。

Method	Type	Key mechanism	Level limitation
Prompt Search			
Promptbreeder [78]	Prompt search	Prompt and mutation co-evolution	Fixed task fitness
GEPA [82]	Prompt search	Trace reflection + Pareto selection	Fixed task metric
MPO [83]	Prompt search (multimodal)	Evaluation-derived semantic feedback guides textual and non-textual prompt candidates	Fixed task metric
C-Evolve [79]	Prompt search	Candidate generation inside a fixed evolutionary protocol	Fixed protocol + task metric
Agent and Harness Search			
Microsoft Foundry Agent Optimizer [133]	Agent search	Rewrites system instructions, skills, and local function-tool descriptions from evaluation failures	Fixed evaluation process
ADAS [66]	Agent search	Meta-agent coding + design archive	Fixed benchmark
AFlow [67]	Workflow search	MCTS over executable workflows	Fixed evaluator
AgentSquare [74]	Agent search	Module evolution + recombination	Fixed benchmark
Autonomous Experimental Search			
AutoResearch [108]	ML experiment	Edit-train-evaluate loop	Fixed metric
GPT-6 Astra NanoGPT [135]	Training-rule search	Agent diagnoses bottlenecks, edits training code, tunes configuration and training loop	Fixed validation target + compute
Auto Research [136]	ML experiment	Specialist agents + shared lineage	External evaluator
AgentNAS [11]	Architecture search	LLM-generated task-specific seed architecture + derived structured search space	Fixed validation metric
AutoKernel [90]	Kernel experiment	Profile-rewrite-benchmark loop	Fixed correctness + speed

才有用。这种组合有助于解释为何当前工业系统集中于编码、模型训练和内核优化，因为在这些领域候选干预措施可执行，且能快速获得反馈。

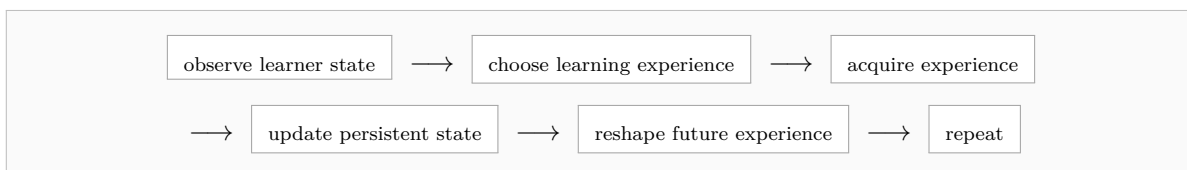
相同的自适应搜索环也会带来特有的评估风险。重复访问开发集会引发基准过拟合，额外的搜索计算可能被误认为是算法改进，而大语言模型评估者也可能与提出者共享盲点。公开的自动化研究系统使这些风险变得具体：报告的行为包括随机种子的选择性使用、捷径发现，以及通过反复评估查询尝试提取测试标签 [20]。一旦基准被自适应地查询，它实际上就成为了最优化曲面的一部分，而非被动的测量工具。公司报告的结果仍可作为可行性与规模的证据，但其来源必须明确，并且不应被视为与独立复现实验等价。

Finding: L2 shifts human effort from proposing individual improvements to constraining the search for improvements. The human still specifies the objective and acceptance criterion, but no longer needs to choose each candidate intervention. This changes the bottleneck from executing a known improvement to searching over possible improvements.

3.4 L3: 未来学习体验的自主性

L3 增加了对接下来要获取的学习体验的自主性。与 L2 关注如何改进的决策不同，L3 所考察的额外自主性关注的是学习议程：系统利用对其当前能力、失败情况以及学习历史的证据，来决定下一步应该学习什么。学习随后会改变后续经验获取所依赖的状态，从而在各轮次之间连接起经验选择和持续改进。

该层次的特征环可概括如下：



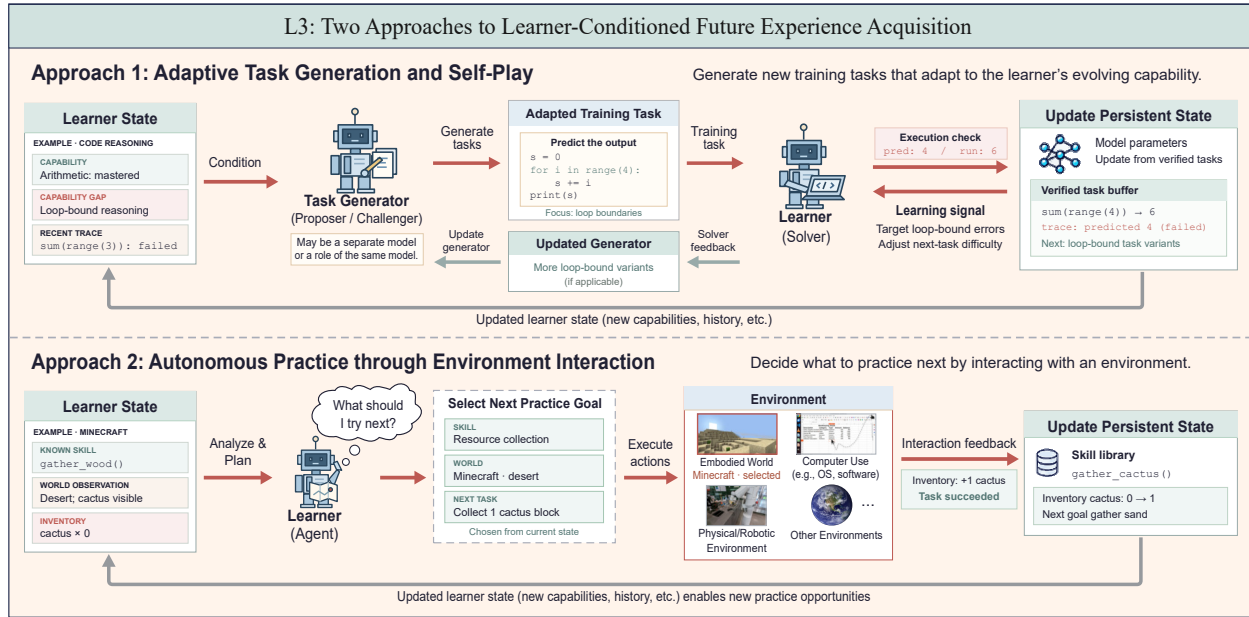


Figure 5 L3 概述：学习器条件化的经验获取。在人类定义的目标与评估约束下，系统利用其当前能力、失败情况以及交互历史，决定下一步应学习的内容。**顶部：自适应任务生成与自我对弈。**任务生成器针对学习器的薄弱环节提出训练任务，例如通过代码推理任务来解决循环边界错误。基于执行的反馈支持学习器的持续更新，并在适用情况下优化生成器。**底部：通过环境交互实现自主练习。**智能体根据当前技能和环境观测选择练习目标，例如在 Minecraft 中收集仙人掌，并将成功的经验整合为可复用的技能库。在两种方法中，持续的更新重塑了后续的任务生成或练习选择，从而在学习与未来经验获取之间形成闭环反馈。

其定义特性为学习器条件化的未来经验获取。关于不断演化的学习器的证据必须指导关于选择、生成或寻求后续学习经验的决策，且由此产生的持续更新必须反馈到后续的经验获取决策中。该决策可由独立的课程组件实现，也可整合到智能体的策略中。仅偶然改变访问状态的策略更新，并不能单独体现对学习议程的自主性。

经验无需从零生成。当选择机制能够适应学习器并参与持续的循环时，反复从外部提供的资源池中选取材料可以表现出这一特性；而一次性过滤或保留则不具备此特征。同样，目标、评估者、更新过程以及经验选择规则可以由人类设计。关键区别在于，学习器的反馈是否能自主改变下一阶段的学习议程，而无需在每轮之后由人类重新设计。持续学习可能存在于模型参数或可重用的外部状态中，包括记忆和技能。

以下示例探讨了该经验-自主机制在特定改进环中的表现。图 5 展示了这一反馈结构的两种代表性实现：自适应任务生成与自我对弈，以及通过环境交互实现的自主练习。在确定整体系统层级时，应结合层次结构中的其他要求来解读该机制的证据。

3.4.1 从工业自动化到体验自主

现代的训练数据流水线自动化了许多数据操作，但仅实现执行自动化并不能建立经验自主性。基于此处回顾的公开文档实践 [137–140]，我们将重复出现的功能归纳为六个组成部分：数据源准备、数据标签、数据选择、数据构建、数据混合与训练编排，以及模型反馈诊断。这些功能无需形成固定顺序；它们可在预训练、监督微调、偏好最优化和强化学习中反复出现。

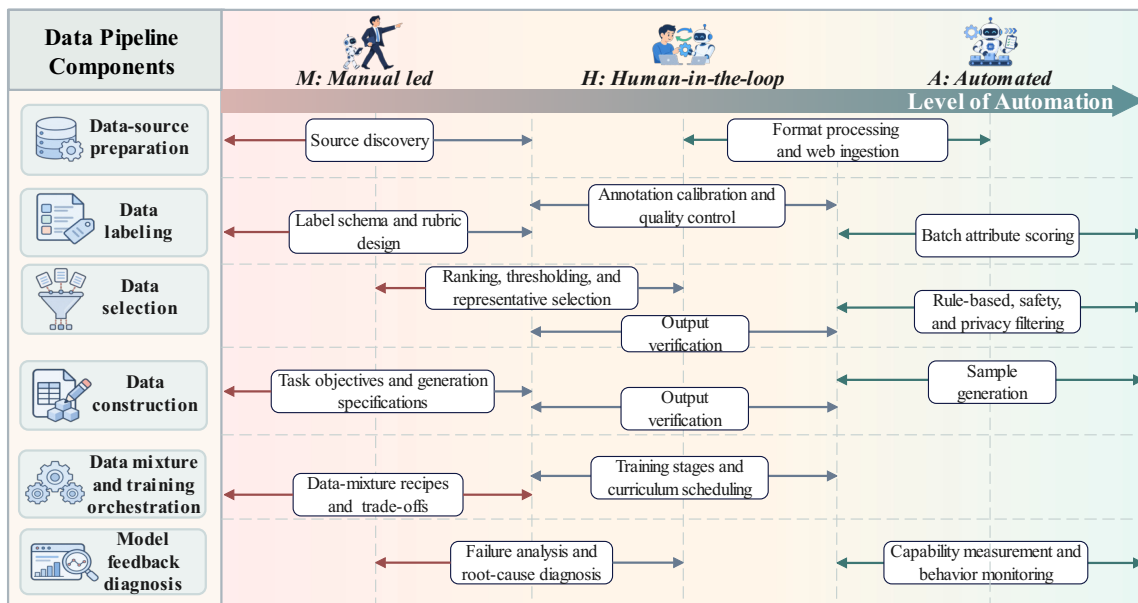


Figure 6 此处回顾的训练数据流水线中自动化示意图。行表示六个功能组件，圆角条形代表重复的操作。水平位置表示从人工主导（M）、经人机协同（H）到自动化的定性进展。位置和跨度是对现有实践的解释性综合，并非定量测量或组织评分。操作自动化与学习器反馈自主重塑各学习轮次中经验获取这一额外要求是不同的。

图 6 以示意图形式总结了这一研究格局。一旦明确了输入数据和接受标准，诸如数据摄入、过滤和样本生成等操作即可大规模执行 [22, 137, 139]。评审报告还描述了研究人员在数据策略、生成与混合策略、训练阶段以及实验评估方面的自主选择 [22, 138, 141]。这些选择体现了自动化操作与将后续学习议程的决策权委派之间的区别。

人工设计的规则并不排除 L3 特定机制的存在。若固定的选择算法能够利用学习器状态的变化反馈来选择后续经验，仍可生成自适应的学习议程。相反，一个反复生成并过滤数据的自动化流水线，仅通过产生新样本并不能建立经验自主性。决定性的证据在于学习器状态、经验获取、持续学习以及后续获取决策之间存在闭环依赖关系。此处引用的流水线描述虽然记录了大量操作自动化，但本身并不足以确立这种完整的依赖关系 [138–140]。这种证据上的区分并不意味着工业系统必然缺乏此类机制。

以下研究实例展示了两种反复出现的模式：自适应任务生成与自我对弈，以及通过环境交互实现自主练习。它们是互补的组织视角，而非穷尽或相互排斥的类别：一个交互式智能体也可能生成自己的任务，且学习器条件选择可以在任一模式中进行，或作用于现有数据池之上。

表 5 按照其改进目标、学习器条件机制和外部约束，对比了具有代表性的环。

3.4.2 自适应任务生成与自我对弈

训练任务的价值随着学习器的改进而变化。固定的任务分布并未显式追踪这一不断演进的能力边界：熟悉的任务可能变得无信息，而远超当前能力的任务则可能提供很少可用的学习信号。自适应生成通过使用依赖于学习器的信号来估计哪些任务可能有助于进一步提升，从而解决此问题。核心挑战在于获取一个有信息量的信号，将其转化为有用的任务分布，并保持所得经验的有效性。

Table 5 代表性改进环展示了学习器条件化的未来经验获取。约束描述了所讨论环的范围，而非独立确定系统的最大自主性水平。

Method	Target	Learner-conditioned mechanism	Externally specified constraints
Adaptive Task Generation and Self-Play			
SSP [98]	Search agent	Solver performance shapes proposer rewards and subsequent search tasks.	Reward design and training procedure.
AZR [99]	Reasoning model	Solver-dependent learnability rewards guide executable task proposal.	Task formats, code executor, and update rules.
R-Zero [103]	Reasoning model	Solver answer consistency supplies a proxy that shapes Challenger tasks.	Uncertainty reward, pseudo-label scheme, and optimization.
STP [100]	Theorem prover	Conjectures near the current prover's frontier train the conjecturer.	Formal domain, proof checking, and training rules.
PSV [101]	Coding model	Solver-derived difficulty labels condition specification generation.	Formal verifier, difficulty scheme, and training recipe.
VisPlay [142]	Vision-language model	Reasoner answer consistency and diversity rewards guide visual questions.	Image source, reward design, and optimization.
Autonomous Practice through Environment Interaction			
VOYAGER [84]	Embodied agent	Agent state and exploration history guide objectives; learned skills persist.	Minecraft interface, curriculum prompts, and skill representation.
SIMA 2 [24]	Embodied agent	In the full ASKA setup, evaluation feedback directs practice toward weaker skills.	Task setter, reward rubric, and training procedure.
SEAgent [61]	Computer-use agent	Trajectory feedback updates a software guidebook that conditions later tasks.	Software interfaces, assessment model, and learning procedure.

搜索自对弈（SSP）[98] 通过使提议者的奖励依赖于当前求解器的表现，将前两个挑战连接起来。随着求解器的变化，未来搜索问题的激励机制也随之改变，从而使得课程能够自动演化，而无需手动重新设计每个任务批量。

绝对零点推理器（AZR）[99] 将可执行推理任务的提议与求解耦合在一个模型中。其基于求解器的可学习性奖励指导任务的提议，同时代码执行器验证所提议的任务并检查解决方案。这分离了不受约束合成可能混淆的两个要求：经验必须可评估，且其难度必须对当前学习器有用。

R-Zero [103] 采用挑战者-求解器架构，在无需外部提供答案标签的情况下适应任务生成。挑战者的不确定性奖励取决于当前求解器多次响应的一致性；因此，对求解器的更新会改变决定后续任务的信号。这种基于一致性的量可作为模型感知难度的代理，而非答案正确性或未来学习收益的直接证据。它在环路中的作用是使经验获取能够响应学习器，即使在缺乏更强正确性信号的情况下也是如此。

形式化领域为可靠的反馈提供了另一种途径。STP [100] 通过联合开发一个猜想生成器和一个证明器，解决了稀疏的证明奖励问题。当前证明器仅能勉强证明的生成猜想为猜想生成器提供了训练材料，而经过验证的证明则提升了证明器的能力。因此，证明器不断变化的前沿重新塑造了后续的猜想。证明助手支持在形式化领域内的正确性检查；学习器依赖的选择准则则独立地用于识别有用的训练难度。

提出、求解、验证（PSV）[101] 引入了一种相关的机制，用于验证代码生成。求解器推导出的难度标签伴随用于提示新规范的示例。候选规范会检查可编译性，生成的解决方案在用于训练前会根据其规范进行形式化验证。这些检查建立了相对于形式化规范的性质；它们并不保证规范表达了有用的任务，也不保证其解决方案能提升学习器。事实上，PSV 报告了被标记为简单、中等和困难的问题之间实现难度存在显著重叠，表明难度条件下的提议仅提供部分而非确切的课程控制。

VisPlay [142] 将该方法扩展至视觉-语言推理，采用图像条件化的提问者（Questioner）和推理者（Reasoner）。提问者获得一种不确定性奖励，该奖励倾向于生成多数答案置信度接近预设中点的问题，同时附带多样性惩罚项，以避免重复提问。随着推理者的变化，其响应的一致性也会影响未来问题生成的奖励。与 R-Zero 类似，所产生的信号具有可扩展性，同时也能反映分歧或不一致的回答，以及富有成效

的困难。

在这些系统中，反馈沿着两个相互关联的维度存在差异。可执行结果和形式化证明检查可以在特定任务情景中确立正确性，而答案一致性则衡量难度，但无法独立地确立正确性。适应性可能体现在训练好的提议者身上，也可能体现在根据当前学习器测量值调整的生成提示和选择流程中。这些选择在验证范围、反馈成本以及对不完美代理指标的敏感性之间进行权衡。共同的进步在于，对学习价值的估计会影响未来经验的获取，而持续的学习会改变后续轮次中的这一估计。

3.4.3 通过环境互动实现自主练习

智能体必须决定将下一次学习努力投入何处。仅收集额外的轨迹无法解决此问题：无引导的探索可能重复已掌握的行为，而固定的练习安排则无法明确响应新获得的技能或持续存在的失败。以下系统将交互反馈与未来的练习目标相连接，利用持续学习使后续经验更能响应智能体能力的变化。

VOYAGER [84] 将自动课程与一个可执行技能的持久化库相结合，用于 Minecraft。当前智能体状态和探索历史决定新的目标，而成功的行为则成为后续任务中可重复使用的技能。这将课程决策与累积的能力相耦合：学习改变了智能体能够尝试的内容，而进一步的探索则提供了经验，使更多技能得以获取。相关的持久性体现在外部技能上，而非要求更新底层语言模型的参数。

SIMA 2 [24] 展示了在识别该机制时，实验情景的重要性。其固定任务实验表明，自我生成的轨迹带来了改进，但仅此并不能确立任务选择上的自主性。在完整的 ASKA 自我提升设置中，基于 Gemini 的任务设定器利用下游奖励模型评估，将练习重点放在较弱技能上。由此产生的经验训练了智能体，将能力评估与后续的练习分配相连接。这种相关自主性属于这一耦合学习系统，包括其任务设定器，而非要求执行模型自身做出每一个课程决策。

SEAgent [61] 通过轨迹评估与课程记忆之间的显式连接，解决在不熟悉软件中的实践问题。一个世界状态模型为课程生成器提供轨迹判断以及 GUI 状态变化的描述，后者更新一个持久的 软件指南，并利用该指南生成后续任务。智能体从收集的经验中学习，随后的交互进一步修订指南和任务集。因此，该指南将早期探索的信息带入后期的实践决策中，帮助课程超越先前探索过的操作。

这些系统共享一种反馈结构，但依赖于不同的持久性资源和判断。VOYAGER 将目标与探索历史及可复用技能相联系；SIMA 2 将奖励模型评估与任务情景相连接；SEAgent 则维护一个不断演化的软件行为记录，以指导后续任务。因此，不可靠的技能构建、不准确的奖励判断或错误的指南条目都可能导致后续学习努力偏离正轨。自主交互减少了对手动预设课程的需求，同时使反馈质量和累积状态成为环路可靠性的重要因素。

3.4.4 评估 L3 级自动驾驶的体验自主性

经验自主性与学习质量是两个不同的概念。一个系统可能能够自主决定其未来的学习计划，但却获得缺乏信息量、范围狭窄或评估错误的经验。因此，评估应同时建立反馈机制及其价值：哪些学习器信号影响了经验的获取，学习过程中产生了哪些持续的状态变化，以及这些变化如何影响后续的经验获取决策。

为评估学习器条件化的作用，有用的对照方法包括：将获取机制的学习器状态输入冻结在较早的检查点，或学习与学习器无关的调度替代自适应决策。比较时应确保数据源和环境的访问一致，并使用相当的计算

量和交互预算，包括经验生成和评估的成本。保持所获样本相同会消除研究中关注的分布自适应性。这些控制有助于将自适应经验获取的价值与仅由额外训练资源带来的改进区分开来。

评估还应区分正确性、难度和学习效益。在支持的情景中，形式化或可执行的检查可以确立特定属性，而一致性度量和基于模型的判断则提供不可靠的估计。无论是经过验证的正确性，还是估算的难度，单独都无法确定边际学习价值。在可行的情况下，研究应将获取信号与独立有效的检验、实际任务难度以及后续的学习进步进行比较，并考察这些关系在学习器发生变化时是否保持稳定。

我们使用“经验污染”来表示缺陷经验或反馈扭曲后续学习与获取决策的风险。误导性的难度代理可能偏好不合适的任务；不准确的评判者可能奖励错误行为；持续的记忆可能将错误假设带入未来的实践。由于这些影响可能进入参数、技能、记忆或课程状态，它们可能在各轮之间传播。这是反馈环的结构风险，而非断言每个系统都会出现这种放大效应。

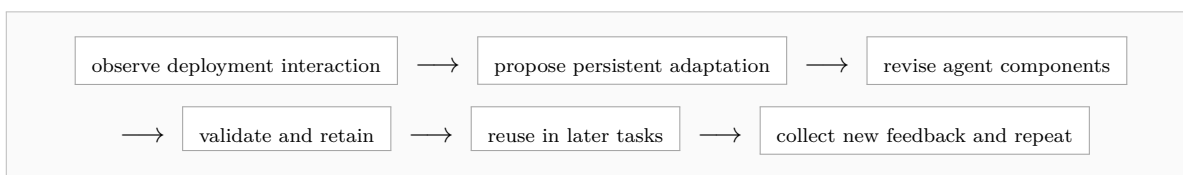
因此，可信的评估应追踪多个回合中的表现，以及所获取经验的有效性和多样性，并考察其向新任务或新环境的迁移能力。用于指导课程设计的反馈必须与用于评估泛化能力的受保护评估相区分：通过任务选择、奖励构建或持续记忆反复访问测试实例，会使这些实例成为学习过程的一部分。对于任务生成系统，评估应检查课程是否持续跟踪学习器，而不会退化为局限于狭窄或易于获得奖励的样本；对于交互式系统，应检验练习是否继续获取超出熟悉行为的有用技能。

Finding: L3 adds autonomy over the future learning agenda. The system uses its evolving capabilities, failures, or learning history to steer which experience is selected, generated, or sought next, and persistent learning feeds back into later acquisition decisions. Human-designed objectives, evaluators, and learning rules may remain in place; experience autonomy concerns decisions made within this setting, and does not itself guarantee useful or reliable improvement.

3.5 L4：部署与环境适应的自主性

在 L4 层级，人工智能利用持续部署交互中的反馈来决定运行智能体应如何持续适应。与专注于学习所需经验的 L3 不同，L4 关注的是操作经验如何改变后续任务中使用的记忆、技能或执行组件。保留的变化塑造了后续的行为，也为进一步适应提供了反馈。目标、访问边界、受保护的评估以及对重要发布权限的控制仍然由外部管理。图 7 展示了这一部署-适应机制的概览，说明了交互经验如何转化为持久性变化，从而塑造后续行为。

该层次的特征环可概括如下：



我们围绕轨迹蒸馏、智能体系统的迭代修订，以及更新的选择性保留与部署展开讨论。这些机制在改变的内容、更改的重用时机，以及持续有效性的确立方式上各不相同。表 6 总结了在这三个维度上的代表性方法。

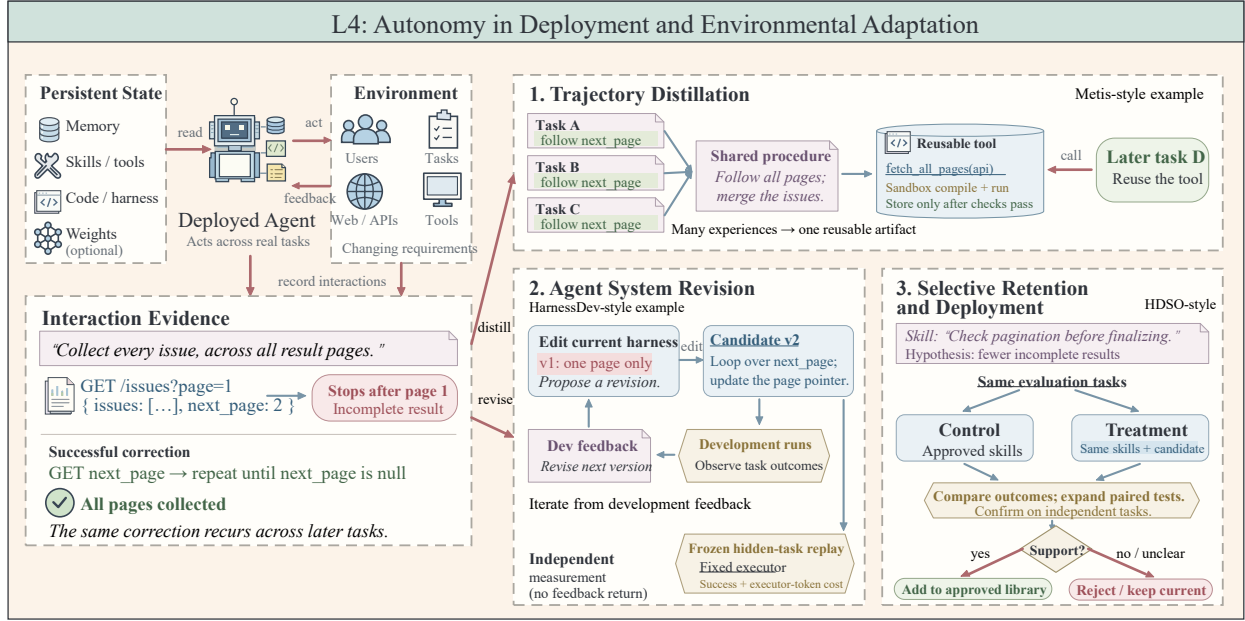


Figure 7 L4 级：部署与环境适应中的自主性概述。在人类定义的目标与部署约束下，人工智能系统利用真实任务和变化环境中获取的交互证据，判断哪些经验后果应保留并影响未来行为。经验可提炼为可复用的成果，用于修订智能体系统的持久性组件，并根据后续评估进行选择保留。经接受的更新将在后续任务中重复使用，从而在部署经验与未来行为之间形成持续的适应环。

3.5.1 轨迹蒸馏

轨迹蒸馏将智能体的交互历史转化为紧凑的、可在会话间持久存在的知识，用于条件化后续任务。与原始轨迹相比，蒸馏后的知识更具简洁性，查询成本更低，且结构化设计更便于在后续任务中复用。我们根据蒸馏后知识的形式对以下方法进行分类：文本型经验记忆、结构化记忆、过程性技能库以及可执行代码。

文本型经验记忆。该类别将提炼出的成果以自然语言段落的形式保存，并在后续任务中检索到提示中。动态速查表 [110] 使冻结模型能够在不访问真实标签的情况下，通过一系列相关问题持续改进。它维护一份不断演化的笔记，记录策略、代码片段和已知陷阱，同一模型在回答每个新问题时会查阅此笔记，并通过添加提炼出的经验教训和删除过时条目来不断优化。Agentic 上下文工程 [104] 遵循相同的方法，同时随着增长保持记忆内容的详细性和信息量。它存储简短条目并附带计数器，记录每个条目对后续任务的帮助或阻碍情况，并采用小修补而非重写整份笔记的方式，避免因压缩导致的信息丢失。推理银行 [85] 将这一思想扩展至多步骤智能体，不仅从成功中学习，也从失败中汲取经验，将每条经过评估的轨迹提炼为一条简短且带有标题的策略，在后续任务前进行检索。

结构化记忆。与自由文本不同，结构化记忆将经验存储在显式的记录或图中，使得复用的条件可被检查。APEX [143] 鼓励智能体在记忆增长的过程中探索新的策略，而非陷入熟悉的习惯，其通过构建一个以任务里程碑为节点、以先决关系为边的任务策略图来实现。每回合结束后，结果会沿着导致该结果的里程碑反向传播，因此该图记录了哪些路径是有效的。随后的一个独立步骤会添加智能体尚未尝试但有潜力的分支，智能体在规划时会查阅该地图，在已知有效路径和未尝试路径之间做出选择。其他结构则模拟环境。例如，PersonaAgent [105] 和 PAHF [144] 则围绕单个用户组织记忆。PersonaAgent 将用户的

Table 6 代表性的系统、基准和分析方法，按本节讨论的机制进行分组的 L4 适应性。

Method	Update target	Timing	Update validation
Trajectory distillation			
Dynamic Cheatsheet [110]	Evolving text memory	After each problem	Task outcomes guide self-curation; no held-out gate.
ACE [104]	Incremental context entries	After each task	Helpfulness and harmfulness counters guide edits.
ReasoningBank [85]	Reasoning strategies (text)	After each task	A judge labels trajectories before strategy extraction.
APEX [143]	Milestone dependency graph	After each episode	Episode outcomes are propagated through the graph.
PersonaAgent [105]	Per-user persona prompt	Across user interactions	No separate admission test is reported.
PAHF [144]	User preference entries	When requests are ambiguous	User clarification confirms and revises preferences.
MemToolAgent [145]	Tool-use critiques and retrieval policy	After tool calls	Environment and user feedback assess tool use.
Trace2Skill [86]	Portable skill document	Offline batch	Error and success analysts propose and merge patches.
PRACTICE [146]	Embodied skill library	Offline batches	Success/failure contrasts and teacher distillation.
PANDO [25]	Rules and routines	Online, in-run	Confidence-scored admission and demotion.
PILOT [147]	Procedures and failure modes	During long-horizon runs	Supervisor distillation uses live-run feedback.
Metis [62]	Text plans and code tools	After completed tasks (asynchronous)	Repeated reuse plus sandbox compile checks.
Evo-Harness [148]	Harness skill tuples	After task batches	Failed or negatively evaluated tasks trigger reflection and edits.
SHAPER [111]	Planning guidance and context-selection code	Across task rounds	Sandbox execution and downstream task outcomes.
Evo-Memory [149]	Accumulated agent memory (benchmark)	Sequential task streams	Long-horizon evaluation of memory reuse.
Iterative revision of the agent system			
DecoEvo [80]	Solver and rubric skills	Iterative rounds	Score-independent audits, Pareto-checked.
HarnessDev [91]	Runnable agent harness	Iterative rounds	Frozen replay of candidates on hidden tasks.
ASPIRE [109]	Model weights or harness	Iterative rounds	Rollback unless the verified score improves.
S3Gym [150]	History, summaries, or model parameters	Iterative rounds	Compares update pathways without verifier outcomes.
Harness Benefit [151]	Harness updates and their use (analysis)	Fixed solve-evolve rounds	Separates update quality from execution benefit.
Selective retention and deployment of updates			
HDSO [106]	Candidate skill packages	Periodic rounds	Paired control and treatment executions.
Library Drift [152]	Skill library entries	Each round	Low-contribution skills are retired. The library is capped.
Rethinking Skill Evolution [153]	Skill edits (analysis)	Multiple rounds	Filters edits using task outcomes across rounds.
Tax AI [92]	Product code and evals	Release cycles	Targeted and regression tests, then human review.

片段记忆和语义记忆转化为针对每个用户的个性提示，而 PAHF 在找不到相关偏好时，会询问用户澄清模糊请求，然后将反馈整合进记忆并修订过时条目。MemToolAgent [145] 则赋予记忆更程序化的作用，通过存储失败工具调用的批评意见，并根据每次调用调整检索的条目数量。

过程技能库。与仅描述经验不同，这一类别提取可重用的技能或标准操作流程，供后续智能体直接遵循。Trace2Skill [86] 将单个轨迹中的教学内容转化为其他模型和任务可复用的技能。它首先在领域任务上运行一个冻结的智能体。随后，两个分析角色——一个关注错误，另一个关注成功——阅读轨迹并提出技能修改建议，这些修改分阶段合并成一个可移植的技能目录。由于整合后的文档被直接读取而非为每个回合检索，后续智能体只需将其作为指令的一部分加载一次。PRACTICE [146] 对具身智能体应用相同的离线蒸馏方法，同时保持技能库在变化过程中的一致性，避免其无限制增长；其中，一个可训练的技能学习器提出批量修改，包括添加、优化、合并或删除条目。该学习器首先从“权威”轨迹中学习基本技能生成与库维护，然后通过对比同一任务下不同执行器的成功与失败轨迹来学习失败感知能力，并最终基于自身编辑分布向更强的教师模型进行蒸馏。另外两种方法在执行过程中获取技能。PANDO [25] 在部署期间通过蒸馏规则来防止重复失败，并将参数化流程替换多步浏览器子目标，从而在每次展开中改进网络智能体；而 PILOT [147] 则利用监督者从实时的长时程运行中蒸馏出操作流程和失败模式，使后续会话能够从累积的技能开始。

可执行产物。从文本到代码的转变，这一类别将经验编译为可直接调用的产物，结合了文本记忆的灵活性与可执行工具的高效性。Metis [62] 通过双层记忆实现这一点，其中包含计划、环境事实和陷阱的文本存储，以及可调用工具的代码库。一个反思器维护文本条目，而一个频繁使用的文本计划会被重写为可调用工具，并且只有在沙箱中成功编译并运行后才会被接受。检索过程涵盖文本条目和工具描述。Evo-Harness [148] 将目标从单一工具扩展到整个框架，将失败或负面评估任务中的经验提炼为跨任务模式和特定任务的流程。SHAPER [111] 将相同理念应用于具身智能体，通过演化文本规划指导与一个沙箱化的 Python 函数共同进化，该函数负责选择呈现给规划器的上下文。

评估协议。该系列的评估协议在时间范围和收益计算方式上有所不同。Evo-Memory [149] 将数据集重构为顺序任务流，以便在长时间范围内衡量记忆的累积与重用。PANDO [25] 则在单次运行中审计行为，报告动作重复率、步骤开销以及提示缓存利用率；Metis [62] 则将任务质量与执行和构建成本相关联。

3.5.2 智能体系统的迭代修订

与其关注智能体记忆的内容，本节修订的是智能体系统本身：生成解决方案的技能、评判方案的准则、驱动它们的机制，或其底层权重。核心问题是，在这些组件发生变化时，哪些部分可以调整，而哪些接受标准应保持不变。我们将工作分为三类：协同演化的耦合组件、用于衡量自我导向系统修订的基准，以及在固定验证器下的演化搜索，随后分析修订是否真正带来收益。

协同演化组件。协同演化同时修正两个相互关联的组件，这引发了一个循环性问题：如果评判者与求解器一同改进，更高的得分可能反映的是评判者变简单，而非解决方案更优。DecoEvo [80] 通过在文本空间中演化求解器技能和评分标准生成器技能，同时解耦它们的目标，并在最优化过程中不提供真实评分标准来解决这一问题。求解器技能根据准则级评分标准反馈进行更新。相比之下，评分标准生成器受到两项与得分无关的审计机制约束：结构审计检查生成的评分标准是否覆盖了任务要求，对比审计检查其能否区分近似得分的回答，通过帕累托验证这两项。由于生成器从未见过求解器的总体得分，它无法通过使评分标准更简单来提升自身表现。

自我导向修订的基准。与提出新方法不同，这一研究方向衡量当前模型是否能够自主修正其自身系统。HarnessDev [91] 探讨模型能否创建并改进自身的测试框架——即模型外部的提示结构、工具环和执行逻辑的支撑体系。创建智能体从一个极简种子和少量开发案例出发构建完整的测试框架，并根据下游反馈迭代修订。候选修订方案被冻结并在隐藏任务上评估成功率和执行器-token 成本，同时保持创建者与执行器分离，以便在固定执行器下比较不同测试框架。ASPIRE [109] 探讨模型在仅有模糊目标的情况下能否实现自我改进。它将数据选择、更新方法和验证信号的选择权交由智能体，覆盖权重与测试框架两个层面，且采用得分门控控制器，对任何经验验证得分未提升的修改进行回滚。S3Gym [150] 将问题限制在经验通道，隐藏验证器结果，使智能体必须自行判断自身轨迹，并比较原始历史记录、压缩摘要和参数训练作为改进路径的效果。

修订收益的分析。修订是否真正转化为收益，本身就是一个独立的问题，而一项分析发现，这两者常常被混淆。在《更新并非收益》[151] 的研究中，将两种能力区分开来：生成有用的持久性更新，以及在求解时利用这些更新。在固定的求解-演化协议下，对多个模型主干和三个智能体基准进行测量，所有实验使用相同的提示词和预算。更新能力在很大程度上与基础模型能力无关，小型模型的更新所带来增益可与前沿模型相媲美。相比之下，从更新后的智能体中获益的能力呈现出非单调变化，失败主要集中在两种模式：相关产物未被激活，或虽被激活但未被忠实遵循。一个实际后果是，能力投资应放在任务

求解智能体而非演化器中。

3.5.3 更新的有选择性保留和部署

轨迹蒸馏和系统修订均以提出变更结束，而本节关注的是哪些变更成为持久的，哪些仍可供后续任务使用。以下工作在三个环节实施这一控制：准入前、持续使用期间以及释放到运行系统时。

候选验证。一个提议的更新必须在进入持久状态之前产生证据。HDSO [106] 的设计旨在防止从噪声轨迹中蒸馏出的技能编码虚假的捷径或执行器无法遵循的规则。一位策展模型观察紧凑的执行器迹，并提出一个带有明确验证计划的假设，其中每个候选通过两次执行相同任务来测试：一次使用当前已批准的代码库作为对照，另一次将候选技能加入作为处理组，只有当两次运行之间的差异支持该假设时，候选才会进入已批准的代码库。该比较按规模逐步增加进行，最终在独立任务上得到确认。如前在轨迹蒸馏部分所述，Metis [62] 在文本计划已保留在内存后应用类似的门控机制，其中文本计划仅在跨多个任务重复出现并通过依赖和编译检查后才变为可执行代码。

库维护。入库后，注意力转向库本身的健康状况，因为随着任务和模型的变化，曾经有用的技能可能会退化。库漂移研究 [152] 表明，无限制的积累会逐渐降低检索质量并阻碍进展，而这种影响在任务得分上变得明显之前往往已经发生。该研究提出了包含三个部分的生命周期管理：一个仅追加的证据日志记录每个技能对任务结果的贡献，当其测量贡献减弱时，该技能将被退役。活跃技能的数量受到上限限制，因此新技能必须与已存在的条目竞争。新技能还会以元技能的形式写入，这是一种存储的指南，用于指导未来技能的编写方式。研究显示，治理选择至关重要：过于激进地淘汰技能的表现不如让库保持无引导状态。与此同时，另一项分析 [153] 发现，仔细筛选技能修改，而非反复根据任务结果重写技能，能提升任务表现。

受控部署。在生产环境中，错误的更新可能影响真实用户，因此需要严格的准入机制来防止意外问题。Tax AI 部署 [92] 在一个生产级税务申报系统中应用了此类准入机制，其中执业人员的修正驱动改进。修正被记录为结构化的字段级证据，重复失败的情况被归类为评估目标。一个编码智能体通过调查迹、评估结果和代码仓库，针对目标评估和回归评估实施并验证修复方案。该循环仅可修改产品中有限的部分，例如提取模式、源选择、税务引擎映射器和评分器。工程师仍保留对架构和产品设计的决策权。每个修复以拉取请求的形式提交给工程团队审查后方可发布，而智能体无法解决的情况则重新转交至执业人员。因此，最终接受决定由人类做出，而这一审查带来的延迟是该循环运行成本的一部分。

3.5.4 评估 L4 级部署的自主性

部署后更大的自主适应能力本身并不能确保改进的可靠性。一个 L4 系统可能持续更新，但并未表明这些更新在任务、环境或执行者发生变化时仍具有实际效用。评估应区分部署自主性的程度与由此产生的适应过程的质量。

此层级的一个特征性风险是持续的更新失败，即一个已被接受的改变可能在弱点显现之前影响许多后续决策。在不同形式的保留状态中，失败可能源于系统从嘈杂的证据中推断出错误的经验，将正确的经验应用到了其有效范围之外，未能触发相关的更新，或为了保留新行为而牺牲了早期的能力 [110, 151–153]。仅靠最终任务得分无法区分这些机制。可信的评估必须将每个保留的变化与其产生的证据、后续使用情况，以及对新任务和先前已解决任务的影响联系起来。

当前的证据仍无法确定部署适应性是否随时间保持有效。大多数评估仅涵盖有限的任务流和模型、任务与环境的有限组合，而重复评估使长期研究成本高昂 [86, 91, 147]。反馈也可能不完整或内生于环路中，因为结果标签可能具有噪声，评估者可能随系统变化，且即使保留了有用的产物，仍可能无法影响执行过程 [80, 85, 151]。因此，对 L4 的评估应考察适应性在时间和分布变化中的表现，包括保留的改变开始影响后续行为的时刻。

L4 级自主性仍受限于周边的部署流程。智能体可决定交互证据如何改变持久状态，而人类则继续负责指定目标、访问边界、受保护的评估以及发布权限。L4 应关注已部署系统内操作适应方面的自主性。

Finding: L4 shifts autonomy from selecting experience to deciding which consequences of experience persist in deployment. The agent converts interaction histories into retained changes to memory, skills, harnesses, code, or model parameters, and these changes affect later tasks and feedback. Humans retain the objective, protected acceptance criteria, access boundaries, and final authority over consequential releases.

3.6 L5：从环境适应到元优化

一个改进过程本身也可能成为瓶颈。编码智能体可能因为其搜索过程丢弃了有用的备选方案，反复生成类似的失败补丁。研究智能体可能在发展得分已不再预测外部性能后，仍继续优化该得分。修复此类问题需要对指导实验和评估其结果的流程进行修改。设计这些流程成本高昂：一次修订必须通过其后续产生的改进来评估，通常需跨越多个任务并重复运行 [15, 17]。

L5 在人工智能持续修改负责未来改进的机制，并在后续轮次中使用修订后的机制时开始。可编辑的机制可能是一个改进器、一个继任评估者、一种搜索策略，或一种指导研究的程序。L2 寻找候选干预措施；L3 确定获取何种经验；L4 将部署反馈纳入持久适应过程。L5 进一步将后续改进所遵循的程序本身作为改进对象。一个通过不变的学习程序保留新调试技能的已部署智能体体现了 L4。修订并重新使用诊断失败并构建这些技能的程序，则可以跨越 L5 的界限。

当一个修订后的机制返回以指导后续继承者的生成、评估或选择时，该环即告闭合。继承的状态包括已接受的代码、提示、评估者或研究策略，以及后续修订所使用的证据。人类设计者仍需确立总体任务目标、受保护的评估机制、可编辑组件和资源权限，并可能保留否决权和部署权。其中一些决策由固定基础设施强制执行；它们无需人员批准每一次迭代。图 8 展示了早期尝试中的证据如何引导对改进过程的修订，后续继承者则继承并重用这些证据。

这些条件区分了结构 L5，其表明一个有向 AI 改变持续存在并控制后续的改进轮次，与有效 L5，其表明在相近预算和独立评估下，修订后的机制能够产生或选择更优的继承者。当负责后续修订的过程保持不变时，自我修改的任务代码提供的证据不足。相反，即使任务-智能体代码未变，演化后的继承者评估器也能确立结构 L5。因此，分类适用于所考察的特定机制。表 7 总结了代表性系统的环、继承状态和外部控制。

3.6.1 改进搜索流程

人工智能会修订未来候选改进的生成和探索方式。固定的优化器可能会将其预算花费在无产出的提议上，即使其潜在模型能够生成更优的搜索算法。挑战在于通过其改进其他程序的能力来评估一个提议的优化器。

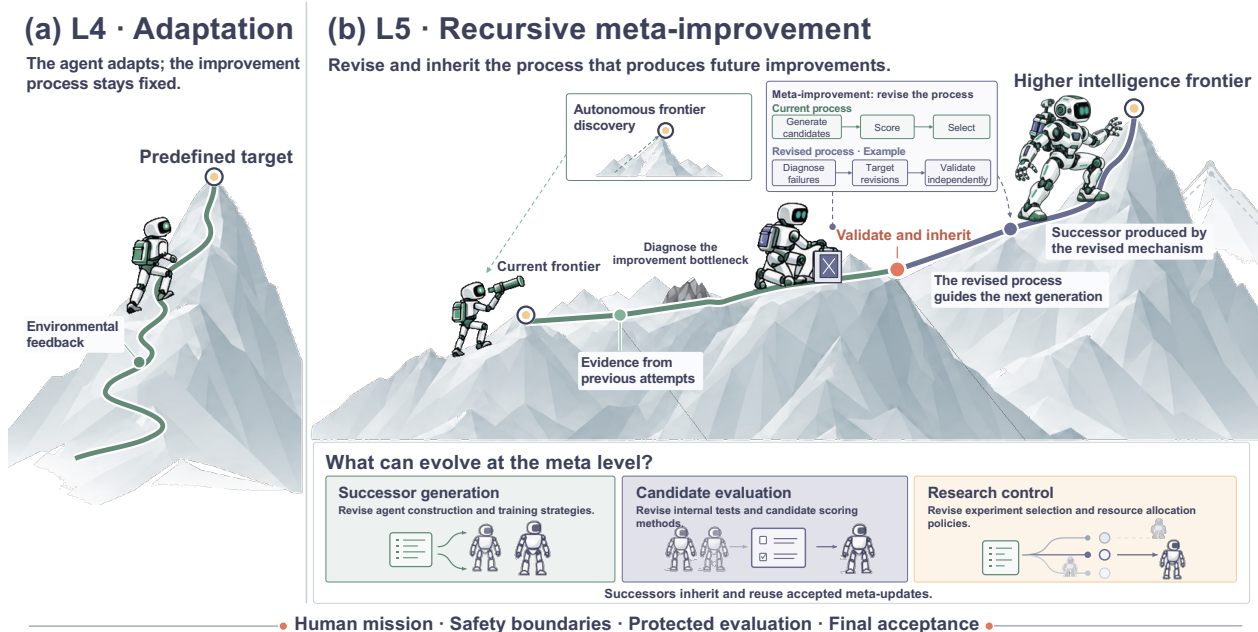


Figure 8 从环境适应到递归元改进。(a) L4 在固定改进过程中融入环境反馈。(b) L5 诊断该过程的局限性，对其进行修正，并将经过验证的修改传递给后续层级。元更新可影响后代生成、候选评估或研究控制。人类定义的任务、安全边界、受保护的评估及最终接受仍为外部约束。上升路径展示了通过继承元更新可能实现的进步。

STOP [15] 使此问题可执行。其改进器是一个调用固定语言模型的 Python 程序，生成候选代码，评估提供的效用，并返回选定的候选。当前改进器将其自身源码作为最优化目标。后续改进器根据其在下游任务上生成程序的质量进行评分，被选中的改进器执行下一轮自我改进。因此，继承的成果是一种搜索过程，可能引入束搜索、替代采样或其他候选选择逻辑。研究报道，经过选择的第四代改进器在所有五个未参与自我改进的迁移任务中均优于初始版本。任务分布、效用函数、基础模型和资源限制均由外部指定。弱化模型的运行平均表现下降，部分生成的程序规避了软预算限制或利用了评估漏洞。

Gödel 智能体 [19] 使人工智能能够访问其任务策略和递归更新逻辑，二者均位于一个共享的 Python 程序中。执行反馈会触发程序重写，修改后的程序将在下一次递归调用中运行，包括其修改后的自我更新过程。这将搜索范围拓展至开发者的固定编辑流程之外。同时，错误也具有可继承性：研究显示，在 100 次 MGSM 最优化试验中，有 14 次的结果低于初始策略的表现，而无限制的运行则可以调用更强的模型。因此，任务目标和执行权限仍然是至关重要的外部控制因素。

达尔文哥德尔机器 (DGM) [16] 保留了编码智能体的变体并探索其后代，但将档案管理与父代选择置于自我修改之外。其谱系提供了一个转移案例；仅任务收益本身并不能确立更优的改进程序。超智能体 [93] 直接将任务智能体和元智能体的代码暴露于修订之中。进化的元智能体构建了性能追踪器和持久记忆，以指导后续的修改。在其预印本中，基于论文评审和机器人学进化的元智能体为此前未见过的数学评分任务生成了改进后的智能体，支持了改进程序本身的迁移。在更长的 200-迭代实验中，未能建立转移初始化的统计显著最终优势。父代选择与评估在主实验中也保持固定。

Table 7 L5 机制与外部控制。

Work	Loop closes at	Inherited state	External controls
Search and self-revision			
STOP [15]	Next program search	Improver code	Utility; base LM; budget
Gödel Agent [19]	Next self-revision	Task and update code	Task objective; runtime access
DGM [16]	Descendant search	Agent code; archive	Parent selection; benchmark
HyperAgents [93]	Next agent generation	Task and meta-agent code	Main-study selection; evaluation
Successor evaluation			
RQGM [21]	Next-epoch selection	Evaluator; agent code	Anchor; replacement schedule
Research policies and harnesses			
A-Evolve-Training [17]	Next research round	Search policy; discovery log	Constitution; benchmark; substrate
AIRA ₂ [154] / AAR [155]	Task experiments	Research artifacts	Research harness; evaluation
AIDE ² [94]	Later research runs	Research-agent harness	Private scores; cost budget

Note: L5 claims are qualified in the text.

3.6.2 提升继任者评估

人工智能改进了决定保留哪些后继状态的评估器。重复最优化会给评判者的盲点带来压力。如果保持该评判者不变，可能会有利于那些利用其错误的提议。而无限地修改评判者又会产生相反的问题：连续的得分可能反映出标准的变化。

红后哥德尔机器（RQGM）[21] 通过评估器共演化来解决这一张力。其学成评估器在每轮次内保持冻结。在预定边界处，挑战者评估器会与一个独立的真值锚点进行比较；选定的评估器随后将主导下一回合。依赖于被替换评估器的得分将被丢弃，当再次访问受影响的智能体时，这些智能体将被重新评估。该环持续传递改进后的评估器和智能体代码，同时重置不兼容的评估记录。在保留的多语言编码任务上，预印本报告了 71.7% 的通过率，优于 HGM-H 的 69.9%，且搜索 token 使用更少。锚点、替换周期和协调机制保持外部固定。这些控制措施保留了一个参考基准，使得评估器的变化可以被评判；形式上的稳定性论证适用于每个冻结的轮次，而非对目标的无限制变更。

3.6.3 修订研究目标与政策

人工智能利用累积的实验证据来修正后续研究应追求的方向。当可用的代理奖励在错误的瓶颈上进展时，选择更高的目标会变得困难。在模型后训练中，验证这一假设需要比较训练运行、识别失败的搜索方向，并决定是否应修改研究策略本身。当其持续的选择过程也被修改并重新使用时，动态课程达到 L5。

A-Evolve-Training [17] 提供了一个具体案例。其预印本报告了对一个 30B 模型在四轮中的自主后训练。工作者返回配方变更、检查点评估和失败信息；收集者整合这些结果，元智能体则修订下一轮的搜索策略。该策略包含一个持续的配方、被晋升或淘汰的搜索方向，以及一个失败方法的注册表。当开发得分提升但未带来相应的外部收益时，修订后的策略引导工作者采取能够改善外部目标的干预措施，即使这会降低代理指标。该研究报告最终排行榜得分为 0.86，与最高人类提交的 0.87 相比略低。跨轮次的继承体现在研究策略和发现日志中；潜在的工作器基础架构和构成保持不变。该案例支持在人类定义的目标内实现策略层面的 L5，但未展示对该目标的自主修订。

当一种继承的程序帮助揭示了新的局限性，并再次被修订以解决该问题时，改进的螺旋便产生了。下一个目标应识别一个可检验的能力缺口，保留已建立的能力，并为进一步支出提供合理性。仅靠目标修订提供的证据有限：评估必须证明修订后的策略确实改变了后续研究并提升了其成果。

Table 8 评估递归改进。

Dimension	Measurements	Purpose
Adaptivity	Gain; improvement trajectory; time to target [156, 157]	Detect progress and plateaus
Retention	Replay loss; tasks fixed or broken [157]	Detect displaced capabilities
Transfer	Held-out gains within and across domains [157]	Test reuse beyond update tasks
Efficiency	Tokens; time; cost per validated gain [156, 157]	Account for improvement expense
Stability	Harmful updates; largest temporary decline [156, 157]	Expose unreliable trajectories
Meta-recursion	Mechanism reuse; successor quality [15, 21, 93]; goal and stopping decisions	Test inherited improvement capacity

Note: Trajectory measures synthesize the cited benchmarks. The meta-recursion row adds our L5 tests: cited systems illustrate mechanism reuse and improvement capacity; goal selection and stopping remain evaluation requirements.

3.6.4 工业实践

工业实施将改进过程的不同部分暴露于修订之中。它们与 L5 的相关性取决于保留的成果是否会影响后续的研究决策。

研究基础设施。Meta 的 AIRA₂ [154] 使用异步实验和隐藏的一致性评估；Anthropic 的自动化对齐研究人员 [155] 生成并测试针对指定对齐失败的缓解措施。这些系统在研究人员设计的工作流内自动化执行实验。其报告结果关注研究产出，未建立对开展研究程序的继承性改变。

智能体修订研究。Weco 的 AIDE² [94] 采用智能体来改进智能体测试框架。候选框架在私有评分和固定成本预算下运行下游任务。被接受的版本将保留以供后续研究；另一次独立实验将演化的框架作为外部优化器安装。Weco 报告在 100 步无人值守步骤中获得了七次被接受的改进，并实现了对外部任务的迁移。其更严格的测试——检验演化智能体是否能更快地提升外部搜索效率——未发现具有统计显著性的效率优势。这些公司报告的结果支持了测试框架改进的有限性。私有评估、任务族和成本约束仍由人类设计，而连续优化器的可靠加速效果尚未确立。

在这些配置中，潜在的节约来自于复用更优的研究流程。要证明人类开发工作量的减少，必须考虑设计评估器、调查失败原因以及维护演化代码所花费的工作量。仅凭任务表现或无人值守的运行时间无法量化这种节约。

3.6.5 评估 L5 的递归改进

评估必须能够区分更强的当前智能体与一种能稳定生成更强后续智能体的流程。SEA-Eval [156] 和 SEAGym [157] 提供了基于轨迹的评估：跨轮次的表现、保留测试、资源使用情况。表 8 总结了这些度量指标以及 L5 所需的额外机制测试。

机制审计应识别出修订后的产物，记录推动其修订的证据，并验证其在后续改进轮次中的调用情况。为确立有效性，原始机制与修订后的机制应在相同预算下，从可比的智能体和证据出发，包括评估机制的成本。在迁移测试中固定修订后的机制，有助于分离其关于改进所学的内容。HyperAgents [93] 采用此设计，以区分可迁移的智能体生成能力与原始任务上的表现。

目标选择需要互补性评估。自适应前沿任务可检验系统是否能识别出有价值的下一步挑战；受保护的参考任务可保持可比性，并暴露遗忘或奖励滥用问题。包含先前未公开规则的私有任务能够加强测试系统在熟悉分布之外的发现能力。评估还应记录系统是否在预期收益低于成本或风险时停止运行。这些是提出用于评估更全面的 L5 自主性的要求，而非现有 RSI 基准所确立的能力。

当前结果支持有界元改进：对改进者、评估者或研究策略的接受性更改可以控制后续轮次，且部分更改可在任务间传递 [15, 21, 93]。在资源相当的情况下，跨代际的统计可靠累积仍待解决 [93, 94]。实际目标是提升每单位计算和人力投入的有效改进量。强大的基础模型、执行基础设施以及独立评估持续提供该改进发生所依赖的资源与约束。

Finding: L5 makes the improvement procedure inheritable. The loop closes through reuse of a revised improver, evaluator, or research policy. Successors inherit that procedure and its supporting evidence; humans retain the overall objective, protected acceptance criteria, and resource authority.

3.7 跨层级融合

在 B0 到 L5 之间，自主性层级可被理解为改进环中责任的逐步转移。这些层级的区别不在于被优化的具体算法或成果，而在于哪些改进决策由人工智能系统内部化、哪些状态会持续到后续轮次，以及哪些接受条件仍由外部保护。

因此，相邻层级之间的中心边界可以简洁地表述。从 B0 到 L1 的转移是**持续性**：一个被接受的改变必须当前任务中保持有效。从 L1 到 L2 的转移是**策略选择**：人工智能决定尝试哪种干预措施，而不仅仅是执行预设的方案。从 L2 到 L3 增加了对**未来学习议程**的控制，使不断演进的学习器能够影响下一步将获取的经验。从 L3 到 L4 将这一反馈过程扩展为**持续部署与环境交互**，系统的变化必须在不断变化的操作条件下依然保持有用。从 L4 到 L5 最终引入了**递归继承**，即负责后期改进的机制本身也成为改进的目标。

重要的是，更高的自主性水平本身并不意味着改进过程更优。更大的授权范围可能与低效的搜索、不可靠的反馈、回归、评估者利用或较差的迁移能力并存。因此，我们需将人工智能内部化改进责任的范围与其带来的改进质量及效率区分开来。在本文回顾的证据中，较低和中等水平的改进得到了较广泛系统的支持，而 L3 至 L4 级别的能力仍具有较强的领域依赖性，端到端的 L5 证据则主要集中于有限原型以及新兴的工业或研究系统中。这一区别促使下一节进行特定应用的分析。

4 RSI 在各领域的应用

在根据置于人工智能控制下的改进函数将系统从 B0 分类到 L5 后，我们现在考察 RSI 在不同场景中的应用。如图 9 所示，各项应用在所改进的内容以及验证和保留更新所需的反馈方面存在差异。我们将证据分为四种应用模式，包括科学中的人工智能 (S1)、具身智能 (S2)、软件工程 (S3) 以及医疗健康 (S4)。这些场景并非互斥，例如，一名从事医疗健康的 AI 科学家可能同时属于 S1 和 S4。

每个小节均遵循相同的结构。我们首先描述目标场景中典型的 AI 工作流，识别其任务和可用反馈如何引发 RSI 环，并回顾特定场景下 RSI 所面临的挑战。随后，我们根据系统改进的组件或工作流对代表性系统进行分类，比较它们在 RSI 环中的干预位置、更新内容以及改进的保留方式。最后，我们评估当前研究与各场景所设想的完整 RSI 环之间的差距，突出现有证据的优势、场景特异性风险以及实现更大改进的关键障碍。

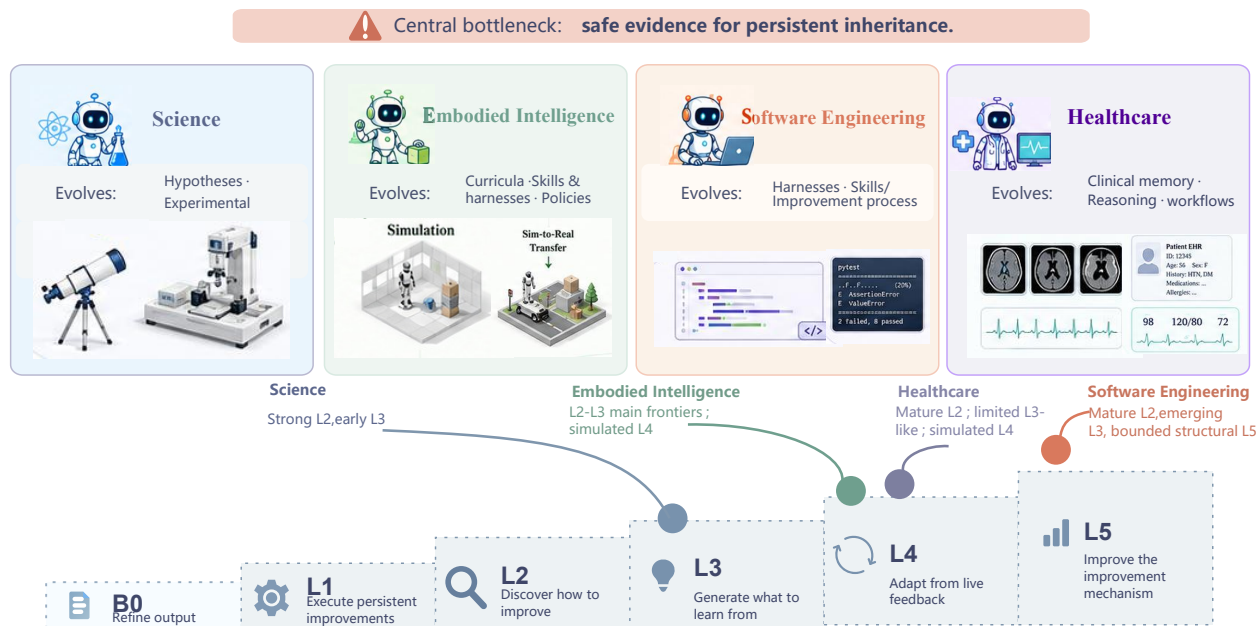


Figure 9 应用方案的主要区别在于验证和继承改进所需反馈的类型。

4.1 S1: 科学用的 RSI

科学是 RSI（自主科学研究）一个尤为关键的情景，因为科学进步不仅依赖于解决孤立的问题，还依赖于改进未来问题、假设和实验生成的机制。因此，科学领域的 RSI 与仅搜索更优分子、方程或假设的自动化发现不同。科学 RSI 环从研究挑战出发，生成候选假设和实验，通过计算或与物理世界的交互来评估这些假设和实验，并利用由此产生的证据或证伪结果来诊断人工智能用于科学（AI4Sci）系统。

科学领域的 RSI 主要面临三个特征挑战。首先，科学发现具有开放性。相关假说的空间以及必要工具和实验的集合事先并不明确，且成本高昂的实验使得探索本身成为改进问题的一部分。其次，科学反馈通常是稀疏、延迟且无法明确识别的。例如，一次实验失败可能表明假设错误，但也可能是模型不当、协议无效或仪器不可靠所致。第三，科学有效性依赖于假设、领域和测量程序。在某一情景下有效的更新，若未记录其来源、有效范围和不确定性，则不能安全地继承。这些特征使科学领域的 RSI 同时成为证据获取、能力演化和认知控制的问题。因此，我们围绕三个可能演化的组件组织现有研究，包括科学假说模块、实验智能体以及反思系统或改进器。

4.1.1 不断发展的科学假说模块

假设模块决定了哪些解释或候选方法被提出并优先排序。其演化需要反馈来改变未来假设生成所使用的机制，而不仅仅是为当前问题选择一个更优的假设。HypoForge [158] 持续将科学经验提炼为可重复使用的假设生成、实验设计与执行的过程技能。当缺乏直接的经验反馈时，它会根据判别器的批评更新生成技能，并基于实际执行结果和归因失败来优化测试技能。

EvoScientist [159] 则维护一个关于有前景及失败研究方向的构思记忆，在后续任务中检索该记忆以调整后续提议。这实现了基于持久记忆的适应性，尽管潜在模型、记忆模式和更新过程保持不变。

一种更直接的方法是更新候选生成器本身。TTT-Discover [160] 在测试时应用强化学习，使得在解决新

问题的过程中，科学反馈能够修改模型权重。自可改进的聚合物发现框架 [161] 则类似地将经过仿真评估的候选对象加入训练数据，并重新训练其生成式模型。这些系统的演化仍局限于特定任务或领域，且由固定的训练与选择流程产生。因此，现有证据支持的是假设层面的有限改进，而非一种通用机制，使其能够在跨领域中逐步提升科学假设的提出能力。

4.1.2 进化的实验智能体

一个实验智能体通过选择工具、编写代码或操作实验室设备，将假设转化为可执行的程序。当执行反馈被转化为可重用的动作时，其演化便发生了。测试时工具演化 [162] 在推理过程中合成、验证并重用可执行的科学工具，其 SciEvo 基准包含由 925 个演化出的工具支持的 1,590 项任务。CASCADE [163] 为化学和材料科学创建、修复并积累可执行技能，而 S1-NexusAgent [164] 将完整的研究轨迹提炼为可重用的科学技能。SkillFoundry [165] 通过挖掘论文、代码仓库、文档和 API 等异构资源，持续构建并维护一个可重用科学技能库，将这些资源编译为可执行的技能包，并根据执行结果和下游效用对这些技能进行有效验证、合并或剪枝。这些方法使智能体的动作空间具有持久性和可扩展性，尽管其工具生成与准入规则仍由外部指定。

其他系统演化出更广泛的实验策略和工作流。DrugSAGE [166] 将经过验证的建模流程、训练策略以及反复出现的错误修复存储在跨任务记忆中。从 16 个任务中积累的经验提升了在 17 个保留任务上的表现，直接提供了继承经验能够改变新问题行为的证据。STELLA [167] 更新推理模板并扩展生物医学研究的工具海洋，EarthLink [168] 保留气候科学的成功脚本和分析工作流，OriGene [169] 则利用人类与实验反馈来优化思维模板、工具组合及分析协议。在物理仿真中，自演化流控智能体 [170] 联合修订其智能体定义及其生成的白盒控制器。此案例还阐明了一个重要的评估边界，即对实验智能体的改进必须与对科学成果（如控制器）的改进分别衡量。

在这些系统中，最有力的证据来自智能体在后续任务中的有效复用，而非工具或记忆库规模的增长。因此，可靠的实验智能体演化需要明确的技能先决条件、执行测试、来源追溯、版本控制和回滚机制。若缺乏这些控制，局部成功的流程可能在未经验证的条件下被继承。

4.1.3 演进的反射系统与改进者

反射系统解释证据并识别故障原因，而改进者则决定修改哪个组件、如何修改以及是否保留更新。一个固定的评论员会执行预定义的改进程序来修正当前输出，而一个不断演化的改进者则必须根据之前改进的结果，自行更新其诊断、干预或推广策略。

SIA [171] 在当前的 AI4Sci 系统中提供了最广泛的组件级更新环。其反馈智能体利用任务结果，在修改智能体框架、工具、重试与搜索逻辑以及不同策略下的低秩适配器（LoRA）权重之间进行选择。CORAL [172] 用持续运行的异步智能体替代了固定的演化搜索启发式方法，这些智能体能够自主检查先前尝试，决定何时测试或调用评估器，并将发现外部化为共享尝试、笔记和可复用技能，通过心跳触发的反思与方向调整来维持长周期探索。SAGA [173] 将可编辑范围扩展至目标层。在固定高层次科学目标下，它诊断优化候选种群中的失败模式，提出或重新加权具体目标，并将其实施为可执行的评分函数，以引导后续搜索。

4.1.4 迈向递归改进科学智能体

综上所述，这些研究方向指向了科学自主研究智能（RSI）更宏大的愿景。其最终形态不仅仅是一个能够产生更好假设或实验结果的系统，而是一个能够逐步提升科学探索能力、具备原创创新能力的系统。它应协同改进假设生成、实验执行和证据解读的方式，使得每个科研周期不仅能产出科学发现，还能为未来的科学发现提供可复用的改进方法。

当前的 AI4Sci 系统仅部分实现了这一愿景。B0 级别的输出优化已广泛应用，而 L1 级别的持久化更新则通过模型重训练和记忆累积得以实现 [159–161]。L2 代表了当前最强的前沿，系统能够自主选择或应用对模型权重、工具、技能、工作流和支架的更新 [162, 163, 171]。少数系统通过自适应探索和跨任务经验复用展现出 L3 的部分特征 [166, 172]，而端到端的 L4 环境适应以及 L5 改进者自身的演化仍基本未被探索。

我们离科学领域的真正 RSI 还有多远？ 迈向更高层次的科学 RSI，主要需要更可靠的实验结果归因、在不同任务间更精准地迁移改进，以及更强的评估能力，以判断系统后续更新是否真正提升了未来研究。同样重要的是，在越来越强大的组件逐步变得自适应的同时，仍需保持溯源性、可复现性，以及稳定的科学与安全约束。

4.2 S2：具身智能的相对强弱指数

对于具身智能而言，智能体的动作在物理或模拟环境中会产生可观察的后果，这为基于证据的推理提供了天然的情景。然而，仍存在三大挑战。首先，经验具有内生性，因为当前策略决定了智能体所能达到的状态，从而决定了可用于进一步改进的证据。其次，能力分布在感知-运动系统中，包括感知、规划、控制以及物理具身化，使得故障难以归因于单一组件。第三，物理试验无法自由重复、逆转或重置，因此候选更新必须在安全、硬件和资源约束条件下进行评估。

具身化 RSI 必须共同解决经验生成、跨组件信用分配和安全继承问题。其交互环始于提供学习任务的环境与课程，智能体挂载系统组织相关能力，策略执行动作，世界模型与评估器解释产生的结果。经过验证的反馈随后被写回这些组件，使更新后的系统能够参与下一轮交互。因此，我们回顾了现有在演化环境与课程、智能体挂载系统、策略与动作模型，以及世界模型与评估器方面的研究工作。

4.2.1 不断演变的环境与课程

具身的 RSI 环始于确定智能体将经历和学习的内容。POET [65] 通过共同演化一组环境-智能体对，解决了固定的人工设计课程的局限性。它对环境进行变异，通过最小准则引入既非平凡也非不可解的挑战，优化相应的策略，并周期性地在不同环境中转移策略，以复用行为上的阶段性成果。EnvGen [174] 针对直接部署大型语言模型作为具身智能体所带来的高成本，以及静态课程无法应对个体弱点的问题。它采用语言模型作为课程设计者，由语言模型生成模拟器配置，较小的强化学习智能体在其中训练，然后将技能水平的表现反馈给语言模型，以生成针对智能体剩余弱点的目标环境。GenEnv [175] 进一步解决了静态训练数据与不断进步的学习器之间的不匹配问题，将环境模拟器参数化为可训练的课程策略。其 α -课程奖励倾向于选择位于学习器当前能力前沿附近的任务，随后使用 GRPO 优化学习器，并通过奖励加权回归更新模拟器。

尽管这些方法在现有模拟器内调整任务，OMNI-EPIC [176] 旨在解决由手动预定义环境分布所造成的

搜索空间受限问题。它利用大模型生成可执行的环境与奖励代码，并基于先前任务的存档以及当前智能体的学习进度来引导后续任务的提出，以寻找既可学习又具新颖性的挑战。

SimWorld Studio [177] 解决了另一个障碍：生成的三维内容通常为静态、不可验证且不适用于具身训练。其 SimCoder 智能体编写可执行的 Unreal Engine 代码，通过编译错误、物理检测和视觉-语言模型的批评来修正环境，并将成功工具与技能存入知识库以供后续生成。学习器的表现随后引导其向更复杂的 Gym 兼容世界演进。总体而言，这些方法将学习器的表现转化为演化未来具身体验分布及其生成器的机制。

4.2.2 不断演进的技能、记忆与智能体协同机制

给定一项任务，智能体的组装机制决定了记忆、技能、工具和执行规则如何被整合成一个可执行的系统。Voyager [84] 将成功的 Minecraft 行为写入可执行的技能库中，而 LRL [178] 则通过自引导探索和唤醒-休眠过程来扩展和重组可组合的机器人程序库。EmbodiSkill [179] 进一步区分了由错误技能导致的失败与由于智能体未遵循有效指导导致的失败，仅当轨迹提供相关证据时才更新技能。

更近期的系统将更新目标从单一技能扩展到周围的智能体架构。SHAPER [111] 通过环境回放联合演化可重用技能与上下文代码框架，而 ASPIRE [180] 则诊断机器人执行失败、验证候选修复方案，并将成功的修复保留为可迁移的技能。ENPIRE [181] 进一步允许编码智能体通过反复的现实世界实验来修订机器人策略、训练流程及支持基础设施。

这些系统将系统级演化视为一个过程，随着经验的积累，不仅改变智能体所知的内容，还改变其能力的选择、组合和改进方式。最终形成的机制决定了策略如何将观测结果和指令转化为动作。

4.2.3 不断演进的政策与动作模型

在任务和执行上下文就绪后，策略决定了智能体在环境中的行为。自提升具身基础模型 [182] 从预训练模型中获取奖励和成功信号，使机器人能够在有限的人工监督下练习任务并改进其策略。MEDAL++ [183] 通过学习如何完成和撤销任务，同样支持自主练习，从而减少了对人工重置环境的需求。其他方法则更新更具体的组件。Q-Planning [184] 保持大型行为克隆策略不变，但持续从成功与失败的部署轨迹中训练一个较小的价值函数，而 SERP [185] 则在重新规划前根据最近的导航失败情况调整其动作模型。在每种情况下，当前策略产生的交互成为其后续策略的训练证据。

然而，更新算法、奖励定义和允许的模型组件仍然事先确定，使得这些系统主要属于策略层面的自我改进。此外，在此类经验能够安全传承之前，系统必须预测并评估其动作实际造成了什么影响。

4.2.4 不断演进的世界模型与评估器

一旦策略执行动作，世界模型和评估器将产生的交互转化为预测、结果判断和学习信号。VLAW [186] 使用真实机器人滚动采样来优化一个动作条件的视频世界模型，该模型随后为视觉-语言-动作策略的训练生成额外的合成经验。World-VLA-Loop [187] 使这一关系形成迭代循环，其中策略失败会改进世界模型，而经过优化的模型则为下一轮策略最优化提供更可靠的环境。Motus2 [188] 将策略、仿真和评估功能整合到一个共享模型中，利用失败和次优的交互来改进其动态特性和价值预测。SAIL [189] 采用类似的循环机制，其中视频规划器执行自身生成的计划，并基于所得轨迹进行微调。

这些方法不仅提升了策略，还改进了预测后果和生成未来训练信号的机制。它们的反馈随后可用于更新下一轮使用的策略、资源或课程，从而闭合具身的递归自指（RSI）环路。然而，它们的目标和更新过程仍然保持外部固定，因此仅实现了有限的模型-策略协同演化，而非无限制的递归改进。

4.2.5 迈向递归改进具身智能体

综上所述，这些研究方向表明，具身的递归自我改进（RSI）不仅应提升任务表现，还应优化智能体获取和验证新能力的过程。环境生成器应提出处于智能体能力边界附近的任务；支架系统应整合相关记忆、技能、工具和模型；策略应与环境进行交互；世界模型和评估器应解释由此产生的结果。经过验证的改进应被策略、世界模型或支架系统继承，从而使每次交互循环都能同时提升当前行为和未来的学习能力。

当前系统仅部分实现了这一环。B0 型的回合内反思与重规划已广泛存在。通过基于具身经验的策略和模型更新，已实现 L1 型持续改进 [182, 183, 189]。L2 代表当前的主要前沿，系统在固定评估流程下选择并应用对技能、提示、控制程序及智能体集成的更新 [111, 179, 180]。一些系统通过根据学习器状态生成经验，并在任务间保留有效技能，展现出 L3 能力 [84, 178]。L4 型智能体-环境协同演化也已出现，主要存在于仿真环境中 [65, 174, 175, 177]。ENPIRE [181] 进一步接近有界 L5，允许编码智能体修改策略、训练流程及支持基础设施。然而，其环境接口、评估器与安全约束仍由外部指定。

我们距离具身智能的真正 RSI 还有多远？真正的具身 RSI 要求智能体主动获取有信息量的经验，对整个感知运动系统的结果进行属性分配，并在无法自由重置的物理条件下验证更新。改进必须在任务、环境和具身性之间保持有效，且具备明确的来源、不确定性以及回滚能力。系统还必须学会在不削弱固定安全约束的前提下，改进自身的驾驭与评估过程。当前的方法虽然分别展示了这些能力，但尚未将它们整合为一个安全、持续且自主的真实世界改进环。

4.3 S3：软件工程中的相对强弱指数（RSI）

软件工程为研究持续学习智能体（RSI）提供了异常具体的场景，因为其产品及开发该产品的智能体均以可执行代码的形式表示。一个典型的环始于编码智能体尝试解决某个问题，观察到基于代码仓库的反馈，如测试失败、运行时迹或代码审查意见，并利用这些证据来修改自身的一个持久部分。被修改的组件可能是智能体的技能、 workflow 或实现方式，而被接受的修改将在后续任务中重复使用。

软件工程中的 RSI 面临三个基本挑战。首先，系统必须持续生成针对其当前能力短板的软件任务，同时确保这些任务既可验证又对未来学习具有价值。其次，软件工程的改进必须评估修改是否在当下有效，以及是否提升了系统进一步改进的能力。一种修改可能提升即时基准性能，却导致后续搜索陷入停滞；而暂时表现较弱的变体反而可能产生更优的后代。第三，协调多个组件的演化过程，同时保持外部契约（如评估标准、改进目标）不变，仍是一项挑战。因此，我们围绕三个不断演化的组成部分组织现有工作，包括智能体实现与工具链、软件工程经验与协作，以及改进过程本身。

4.3.1 演进的编码智能体实现与应用

软件工程类重复性应激障碍（RSI）最直接的形式将编码智能体本身视为一个可编辑的软件项目。SICA [190] 允许智能体检查先前版本和基准测试结果，修改自身 Python 代码库，并在下一轮改进中使用选定的版本。由于同一系统既充当软件开发者的角色，又作为被开发的对象，对文件编辑工具、子智能体或上下文管理

的改动，既能提升常规编码能力，也能改善后续的自我修改。然而，其基准、效用函数和沙箱环境仍保持外部固定。

最近的工作将自我修改限制在智能体的 智能体约束（例如，中间件、验证规则和控制逻辑）上。自约束 [23] 利用执行迹来识别反复出现的弱点，要求同一潜在模型提出对其当前约束的小幅修改，并仅在回归测试通过后保留该修改建议。智能体约束工程 [63] 将约束组件暴露为可独立编辑和回滚的文件，将长轨迹提炼为结构化证据，并检查每个修改提议是否产生其预期效果。其分离的演进者角色使该环路相较于自约束更为间接地自我参照，但被接受的约束仍会在后续执行中持续存在。Ouroboros [18] 将这一思想扩展至部署阶段：经过审查的对智能体工具、上下文组装、提示词及核心实现的修改，成为后续工作所使用的运行时环境。这些系统共同展示了版本控制、回归测试与回滚如何将智能体自我修改转变为一种持久且可审计的过程。

4.3.2 不断发展的软件工程经验、技能与协作

一种轻量级的自演化形式改变了编码智能体利用先前开发经验的方式，而无需重写其完整实现。SWE-Exp [60] 将成功和失败的问题解决轨迹都提取到经验库中，并在后续仓库任务中检索相关经验教训。CODESKILL [191] 超越了单纯存储案例，它学习一种管理策略，能够从多个层次提取过程性技能，随着新轨迹的到来更新或移除这些技能，并为未来智能体维护一个紧凑的技能库。在这两种情况下，持久输出都是可复用的开发知识，支持跨任务适应。然而，它们固定的提取和更新过程本身并不会通过使用而变得更好。

演化还可以改变软件团队的组织结构。EvoMAC [192] 将多智能体编码 workflows 表示为角色提示与通信链接构成的网络，然后利用测试反馈和文本反向传播来修改智能体及其连接。这改善了当前程序构建所使用的工作流，但报告的更新是在每个任务的测试阶段发生的。

4.3.3 持续改进过程的演进

最强的系统不会遵循单一的、被接受的编辑序列；它们还会搜索替代的演化路径。达尔文-哥德尔机器 (DGM) [16] 维护一个编码智能体变体的档案，选择先前的智能体作为父代，让它们修改自身的实现，并在可执行的编码任务上评估其后代。保持多条谱系可以防止一次有害的编辑永久阻塞进展，并允许暂时较弱的变体成为有用的过渡步骤。赫胥黎-哥德尔机器 (HGM) [59] 进一步指出，当前基准性能并不等同于产生更优后代的能力。因此，它从智能体的后代谱系中估计元生产力，并将评估资源分配给具有更强长期改进潜力的智能体。

基于种群的方法拓宽了跨谱系可继承的内容。Group-Evolving Agents [193] 允许多个父智能体在生成新群体时共享成功的修改和失败经验，因此有用发现不会局限于单一分支。DarwinX [194] 在冻结模型的基础上演化完整的约束集种群，并仅在新变体扩展任务覆盖范围且不退化先前已解决案例时才予以接纳。HELIX [195] 提出了一种更进一步的模型-约束环，其中约束集的演化改进当前执行过程，并生成用于更新模型的验证轨迹，随后针对变更后的模型重新构建约束集。

4.3.4 迈向递归改进软件工程智能体

综上所述，这些工作表明了一种软件工程中的 RSI 环：开发挑战 → 智能体级修改 → 基于代码库的试验 → 回归感知的选择 → 版本化继承。代码库反馈触发智能体的持续更新，该更新在当前任务和保留

任务上进行评估，随后作为可追溯且可逆的版本被继承，以支持未来的开发与自我改进。

当前系统仅部分实现了这一环。B0 任务局部优化广泛存在，而 L1 持久更新则出现在跨任务经验与技能学习中 [60, 191]。L2 是已建立最充分的层级，智能体在固定目标与评估者下选择对夹具、工具或工作流的更改 [23, 63, 194]。L3 通过自对弈 SWE-RL 和苏格拉底式 SWE 中的学习器条件化任务生成正在涌现 [196, 197]。L4 环境适应仍基本缺失。SICA 与 DGM 通过修改塑造后续自我改进的代码表现出有限的 L5 特性 [16, 190]，而 HGM 引入基于谱系的元选择机制，但并未调整选择规则本身 [59]。软件开发改进器的完整 L5 改进尚未得到展示。

我们离软件工程领域的真正 RSI 还有多远？真正的软件工程 RSI 需要具备稳健的规范、可归因的改进以及跨代码仓库和软件版本的可靠迁移能力。随着测试生成与优化策略变得更具自适应性，用户意图、安全约束、可审计性以及回滚机制必须始终保持外部保护。

4.4 S4: 医疗保健领域的相对强弱指数 (RSI)

作为人工智能应用中最突出的场景之一，医疗健康涵盖诊断、医学影像分析和治疗方案制定等任务。一个典型的闭环 (RSI) 从智能体执行临床任务并获得可验证的反馈（如诊断结果、临床医生修正或临床指南）开始。随后，该智能体持续更新其组成部分（例如临床知识、推理策略、工作流），并将改进内容保留在记忆、技能库或模型参数中，以供后续案例复用。

医疗领域的 RSI 主要面临三个特征的挑战。首先，医疗领域不允许无限制的试错，因为诊断和治疗动作可能对患者造成伤害，因此需要专家、伦理及机构层面的监督。其次，临床反馈往往延迟、异质且混杂，难以将患者结果归因于特定智能体的决策或更新。第三，临床改进具有人群和机构依赖性，因此在一处情景中验证的更新无法安全地继承，除非记录其来源、适用范围和不确定性。这些特征使得医疗 RSI 成为安全反馈获取、可靠信用分配和受控能力传承的问题。相应地，我们围绕三个可能演进的组成部分组织现有工作：临床记忆与知识、临床推理策略，以及医疗工具与工作流。

4.4.1 不断发展的临床记忆与知识

医疗领域中最常见的强化学习系统 (RSI) 形式是智能体的临床记忆演化，其中已完成病例的反馈被转化为可持久化知识，从而影响后续患者的决策。早期系统主要保留病例级别的经验。Agent Hospital [198] 中的 MedAgent-Zero 存储成功的治疗案例，并对失败案例进行反思，以推导出可复用的诊断规则，使模型权重冻结的医生智能体能够在后续的模拟交互中检索这些经验。类似地，Agent Mental Clinic [199] 通过监督模块将精神科医生智能体的诊断结果与标注结果进行对比，并将所得的诊断经验写入分层记忆架构中。MedAgentSim [200] 进一步将医疗记录与经验记录相结合，使诊断智能体能够复用早期模拟医患互动中积累的知识。在这些系统中，一次交互的输出成为智能体对未来病例进行推理的持久上下文的一部分。

更近期的方法侧重于演化记忆知识的结构与可靠性，而非简单累积原始轨迹。DxEvolve [201] 将诊断性遭遇提炼为 诊断认知原语，这些原语编码了信息获取与诊断推理的可复用模式，可在新病例中被检索。GSEM [202] 将经验组织为双层图结构，并利用后续反馈重新校准结点质量与跨经验边的权重，从而同时改变记忆的内容以及哪些记忆被认为适用。Evo-MedAgent [203] 维护了用于回顾性临床回合、程序性启发式规则以及工具可靠性的互补存储。在每个胸部影像学病例后，反思机制会更新这些存储，使后续病例能够继承临床经验与对诊断工具信任度的修订。然而，目前大多数证据仍基于模拟或回顾性病例，

且依赖外部提供的标签，真实世界病例中演化的验证仍然不足。

4.4.2 不断发展的临床推理策略

除了演化临床事实外，医疗智能体还可以演化决定如何获取、解释和整合证据的策略。EvoClinician [204] 通过一个 诊断—评分—演化环来实现这一过程，其中演员（Actor）依次提出问题并安排检查，过程评分器（Process Grader）根据每项动作的临床收益和资源成本进行评估，演化器（Evolver）则利用这些反馈来修改演员的提示和记忆，以便进入下一个病例。因此，继承的演化对象即为一种诊断策略，例如，应优先明确哪个症状或应优先安排哪项检查。

推理演化也可以发生在多智能体协作的层面。EvoMDT [205] 将角色专业化智能体之间的肿瘤学决策进行分解，并利用专家评估和以结果为导向的信号来更新提示、共识权重和检索范围。因此，反馈不仅能够改变单个智能体的推理过程，还能影响后续案例中专家意见的加权与协调方式。在心理辅导领域，PsychAgent [206] 从历史辅导轨迹中提取基于实践的技能，将其加入不断演化的技能库中，并通过拒绝微调进一步内化选定的技能。

4.4.3 不断发展的医疗工具与 workflow

第三类工作通过将成功的执行过程转化为可重用的工具、技能或临床 workflow，扩展了智能体的动作库。MACRO [207] 从一组固定的医学影像工具开始，但会识别经过验证的执行轨迹中的重复性多步模式，将其合成复合工具，并将这些复合体注册为后续影像任务的新高层次动作。因此，该智能体改变的是其可以直接调用的内容，而不仅仅是检索先前的解决方案。SkeMex [208] 在程序化技能层面采用类似方法：它将具有信息量的临床轨迹提炼为通用、任务特定且动作级别的技能，根据环境反馈估算其上下文依赖的效用，并通过一个 读取—写入—评估—管理的生命周期来管理这些技能，从而实现促进、合并、更新或删除条目。在两种情况下，继承机制都通过不断演化的动作空间实现，其中各组件的选择依据是在后续任务中展现出的实用性。

其他系统演化出更复杂的分析 workflow 及其支持组件。TissueLab [209] 允许领域专家检查中间的医学影像结果，并提供修正，以指导主动学习、分类器优化以及病理学、放射学和空间组学任务中的后续 workflow 构建。HealthFlow [210] 将完成的电子健康记录（EHR）分析转化为持久的保障措施、可复用的 workflow、针对数据集的锚点以及代码片段；评估者首先识别执行或方法学缺陷，随后反思者验证、更新或淘汰经验，之后这些经验才可被检索用于后续分析。

这些系统将医疗领域的 RSI 从选择预定义工具转变为构建可复用的流程。然而，由于技术成功或特定病例的临床医生反馈并不能保证在患者群体中均有效安全，因此演化后的 workflow 在跨病例继承前必须经过有效验证并具备可逆性。

4.4.4 迈向递归改进的医疗智能体

医疗领域中 RSI 的最终形态不仅仅是能够生成更优诊断、建议或分析结果的系统，而是一个在保持临床有效性和安全性的前提下，持续提升执行医疗任务能力的系统。每个回合都应产生一个任务结果，以及对智能体记忆、推理策略或动作库的有效改进，使得由此产生的能力能够安全地在后续案例中重复使用并进一步评估。

Table 9 早期在 Theseus 环境阶段的工作区实验。(a) 清洁环境与噪声环境的初步对比：在 30 个工作区任务（共 1,280 个评估标准）上，八个前沿模型-工作区配置的通过率（%），比较了清洁工作区与含噪声工作区的表现。(b) 生产力研究：五个固定工作区-模型配对在 30 个任务（共 547 个评估标准）上的评估得分，比较了原始工作区与重构环境（包含一个集合映射和一个事件日志的共享包）的表现；“环境任务”报告任务层面的胜/平/负情况，“评估标准反转”统计从失败变为通过与从通过变为失败的评估标准数量。

(a) Clean-versus-noise pilot — clean vs. noise-laden workspace					
Model	Harness	Clean	Noise	Δ (pp)	
DeepSeek-V4-Pro	DSH	98.2	46.6	+51.6	
DeepSeek-V4-Flash	DSH	90.1	63.7	+26.4	
GPT-5.6-Sol	Codex	89.1	54.1	+35.1	
GLM-5.3	ClaudeCode	88.8	57.2	+31.6	
Kimi-K3	ClaudeCode	87.6	53.7	+33.9	
Grok-4.6	ClaudeCode	84.7	63.0	+21.7	
GPT-5.6-Luna	Codex	84.6	38.1	+46.5	
Muse-Spark-1.2	ClaudeCode	84.5	53.3	+31.2	

(b) Productivity-workspace study — bare vs. reconstructed environment						
Harness	Model	Bare	Reconstructed	Δ (pp)	Env. tasks	Rubric flips
Codex CLI	GPT-5.6-Sol	60.51%	92.50%	+31.99	24/3/3	190/15
Codex CLI	DeepSeek-V4-Flash	54.30%	84.83%	+30.53	24/3/3	201/34
Claude Code	DeepSeek-V4-Flash	61.06%	79.71%	+18.65	23/2/5	140/38
PI	DeepSeek-V4-Flash	62.52%	89.03%	+26.51	22/6/2	155/10
PI	GPT-5.6-Sol	50.27%	89.95%	+39.67	23/2/5	240/23

B0 级的个案反思与答案优化已广泛应用，但并未持续改变未来行为。L1 级的持久继承已通过案例记忆、诊断规则和可复用的临床知识得以验证 [199–201]。L2 代表当前最强的前沿：系统能够诊断故障，并自主选择或应用对提示、记忆、推理策略、工具、工作流及协调机制的更新，而其目标、评估器和部署门限保持外部固定 [202–205, 207, 208, 210]。少数系统表现出类似 L3 的特性，利用当前不确定性或累积经验来影响后续证据获取、模拟遭遇或专家标注 [198, 209]；然而，其患者群体、任务分布和入院标准仍主要由外部指定。EvoPatient 进一步探索了类似 L4 的适应性，通过模拟患者与医生的共同演化 [211]，但模拟器的临床真实性和评估标准仍由外部决定。从真实纵向结果实现端到端的 L4 适应，以及临床改进者自身的 L5 演化，仍基本未被探索。

我们离真正的医疗领域 RSI 还有多远？ 迈向更高层次的医疗领域 RSI，主要需要可靠地归因于患者的长期结果、在不进行无限制临床探索的前提下安全获取改进反馈，以及在不同人群和机构间选择性地传递更新。同样重要的是，必须建立一种受控的能力-承诺流程，确保每一次持续的更新都经过临床验证，进行前瞻性监控，可追溯到其支持的证据，并且在其假设或安全保证不再成立时能够被撤销。

5 产业格局与初步实践

5.1 忒修斯：环境—数据—模型协同演化用于遥感智能

忒修斯将环境—数据—模型的协同演化定位为迈向下一代 RSI 智能的路径。其核心理念是：环境使知识可获取，使动作可验证；任务执行产生学习所需的证据；模型的改进则拓展了可探索问题的值域。在这一愿景下，每一阶段的工作都应后续阶段贡献可复用的能力，使现实世界的实践持续成为智能增长的源泉。

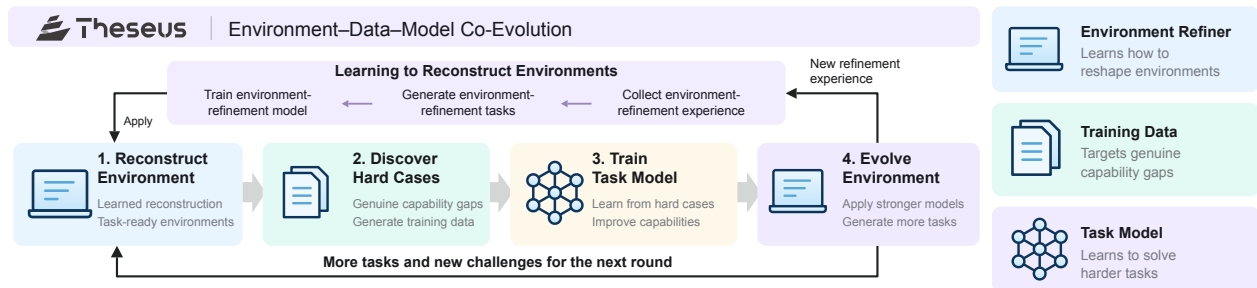


Figure 10 忒修斯提出的四阶段共演化环：重构环境，揭示真实的能力差距并生成训练数据，训练任务模型，并迭代环境以产生更多任务。上层环从优化经验与生成的任务中学习环境优化模型，而后续的迭代则提供新的经验以实现进一步优化。

如图 10 所示，Theseus 通过四个步骤实现这一愿景。首先，将精炼经验转化为环境精炼任务，以训练环境精炼模型。其次，智能体利用重构的环境识别真正的任务求解困难，并生成有针对性的训练数据。第三，这些数据用于训练任务模型，以提升其能力。第四，更强的模型支持进一步的环境迭代以及更多任务的生成。后续的迭代过程既产生新的任务-训练数据，也产生新的环境精炼经验，从而将任务求解进展与未来学习条件构建的进展相联系。

早期结果为该方向在环路的工作空间阶段提供了初步的实证基础。一项试点研究在 30 个源自 Workspace-Bench [212] 的工作空间任务上，对八种前沿模型—工具链配置进行了测试，由 1,280 条评分标准进行评估，首次测量了对工作空间质量的敏感性：相较于噪声密集的工作空间，干净的工作空间使所有八种配置的通过率提升了 21.7 至 51.6 个百分点（表 9a），表明工作空间状态而非模型能力本身，可能限制智能体的表现。基于这一观察，后续的一项生产力研究探讨了在保持底层模型和工具链不变的情况下，丰富环境是否能提升任务表现：五组固定的模型—工具链组合在裸工作空间与包含集合地图和事件日志的重构环境中分别完成 30 项任务，总计由 547 条评分标准进行评估；重构环境使所有测试配置的综合评分提高了 18.65 至 39.67 个百分点（表 9b），尽管少数任务出现了范围蔓延的退化现象，即智能体生成的修改超出任务要求。团队还已完成一个可复用的文件验证模块的训练。这些早期结果标志着协同演化环路的第一步，后续各轮将逐步扩展环境重构、数据生成和模型训练，形成持续累积改进的循环。

5.2 Lark：构建可靠企业级 RSI 的数据基础

图 11 总结了 Lark 的数据基础环。Lark 从一个常被忽视的前提出发来应对 RSI：在智能体能够自我改进之前，必须存在一个持续演化的数据基底，以支撑改进的实现。在企业协作中，文档、消息、会议和任务持续生成新的信息，而智能体则消耗这些数据并产生新的交互迹。因此，Lark 将数据生产、质量评估和失败归因视为其 RSI 流水线的基础。核心目标不仅仅是积累更多数据，而是确保每次迭代都有新的经验可供学习，有可靠的准则来判断进展，并具备足够细粒度的诊断能力以确定应进行哪些调整。

企业知识图谱。一个典型的例子是企业知识图谱的构建与持续优化。分散在文档、消息和会议记录中的信息被关联为实体、事件、人员以及时间关系，使智能体能够回答需要跨源推理的问题。由于图谱错误会直接传播到下游答案中，因此图谱构建本身被视为一个迭代的数据质量问题。Lark 通过批量图谱构建和委托授权检查来解决可扩展性问题，同时持续评估基于图谱的检索是否能提升下游问答系统的性能。在内部评估中，与基于 RAG 的基准相比，使用其基于图谱的流水线后，人工评分的任务可用性从 52% 提升至 65%，而自动化评估的可用性也从 47% 提升至 56%。

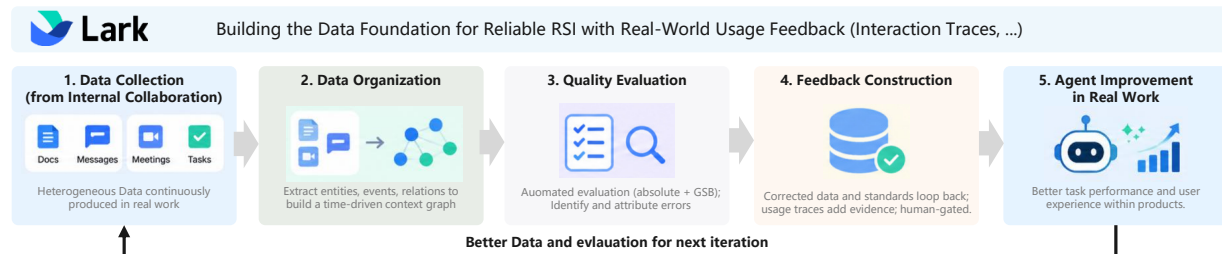


Figure 11 Lark 针对 RSI 的数据基础环路，通过持续地结构化、评估和优化协作数据，将其转化为高质量的知识与训练信号，同时将现实世界中智能体的使用产生的新证据反馈至后续的改进循环中。

自动化数据质量评估。另一个例子是一个自动化数据质量评估系统，作为后续改进的测量层。该系统不完全依赖昂贵且不一致的人工审查，而是首先应用绝对判断来识别明显不可用的输出，然后使用对比性的 GSB 评估来决定是否应提升新系统版本。重要的是，评估与明确的故障归因相结合。不良案例被分类为具体问题（例如时间不一致、查询扭曲或源质量差），从而使每一种失败都对应一个具体的改进方向。例如，对时效性敏感的问题通过查询时间过滤加以约束，以避免使用未来信息；低质量的合成查询在重新进入评估流水线前会被重写。初始的自动化评估器与人工判断的吻合度约为 84%，其中大多数分歧源于自动化评估器比人工评审更严格，为后续校准提供了明确信号。

从 RSI 的角度来看，这些试验的意义在于为未来智能体的改进创造了反馈结构。企业活动持续产生新的图数据和交互迹，自动化评估确定哪些输出和数据是可靠的，错误归因识别出流水线失败的位置，随后修正后的数据和评估标准成为后续迭代的输入。由此形成的循环将生产数据从被动资源转变为持续进化的改进基础。与此同时，Lark 当前的做法仍刻意保持人工管控：智能体完成大部分重复性的数据处理和评估工作，而人类则保留对标准制定、关键样本审查以及重要变更审批的责任。

5.3 Humanlaya：面向交付的 RSI 数据质量保障

Humanlaya 为基础模型开发者提供训练和评估数据，其生产工作流以复杂的多文件任务包为核心，包含任务描述、输入附件、参考答案、评分标准以及可执行或过程性验证要求。在这些情景下，质量缺陷往往较为隐蔽：单个数据项在孤立状态下看似正确，却与任务规范、评分逻辑或下游评估协议存在不一致。随着任务类型和客户需求的演变，反复修复个别有缺陷的样本已不足以应对问题，质量控制系统本身必须从重复出现的失败中学习。因此，Humanlaya 将其数据质量智能体及其相关的规则、提示、few-shot 示例和技能视为持续改进的目标，包括内环和外环，如图 12 所示。

以数据为中心的内环。内环在当前交付批量内运行，专注于修复正在生成的数据。每个任务包首先经过质量检查器，该检查器检测内容质量和跨组件一致性，随后通过修复器对已确认的缺陷进行修复，最后经过交付检查器验证格式、结构和交付约束。若检查失败，则触发重复的本地化、修订和重新验证，直至交付。例如，如果若干可独立评分的结果被错误地合并为单一粗粒度评分项，检查器可能识别出该评分项原子性不足，并要求重构后才接受该任务包。此环路能够改进当前产物，但其本身并不构成系统性的持续改进。

以改进为中心的外环。外环在交付批次间运行，是核心的 RSI 机制。交付后，系统会聚合诸如内部评审、客户反馈和下游使用等证据。一个智能体分析这些案例中的反复出现的原因，并提出对质量控制系统版本化的修改建议，包括精炼器指令、提示词、few-shot 示例以及可复用技能等组件。候选更新会在

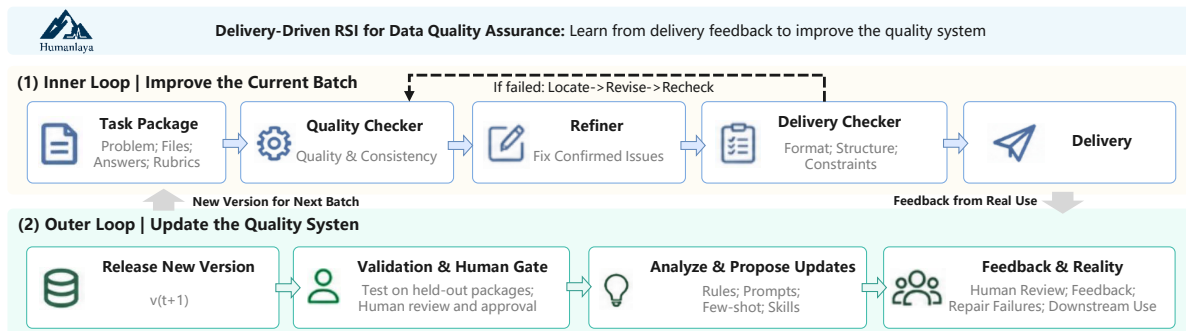


Figure 12 人类学家的以交付为导向的数据质量保障环。

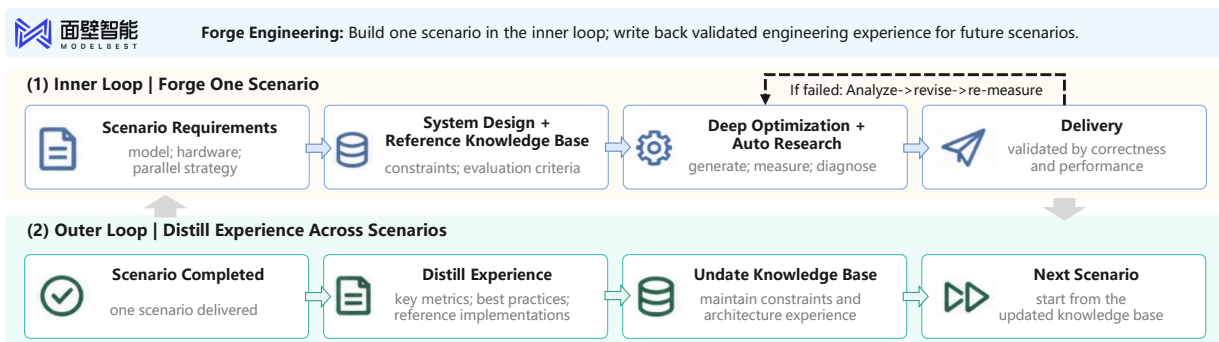


Figure 13 ModelBest：将工程锻造视为一个两级的 RSI 环。

未参与修改生成的任务包上进行评估，并在晋升前由人工额外审查。一旦获得批准，新的质量系统版本将取代旧版本，并用于后续的生产批次。新任务随后生成新的证据，再次进入下一个改进循环。

一个典型案例涉及一份三页的扫描 PDF 文件，提取工具无法解析该文件，导致质量检查器将附件标记为不可用。人工审查显示，扫描本身清晰，问题源于将提取失败误判为文档质量差。因此，系统生成新的决策规则和 few-shot 示例，以区分这两种情况。经过保留样本验证和人工审核后，更新内容被纳入下一版本系统，并用于未来任务。

内环修复当前任务包，而外环则更新用于检查和修复未来任务包的方法。因此，Humanlaya 代表了一种由生产反馈驱动的、实际可行的层级式 RSI 形式。

实测改进。 Humanlaya 报告了在四轮反馈驱动更新后，V0 与 V4 的内部对比结果。在使用相同的基础模型、工具和处理预算的前提下，两个版本均在 600 个未参与更新过程的任务包上进行了评估。经过自动化修复后，包含关键缺陷的任务包比例从 9.0% 下降至 3.7%，同时平均每项任务的人工处理时间从 48 分钟降至 27 分钟。

在当前模型能力仍快速发展的阶段，某些任务尚无法由智能体完全自主完成。Humanlaya 因此提供了一个将 RSI 与人工监督相结合的实用范例：智能体执行大部分自我改进过程，而人类则在关键阶段提供部分真实值、审查和最终批准，而非允许不受约束的自我改进环。

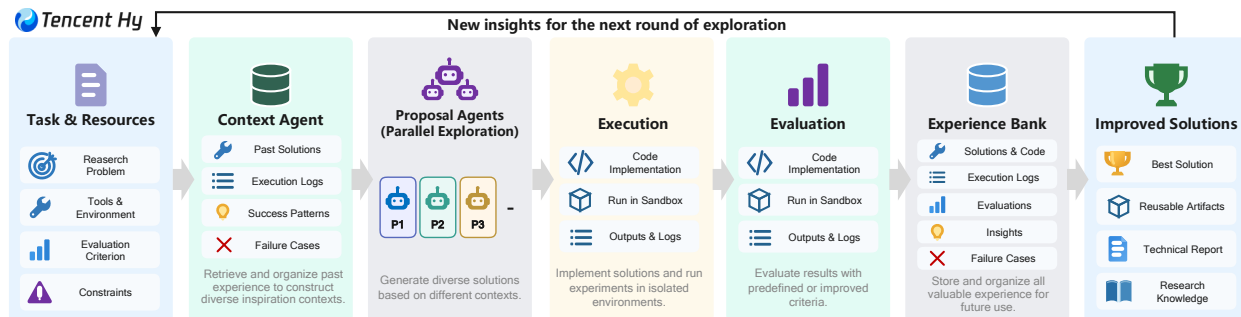


Figure 14 腾讯混元 Hyra 中的经验驱动型自我改进。上下文智能体整理先验经验，以指导并行的解决方案提出，这些方案在隔离环境中执行并接受评估。生成的代码、执行日志和评估反馈将被保存至经验库，用于指导后续探索，并在需要时支持评估器的优化。

5.4 ModelBest: Zero-Human 工业人工智能工程

ModelBest 旨在通过自动化人工智能工程来推动 RSI 的发展，其前提为人工智能工程可能比开放式人工智能研究更早实现自主化。工程任务（例如，实现训练框架、优化内核、配置并行性、运行基准测试）需要大量的人力投入，但通过执行可获得相对廉价且客观的反馈。为此，为了在无需人工干预的情况下生成可投入生产的实现，ModelBest 提出了 *Forge Engineering*，这是一种端到端范式：一个 AI 系统以目标模型、硬件平台和并行化要求作为输入，从一个初始为空的代码库开始，构建、测试并不断优化实现。图 13 展示了其项目内最优化与跨项目经验复用所形成的双层环结构。

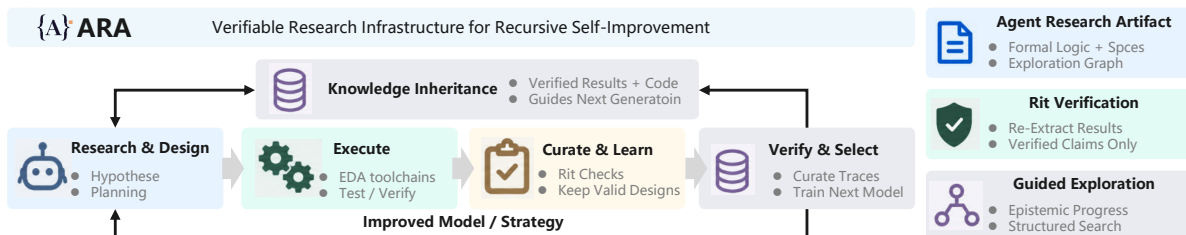
工业级 RSI 环。在目标场景中，系统将受约束的架构设计与自主的低层级最优化相结合。人类与人工智能共同维护一个知识库，其中包含规格说明、评估标准以及积累的工程经验。智能体首先利用该知识建立合理的高层架构，并对搜索空间施加约束。随后，AutoResearch 环持续生成实现方案，测量正确性与性能，诊断失败原因，并修复瓶颈问题。成功的最优化成果，包括 GEMM 和 FlashAttention 等内核，逐步集成到主执行路径中。这种设计在保持全局架构约束稳定的同时，允许在实现层面进行激进的自主优化。

在不同场景中，成功的和失败的工程经验都会被反馈到共享知识库中。一个项目中的性能度量、有效的最优化策略、参考实现以及失败规则，会成为后续项目的起点。因此，相同的工程流程可以在不同的模型、硬件平台和工作负载之间复用。ModelBest 报告指出，该范式已在预训练框架、算子库、强化学习基础设施、推理引擎、微调、压缩、量化以及多个硬件生态系统的边缘部署中得到应用。

实测提升。从一个空目录以及参考脚本和模型规范开始，ForgeTrain [213] 在大约 8 小时内生成了一个与 H100 上 Megatron-LM v0.15 相当的预训练框架，并在 1.5–2.5 天内报告实现了超越，相比之下，手动开发达到类似水平预计需要 3–5 名工程师工作 6–12 个月。报告的模型浮点运算利用率对于 MiniCPM4-0.5B 从 40.1% 提升至 44.1%，对于 8B 模型从 47.0% 提升至 50.9%。ForgeStencil [214] 将相同方法扩展至科学计算内核，公司报告其相对于公开的最先进实现获得了 1.15–1.9× 的加速比，端到端中位加速比达到 1.41×。

Table 10 递归方法与 hyra-1.0 在人工智能研发基准上的比较。

Benchmark	Task	Metric	Recursive	hyra-1.0
NanoChat Autoresearch [215]	Model training	Validation BPB ↓	0.9109	0.9015
NanoGPT Speedrun [216]	Model training acceleration	Time to 3.28 loss ↓	77.5 s	76.4 s
SOL-ExecBench [217]	GPU kernel optimization	Mean SOL ↑	0.754	0.771[†]

**Figure 15** 智能体原生研究实验室的智能体原生 RSI 环，实现可验证的工程发现，融合引导探索、确定性验证、知识继承与下一代学习，支持研究周期中的持续改进。

5.5 腾讯混元：基于体验的自我优化

腾讯混元的 Hyra（混元研究智能体）在以性能为导向的研究与工程任务中探索 RSI。图 14 描绘了其基于经验的搜索环。其设计遵循一种刻意轻量化的理念：与其编码日益复杂的由人类设计的工作流，Hyra 为智能体提供广阔解空间，并反复利用实验证据来引导后续搜索。给定一个任务后，系统将持续探索，直至决定终止或耗尽其资源预算，返回过程中发现的最佳解。

核心组件是经验库，它保存的状态信息比传统文本记忆丰富得多（例如，先前的解决方案代码、生成的成果、执行日志、得分以及评估者反馈）。上下文智能体将这些证据重新组合成多样的启发式上下文，而多个提案智能体则异步地消费这些上下文，构建新的解决方案，并在全新的隔离沙箱中执行。由此产生的成果和评估结果被写回经验库，使后续的提案能够直接复用、组合或规避早期尝试中发现的模式。

Hyra 进一步将这一思想扩展到评估本身。对于开放式任务，当初始评估者存在不完整或可被利用的情况时，可以利用累积的搜索经验来改进评估机制——例如提高其粒度、强化比较基准或关闭奖励欺骗的漏洞——然后在改进后的准则下继续后续搜索。这一点对 RSI 尤其重要，因为持续更新不再仅限于获得更优的解决方案：决定未来哪些解决方案被视为改进的机制本身也可以发生变化。

实测改进。如表 10 所示，发布的配套资源报告了在 AI-for-AI 和 AI-for-Science 任务中的改进。例如，Hyra 在 nanochat AutoResearch [215] 上的验证 BPB 为 0.9015，优于之前引用的最佳结果 0.9109；在 nanoGPT Speedrun [216] 上耗时 76.4 秒，优于之前的 77.5 秒；在 SOL-ExecBench [217] 上的得分为 0.771，优于之前的 0.754。作为工业界 RSI 实践，Hyra 突显了保留可执行经验而非仅保留最终答案的价值。

5.6 智能体原生研究实验室：可验证的机器人系统研究基础设施

实现持续的递归自我改进不仅需要日益自主的研究智能体，还需要一个基础设施，以确保研究经验能够可靠地记录、验证、传承并跨代重用。智能体原生研究实验室专注于这一基础架构层，如图 15 所示。

- **(1) 以人为本与智能体原生的研究成果。**传统科学论文是为人类交流设计的，会丢弃自主智能体继续

先前工作所需的大批操作状态（例如，失败的实验、中间证据、可执行规范和实现细节）。对于 RSI 而言，这种信息丢失直接削弱了跨代知识积累。

为解决这一问题，他们提出智能体原生研究成果（ARA）作为传统科学文档的可执行替代方案。ARA 共同保存了正式的科学逻辑、可执行代码与规范、成功及被放弃分支的探索图，以及原始实证证据。因此，后续智能体不仅能够检查结果是如何获得的，还能了解哪些备选方案失败，以及实际使用了哪些假设或参数。实验结果显示，使用 ARA 在问答任务上的准确率达到了 93.7%，优于使用传统论文的 72.4%；同时，RE-Bench 复现成功率从 57.4% 提升至 64.4%。由此可见，该研究的重要贡献不仅在于更优的文档记录，更在于为后续研究周期提供了更丰富的继承基础。

• **(2) 针对研究主张的确定性验证。**此外，随着自主智能体以机器速度生成实验和主张，验证而非生成成为主要瓶颈。其 rit 协议将实证主张直接锚定于执行迹，从日志中重新提取报告结果，而分析性主张则可通过如 Lean 4 等形式化验证系统进行检查。只有通过这些可被机器验证的关卡的主张才会被纳入共享研究状态。这防止了幻觉或错误报告的结果被后续生成过程递归继承并放大。同时，它们还通过源自人类科学推理的信念进展引导和结构化先验，增强了基于稀疏结果的最优化。智能体不仅被奖励最终基准得分，更被鼓励选择能够减少不确定性并揭示问题有用结构的实验。此举旨在提升在巨大假设空间中的研究效率，避免朴素爬山法反复利用熟悉参数却无法产生深层理解。

测量到的改进。这些思想在硅设计中得以实现，其中反馈环路异常快速且具有确定性。自主智能体生成可综合的 SystemVerilog 代码，构建测试平台，并提出微架构修改建议；候选设计随后使用工业级 EDA 工具进行综合、布局布线、时序分析和形式等价性验证。无效设计会自动被过滤，而经过验证的实现、执行迹、失败分支以及更优的功耗-性能-面积轨迹则被保留下来，作为后续代际训练和研究的证据。在 Chip-Bench 上，他们报告了 5.8416 的 Level-3 CPU 优化得分，其设计在相同模型基础上比标准智能体基准快 2.7%，面积缩小 21.5%，同时通过了全部 97 个有向测试和 350 个随机差分程序。

6 挑战与未来方向

前几章展示了在自动化执行改进、策略选择、经验获取、部署适配以及改进机制修订方面的进展。它们还表明，更高的自主性、持久的能力提升以及更有效的改进过程是三个不同的属性。因此，推动本次综述的三个瓶颈目前仅得到部分解决：基础模型开发需要大量资源和协调；可扩展学习依赖于有用的经验和可靠的验证；已部署系统需要持续努力进行诊断、验证和更新发布。基于上述自主性层级、四种应用模式及工业实践，我们识别出八个研究方向，以将这些局部进展整合为持续的递归改进。

跨组件诊断与协同改进。从 L1 到 L2 的转移暴露出一个诊断问题：观察到的失败很少能明确指出应更改的组件。错误答案可能源于有缺陷的源数据、缺失的上下文、工具接口、模型或评估者。第 5.3 节中的人类案例使这一分歧具体化：一次失败的提取最初被当作文档质量差的证据，而正确的干预措施实际上涉及质量检查器的决策规则。在科学和具身智能中，同样的问题也延伸至实验协议、感知、控制和环境条件。随着多个组件同时调整，改变其中一个也可能使另一个所依赖的假设失效。

未来系统应将诊断转化为可检验的干预假设，并利用受控比较来估计个体效应及相互作用。选择性组件冻结、目标性消融以及依赖关系的显式记录有助于区分有效的修复与上游缺陷的补偿。随后，协调搜索应在数据、模型、资源和环境之间分配努力，以适应约束条件的变化。第 5.1 节中的 Theseus 工作区研究说明了将环境改进与任务模型变化相分离的重要性；报告的组件级提升尚需建立完整的学习循环。研究目标是确定在何种条件下何种干预能够改善整体系统，以及所得知识是否能指导后续的改进决策。

学习器条件化经验获取与可靠学习信号。 L3 分析区分了经验的三个属性：是否有效、对当前学习器而言难度如何，以及从中学学习能否产生持久收益。AZR 采用可执行检查结合学习器依赖的任务提议，而 R-Zero 则使用求解器一致性作为难度的代理 [99, 103]。仅正确性或估计难度本身并不能确立学习价值。自适应课程可能反复选择模糊任务，强化评估者错误，或集中于一个进展易于衡量的狭窄区域。此类错误可能同时影响学习器及其决定下一步经验的状态。

未来的获取机制应在多轮中估计学习价值，同时保持先前获取能力的有效性、多样性和覆盖范围。这包括从现有数据中选择、生成新任务、寻求交互，以及在内部判断不足时请求更强的外部反馈。一个核心挑战是决定昂贵的验证是否值得其成本，以及不确定的经验应如何影响后续训练。评估应允许所获取的分布存在差异，同时匹配数据源访问和总获取与学习预算。将学习器状态输入冻结于获取机制，或用学习器无关的调度替代自适应选择，可以检验学习器条件化是否能提升迁移效果，超越额外训练的影响。

持续状态管理与可靠复用。 L4 系统继承参数、记忆、技能、工具和利用代码，但保留一个成果并不意味着后续智能体能够从中获益。L4 分析揭示了激活和执行两个方面的失败：相关技能可能从未被检索到，或智能体虽已检索到却未能正确执行。累积效应带来了额外的挑战。库漂移现象表明，不断扩大的技能库可能降低检索效率并阻碍改进，而过于激进的淘汰策略同样有害 [152]。因此，持续性要求管理保留变更的持续适用性及其相互作用。

未来的工作应将继承的成果与其支持的证据、适用条件、依赖关系以及对后续任务的观测影响相联系。如 HDSO 的成对评估等入学测试提供了一个起点 [106]，但随着任务分布、执行者和周围系统的改变，验证工作必须持续进行。研究应分别衡量更新质量、活性值、忠实使用情况以及下游收益，然后探究在何时合并、修改、退役或重新验证这些成果。跨模型复用引入了进一步的问题：对作者有用的技能可能对不同的执行者不适用。解决这些问题将把成功的更新生成与跨会话和继承者的可靠能力迁移联系起来。

领域特定反馈下的受控适应。 四种应用场景表明，通用改进环需要不同的验证策略。在科学领域，一次失败的试验可能反映假设、实验流程或仪器问题，证据必须保留其假设和不确定性。在具身智能领域，当前策略决定了可观测的状态，而物理试验会消耗资源，并可能产生不可逆的后果。在软件领域，可执行反馈存在，但通过测试仍取决于规范和测试覆盖范围。在医疗领域，结果具有延迟性和混杂性，一个在某一群或机构中验证的更新可能无法推广到其他人群或机构。这些差异既限制了系统自主学习的能力，也限制了支持部署的证据范围。

未来的研究应开发能够区分瞬态故障与持续性局限的更新策略，并选择与所提变更相适应的验证流程。仿真、回溯重放和沙箱执行可支持初步筛选，随后在预期运行条件下进行监督式或分阶段评估。监控应检验效益是否在需求、人群、工具和环境发生变化时仍能持续存在。恢复软件检查点无法逆转物理或临床后果，因此在这些场景中，发布前验证至关重要。对于具有重大影响的发布、访问权限及安全约束，人类的控制权必须保持明确。当前未解难题在于如何在控制错误跨未来交互传播的同时，提升有益的适应能力。

可信的改进机制演化。 在 L5 层级，对改进器、评估器或研究策略的更改会改变后续后代的生成和接受方式。这导致了一个耦合的验证问题：更强的求解器可能暴露其评估器的弱点，但更改评估器也可能使得分不可比较或引发奖励利用。RQGM 通过在一轮内冻结其评估器，将替换项与一个独立的基准进行验证，并重新审视受替换影响的得分来解决该问题的一部分 [21]。Hyra 中描述的评估器优化提出了更

广泛的问题：随着搜索的推进，自适应测量如何保持可信度？

未来系统应区分可编辑的内部反馈机制与独立维护的接受标准，并保留将每次机制修订与后续决策相联系的证据。受控比较应在归因于两者共同演化之前，分别评估候选生成和评估的变化。A-Evolve-Training 展示了在固定高层目标下的策略继承 [17]；这种策略是否能在其演化研究情景之外实现迁移，仍有待验证。开放性问题包括如何在反复自适应访问下维持独立评估、检测协调一致的提议者—评估者错误，以及在不丢失有用经验的前提下逆转有害机制变化。修订操作研究优先级无需转移对系统整体使命的控制权。

继承改进能力的长周期评估。结构上的 L5 与有效性的 L5 之间的区别应指导评估。结构性证据确立了修改后的机制已被继承并被调用；而有效性则要求证明该机制能够产生或选择出更优的后续改进。这仍然是一个实质性的实证空白。例如，本综述中回顾的 AIDE² 实验并未在将进化得到的夹具作为外层改进器时，建立统计上显著的效率优势 [94]。软件谱系研究进一步支持将当前任务表现与产生有用后代的能力分开考量，因为暂时较弱的变体可能有助于后期进展。

未来的基准应比较来自可比初始系统的原版与修订版机制，并在匹配的总预算下进行对比，包括开发和评估机制的成本。将演化后的机制固定在新任务上，可以检验其改进策略是否具备迁移能力，如 Hyper-Agents [93] 所示。更长期的研究应报告多次运行中的完整轨迹，包括被拒绝的更新、回归、恢复、保留的能力以及资源使用情况。受保护的参考任务可保持可比性，而新任务和不熟悉约束则用于测试迁移与发现能力。这些协议应确定继承性变化是否影响后续改进容量，何时收益达到平台期，以及表面加速是否在计入额外搜索与评估努力后依然成立。

资源感知的改进与人类协作。推动 RSI 的效率目标需要考虑整个改进过程。更快的内核、更好的训练方案或更长时间的无人值守运行，仍可能需要大量的候选生成、评估、基础设施维护以及人工审查。Meta 的回归分析工作流保留了对所提更改的工程审查 [5]；Humanlaya 同样保留了人工验证，并报告处理时间与质量。这些实践表明，自主性应与所需的人力投入量和类型一同评估。更强的基础模型和更多的并行试验可能会进一步混淆不同改进策略之间的比较。

未来的工作应研究如何在诊断、经验获取、候选搜索、验证和部署监控之间分配总体预算。廉价的筛查可以减少昂贵的试验次数，前提是其预测结果与独立结果保持校准。重复使用已验证的工具和先验失败案例可减少重复工作，但其维护成本也必须计入。评估应报告达到已验证能力目标所需的时间和总体代价，同时包括评审工作量和返工情况。自适应停止是另一个研究重点：系统应通过预期学习价值和不确定性来证明继续实验的合理性，并明确设定暂停、升级至人工或终止无效搜索的条件。

可复现的跨轮次继承基础设施。后续系统需要能够访问改进方法的获取过程，包括未成功的备选方案以及其证据成立的条件。工业案例揭示了互补的需求：Lark 强调时间锚定的数据和失败归因；Humanlaya 保留版本化的质量体系更新；Hyra 存储可执行的经验；Agent-Native Research Lab 提出了保存代码、规范、探索历史和实证迹的科研成果。这些实践推动了跨轮次携带科研证据的共享基础设施的发展，而此类继承的完整纵向效益仍有待确立。

可重复使用的工件格式应连接父状态、提议的变更、驱动证据、评估配置、接受决策及后续使用。版本化数据、环境和评估者将使结果更易于重放，并明确在更新后哪些比较仍保持有效。确定性重新提取可用于验证报告的度量值，形式化验证可建立相对于规范的属性；但单独任一方法都无法证明实验或规范捕捉了预期能力。因此，公开评估应将可检查的工件与重放研究相结合，分别报告结果溯源、工件可用

性以及独立复现情况。当数据涉及机密时，可共享的评估子集和受控重放接口可在保留可检查范围限制的同时支持部分验证。

在这些方向上，真正的 RSI 进展应通过可重复、可归因且可转移的改进来体现，即在提升能力方面实现持续进步。决定性证据是，遗传性变化有助于后续轮次在明确的资源和权限约束下，获得更有价值的经验、发现更优的干预措施，或更有效地验证继承者。

7 结论

在本报告中，我们提出了一条以自主性为中心的递归自我改进路线图，涵盖五个阶段：改进执行、策略选择、经验获取、环境适应以及递归元改进。以改进环为分析单元，我们明确了系统发生何种变化、后继系统继承了什么内容，以及哪些决策仍由人类控制。我们将该路线图与科学发现、具身智能、软件工程及医疗健康等领域相联系，揭示了反馈可得性、验证成本和部署约束如何在各领域塑造向递归自我改进（RSI）迈进的进程。产业实践和初步的实证发现进一步将该路线图建立在已验证的能力和当前局限性之上。需要进行长期评估，以确定这些机制在明确资源约束和人类监督下是否能够实现可迁移的收益。最终，RSI 的潜力在于使每一世代的人工智能都能让未来的改进更加可靠、高效，并促进新发现的产生。

References

- [1] Moonshot AI, “Kimi K3: Open frontier intelligence,” Kimi Research Blog, Jul. 2026. [Online]. Available: <https://www.kimi.com/en/blog/kimi-k3>
- [2] Alibaba Cloud, “Alibaba unveils Qwen3.8-Max: Its largest and most capable flagship model to date,” Alibaba Cloud Press Release, Aug. 2026. [Online]. Available: <https://www.alibabacloud.com/en/press-room/alibaba-unveils-qwen3-8-max>
- [3] OpenAI, “GPT-5.6: Frontier intelligence that scales with your ambition,” OpenAI Product Announcement, Jul. 2026. [Online]. Available: <https://openai.com/index/gpt-5-6/>
- [4] I. Moshkov, D. Hanley, I. Sorokin, S. Toshniwal, C. Henkel, B. Schifferer, W. Du, and I. Gitman, “Aimo-2 winning solution: Building state-of-the-art mathematical reasoning models with openmathreasoning dataset,” *arXiv preprint arXiv:2504.16891*, 2025.
- [5] Meta Engineering, “Capacity efficiency at meta: How unified ai agents optimize performance at hyperscale,” <https://engineering.fb.com/2026/04/16/developer-tools/capacity-efficiency-at-meta-how-unified-ai-agents-optimize-performance-at-hyperscale/>, Apr. 2026, published April 16, 2026; accessed September 9, 2026.
- [6] T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. Fishman, M. Aljubei, P. Thacker, L. Fauconnet *et al.*, “Gdpval: Evaluating ai model performance on real-world economically valuable tasks,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 24 005–24 040.
- [7] L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi *et al.*, “Humanity’s last exam,” *arXiv preprint arXiv:2501.14249*, 2025.
- [8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 54 107–54 157.
- [9] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” *arXiv preprint arXiv:2406.12045*, 2024.
- [10] M. Ferrari, P. Tillet, A. Ibrahim, J. Gershenson, and S. Coffey, “How GPT-5.6 fuses frontier intelligence with frontier efficiency,” OpenAI Engineering, July 2026, published July 29, 2026; accessed September 3, 2026.
- [11] S. Jeong, M. Kim, and T. Kim, “Agentic neural architecture search,” *arXiv preprint arXiv:2607.07984*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.07984>
- [12] DeepSeek-AI, A. Liu, A. Mei *et al.*, “Deepseek-v3.2: Pushing the frontier of open large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.02556>
- [13] Anthropic, “How we built our multi-agent research system,” 2025. [Online]. Available: <https://www.anthropic.com/engineering/multi-agent-research-system>
- [14] J. Schmidhuber, “Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements,” *arXiv preprint cs/0309048*, 2003. [Online]. Available: <https://arxiv.org/abs/cs/0309048>
- [15] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai, “Self-taught optimizer (STOP): Recursively self-improving code generation,” in *Conference on Language Modeling*, 2024. [Online]. Available: <https://arxiv.org/abs/2310.02304>
- [16] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, “Darwin gödel machine: open-ended evolution of self-improving agents,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 104 223–104 294.

- [17] Z. Shi, B. He, Y. Sang, H. Lu, and B. Dumoulin, “A-evolve-training: Autonomous post-training of a 30b model,” *arXiv preprint arXiv:2606.20657*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.20657>
- [18] A. Razzhigaev, A. Gritsaev, A. Kaznacheev, N. Dragunov, R. Yampolskiy, and A. Kuznetsov, “Ouroboros: A self-developing frontier coding agent with reviewed core evolution,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.08311>
- [19] X. Yin, X. Wang, L. Pan, L. Lin, X. Wan, and W. Y. Wang, “Gödel agent: A self-referential agent framework for recursive self-improvement,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025. [Online]. Available: <https://aclanthology.org/2025.acl-long.1354/>
- [20] J. Wen, L. Qiu, J. Benton, J. H. Kirchner, and J. Leike, “Automated weak-to-strong researcher,” Anthropic Alignment Science Blog, 2026. [Online]. Available: <https://alignment.anthropic.com/2026/automated-w2s-researcher/>
- [21] A. Iacob, A. Jovanović, W. F. Shen, D. Burkhardt, M. Kurmanji, N. Tastan, L. Sani, N. A. E. Venanzi, A. Odonnat, Z. Cao, B. Marino, X. Qiu, and N. D. Lane, “The red queen gödel machine: Co-evolving agents and their evaluators,” *arXiv preprint arXiv:2606.26294*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.26294>
- [22] G. Penedo, H. Kydlíček, L. B. allal, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra, and T. Wolf, “The fineweb datasets: Decanting the web for the finest text data at scale,” in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 30 811–30 849. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/370df50ccfd8bde18f8f9c2d9151bda-Paper-Datasets_and_Benchmarks_Track.pdf
- [23] H. Zhang, S. Zhang, K. Li, C. Zhang, Y. Chen, Y. Zhang, L. Bai, and S. Hu, “Self-harness: Harnesses that improve themselves,” *arXiv preprint arXiv:2606.09498*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.09498>
- [24] S. team, A. Bolton, A. Lerchner, A. Cordell, A. Moufarek, A. Bolt, A. Lampinen, A. Mitenkova, A. O. Hallingstad, B. Vujatovic, B. Li, C. Lu, D. Wierstra, D. P. Sawyer, D. Slater, D. Reichert, D. Vercelli, D. Hassabis, D. A. Hudson, D. Williams, E. Hirst, F. Pardo, F. Hill, F. Besse, H. Openshaw, H. Chan, H. Soyer, J. X. Wang, J. Clune, J. Agapiou, J. Reid, J. Marino, J. Kim, K. Gregor, K. Sridhar, K. McKinney, L. Kampis, L. M. Zhang, L. Matthey, L. Wang, M. A. Raad, M. Loks-Thompson, M. Engelcke, M. Kecman, M. Jackson, M. Gazeau, O. Purkiss, O. Knagg, P. Stys, P. Mendolicchio, R. Hadsell, R. Ke, R. Faulkner, S. Chakera, S. S. Baveja, S. Legg, S. Kashem, T. Terzi, T. Keck, T. Harley, T. Scholtes, T. Roberts, V. Mnih, Y. Liu, Z. Wang, and Z. Ghahramani, “Sima 2: A generalist embodied agent for virtual worlds,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.04797>
- [25] Y. Li, Y. Miao, Y. Shen, and Y. Liu, “Pando: Efficient multimodal ai agents via online skill distillation,” *arXiv preprint arXiv:2605.24785*, 2026.
- [26] Z. Tao, T.-E. Lin, X. Chen, H. Li, Y. Wu, Y. Li, Z. Jin, F. Huang, D. Tao, and J. Zhou, “A survey on self-evolution of large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14387>
- [27] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan, H. Liu, S. Liu, J. Qiu, X. Qi, Q. Ren, Y. Wu, H. Wang, H. Xiao, Y. Zhou, S. Zhang, J. Zhang, J. Xiang, Y. Fang, Q. Zhao, D. Liu, C. Qian, Z. Wang, M. Hu, H. Wang, Q. Wu, H. Ji, and M. Wang, “A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence,” *Transactions on Machine Learning Research*, Jan. 2026. [Online]. Available: <https://openreview.net/forum?id=CTr3bovS5F>

- [28] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, Z. Ren, N. Aletras, X. Wang, H. Zhou, and Z. Meng, “A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.07407>
- [29] Z. Ren, Y. Chen, D. Guo, G. Rong, T. Li, R. B. Xiong, Q. Lan, W. Wang, L. Nanbo, Y. Yang, M. Zhuge, and J. Schmidhuber, “Self-improvements in modern agentic systems: A survey,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.13104>
- [30] S. Liu, Z. Lin, Y. Zhang, Y. Ren, Y. Wu, Y. Li, Z. Wang, Z. Fu, and J. Ye, “The path to recursive self-improving agents: Foundation, framework, and future directions,” *Preprints*, Aug. 2026. [Online]. Available: <https://www.preprints.org/manuscript/202608.0051>
- [31] Q. Zong, J. Liu, J. Shen, Z. Tang, L. Wu, Y. Liu, R. Wang, Z. Wang, W. Wang, C. Qian, X. Chen, and Y. Song, “Co-Evolution in Agentic Systems: Toward Self-Directed Evolution Beyond Human Design,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.10299>
- [32] Y. Xu, W. Zhang, Y. Chen, X. Lin, and Y. Zhang, “Self-Evolving Agents as Dynamic Graph Transformation: A Survey and New Perspective,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.18104>
- [33] D. Ye, Z. Zhang, H. Wang, and C. Miao, “A Survey on AI for AI: When the Improver Becomes the Improvee,” *Preprints.org*, 2026, preprint. [Online]. Available: <https://www.preprints.org/manuscript/202609.0591>
- [34] D. Li, B. Jiang, L. Huang, A. Beigi, C. Zhao, Z. Tan, A. Bhattacharjee, Y. Jiang, C. Chen, T. Wu, K. Shu, L. Cheng, and H. Liu, “From generation to judgment: Opportunities and challenges of LLM-as-a-judge,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 2757–2791. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.138/>
- [35] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer, “A Survey on Evaluation of LLM-based Agents,” in *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 2026, pp. 26 690–26 714. [Online]. Available: <https://aclanthology.org/2026.findings-acl.1330/>
- [36] Z. Shan and F. Shao, “A Survey on Rubric-Guided Reinforcement Learning for Language Models,” 2026, arXiv version 2, revised 2026-08-31; Accepted to Findings of EMNLP 2026. [Online]. Available: <https://arxiv.org/abs/2608.27505>
- [37] H. Kim, D. Ryu, X. Yi, J. Yao, J. Lian, M. Huang, S. Duan, J. Bak, and X. Xie, “The Road to Artificial SuperIntelligence: A Comprehensive Survey of Superalignment,” 2024, arXiv version 4, revised 2026-06-17. [Online]. Available: <https://arxiv.org/abs/2412.16468>
- [38] P. Slattery, A. K. Saeri, E. A. C. Grundy, J. Graham, M. Noetel, R. Uuk, J. Dao, S. Pour, S. Casper, and N. Thompson, “The AI risk repository: A meta-review, database, and taxonomy of risks from artificial intelligence,” 2024, arXiv version 3, revised 2026-05-05. [Online]. Available: <https://arxiv.org/abs/2408.12622>
- [39] L. Pan, M. Saxon, W. Xu, D. Nathani, X. Wang, and W. Y. Wang, “Automatically correcting large language models: Surveying the landscape of diverse automated correction strategies,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 484–506, 2024. [Online]. Available: <https://aclanthology.org/2024.tacl-1.27/>
- [40] V. Venkatesh, M. Rathee, and A. Anand, “Trust but Verify! A Survey on Verification Design for Test-time Scaling,” 2025, arXiv version 3, revised 2025-09-09. [Online]. Available: <https://arxiv.org/abs/2508.16665>

- [41] S. Niu, G. Chen, Y. Chen, Z. Wen, J. Hu, Z. Deng, D. Chen, S. Zhang, R. Chen, Z. Lian, S. Xu, G. Dai, Y. Zhang, W. Luo, Y. Zhang, M. Tan, and C. Deng, “A Survey on Self-Improving Test-Time Intelligence: Feedback-Driven Adapting, Learning, and Scaling at Inference,” 2026, accepted by Machine Intelligence Research. [Online]. Available: <https://arxiv.org/abs/2609.01679>
- [42] L. Long, R. Wang, R. Xiao, J. Zhao, X. Ding, G. Chen, and H. Wang, “On LLMs-driven synthetic data generation, curation, and evaluation: A survey,” in *Findings of the Association for Computational Linguistics: ACL 2024*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 11 065–11 082. [Online]. Available: <https://aclanthology.org/2024.findings-acl.658/>
- [43] J. Zheng, C. Shi, X. Cai, Q. Li, D. Zhang, C. Li, D. Yu, and Q. Ma, “Lifelong Learning of Large Language Model based Agents: A Roadmap,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026, arXiv version 2, revised 2026-01-11. [Online]. Available: <https://arxiv.org/abs/2501.07278>
- [44] W.-C. Huang, W. Zhang, Y. Liang, Y. Bei, Y. Chen, T. Feng, X. Pan, Z. Tan, Y. Wang, T. Wei, S. Wu, R. Xu, L. Yang, R. Yang, W. Yang, C.-Y. Yeh, H. Zhang, H. Zhang, S. Zhu, H. P. Zou, W. Zhao, S. Wang, W. Xu, Z. Ke, Z. Hui, D. Li, Y. Wu, L. He, C. Wang, X. Xu, B. Huang, J. Tan, S. Heinecke, H. Wang, C. Xiong, A. A. Metwally, J. Yan, C.-Y. Lee, H. Zeng, Y. Xia, X. Wei, A. Payani, Y. Wang, H. Ma, W. Wang, C. Wang, Y. Zhang, X. E. Wang, Y. Zhang, J. You, H. Tong, X. Luo, X. Liu, Y. Sun, W. Wang, J. McAuley, J. Zou, J. Han, P. S. Yu, and K. Shu, “A Survey of Agent Memory in the Second Half: Towards Self-Evolving and Long-Horizon Agents,” *Transactions on Machine Learning Research*, 2026, arXiv version 4, revised 2026-08-04; Survey Certification. [Online]. Available: <https://arxiv.org/abs/2602.06052>
- [45] C. Zhou, H. Chai, W. Chen, Z. Guo, R. Shan, Y. Song, T. Xu, Y. Yang, A. Yu, W. Zhang, C. Zheng, J. Zhu, Z. Zheng, Z. Zhang, X. Lou, C. Zhang, Z. Fu, J. Wang, W. Liu, J. Lin, and W. Zhang, “Externalization in LLM Agents: A Unified Review of Memory, Skills, Protocols and Harness Engineering,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.08224>
- [46] K. Ramnath, K. Zhou, S. Guan, S. S. Mishra, X. Qi, Z. Shen, S. Wang, S. Woo, S. Jeoung, Y. Wang, H. Wang, H. Ding, Y. Lu, Z. Xu, Y. Zhou, B. Srinivasan, Q. Yan, Y. Chen, H. Ding, P. Xu, and L. L. Cheong, “A systematic survey of automatic prompt optimization techniques,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 33 078–33 110. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.1681/>
- [47] Y.-A. Lee, G.-T. Yi, M.-Y. Liu, J.-C. Lu, G.-B. Yang, and Y.-N. Chen, “Compound AI systems optimization: A survey of methods, challenges, and future directions,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 28 760–28 775. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.1463/>
- [48] L. Yue, K. R. Bhandari, C.-Y. Ko, D. Patel, S. Lin, N. Zhou, J. Gao, P.-Y. Chen, and S. Pan, “From Static Templates to Dynamic Runtime Graphs: A Survey of Workflow Optimization for LLM Agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.22386>
- [49] J. Li, Z. Jin, T. Men, Y. Hao, K. Zhu, L. Wang, D. Huang, L. Wang, S. Hua, L. Wang, J. Gao, H. Yuan, R. Xu, K. Liu, and J. Zhao, “Agentic Environment Engineering for Large Language Models: A Survey of Environment Modeling, Synthesis, Evaluation, and Application,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.12191>

- [50] T. Ding, A. Nannapaneni, B. Liu, and L. Zhang, “Autonomous Research Agents: A Survey of AI Scientists and the Verification Gap,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.05179>
- [51] H. Zhou, H. Hu, T. Luo, Y. Shang, C. Fang, Z. Chen, L. Xiao, and Q. Zhang, “Self-Evolving Coding Agents,” 2026, arXiv version 3, revised 2026-08-29. [Online]. Available: <https://arxiv.org/abs/2608.03392>
- [52] C. Zhu, L. Tian, B. Tan, Z. Zhou, Y. Sun, Y. Wang, C. Lv, Y. Wen, Y. He, J. Lin, Y. Chen, C. W. Tan, Q. Wei, L. Zhao, B. Pu, K. Li, Y. Xue, and J. Lin, “The Path to Self-Evolving Clinical Systems: Scaling Medical Agents from Assistance to Autonomy,” 2026, arXiv version 2, revised 2026-08-12. [Online]. Available: <https://arxiv.org/abs/2607.11175>
- [53] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, J. Shen, G. Ye, H. Lin, J. Poulos, M. Wang, M. Nezhurina, J. Jitsev, D. Lu, O. M. Mastromichalakis, Z. Xu, Z. Chen, Y. Liu, R. Zhang, L. L. Chen, A. Kashyap, J.-L. Uslu, J. Li, J. Wu, M. Yan, S. Bian, V. Sharma, K. Sun, S. Dillmann, A. Anand, A. Lanpouthakoun, B. Koopah, C. Hu, E. Guha, G. H. S. Dreiman, J. Zhu, K. Krauth, L. Zhong, N. Muennighoff, R. Amanfu, S. Tan, S. Pimpalgaonkar, T. Aggarwal, X. Lin, X. Lan, X. Zhao, Y. Liang, Y. Wang, Z. Wang, C. Zhou, D. Heineman, H. Liu, H. Trivedi, J. Yang, J. Lin, M. Shetty, M. Yang, N. Omi, N. Raoof, S. Li, T. Y. Zhuo, W. Lin, Y. Dai, Y. Wang, W. Chai, S. Zhou, D. Wahdany, Z. She, J. Hu, Z. Dong, Y. Zhu, S. Cui, A. Saiyed, A. Kolbeinsson, J. Hu, C. M. Rytting, R. Marten, Y. Wang, A. Dimakis, A. Konwinski, and L. Schmidt, “Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.11868>
- [54] W. Huang, C. Lee, L. Tng, and S. Ge, “Deepswc: Measuring frontier coding agents on original, long-horizon engineering tasks,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.07946>
- [55] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song, “Cybergym: Evaluating ai agents’ real-world cybersecurity capabilities at scale,” 2026. [Online]. Available: <https://arxiv.org/abs/2506.02548>
- [56] S. Lee and D. Brumley, “Exploitbench: A capability ladder benchmark for llm cybersecurity agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.14153>
- [57] D. Shepard and R. Salimans, “Automationbench,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.18934>
- [58] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese, “Browsecomp: A simple yet challenging benchmark for browsing agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.12516>
- [59] W. Wang, P. Piękos, L. Nanbo, F. Laakom, Y. Chen, M. Ostaszewski, M. Zhuge, and J. Schmidhuber, “Huxley-g\”odel machine: Human-level coding agent development by an approximation of the optimal self-improving machine,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=T0EiEuhOOL>
- [60] S. Chen, S. Lin, Y. Shi, H. Lian, X. Gu, L. Yun, D. Chen, L. Cao, J. Liu, N. Xia, and Q. Wang, “Swe-exp: Experience-driven software issue resolution,” 2026. [Online]. Available: <https://arxiv.org/abs/2507.23361>
- [61] Z. Sun, Z. Liu, Y. Zang, Y. Cao, X. Dong, T. Wu, D. Lin, and J. Wang, “SEAgent: Self-evolving computer use agent with autonomous learning from experience,” in *Forty-third International Conference on Machine Learning*, 2026. [Online]. Available: <https://openreview.net/forum?id=AvBgmOGsdQ>
- [62] Z. Dai, S. He, H. Li, Q. Zhou, J. Li, M. Song, G. Long, H. Si, X. Yao, L. Zhang *et al.*, “Metis: Bridging text and code memory for self-evolving agents,” *arXiv preprint arXiv:2606.24151*, 2026.
- [63] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, “Agentic

- harness engineering: Observability-driven automatic evolution of coding-agent harnesses,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.25850>
- [64] H. Shi, Z. Xu, H. Wang, W. Qin, W. Wang, Y. Wang, Z. Wang, S. Ebrahimi, and H. Wang, “Continual learning of large language models: A comprehensive survey,” *ACM Comput. Surv.*, vol. 58, no. 5, Nov. 2025. [Online]. Available: <https://doi.org/10.1145/3735633>
- [65] R. Wang, J. Lehman, J. Clune, and K. O. Stanley, “Paired open-ended trailblazer (poet): Endlessly generating increasingly complex and diverse learning environments and their solutions,” 2019. [Online]. Available: <https://arxiv.org/abs/1901.01753>
- [66] S. Hu, C. Lu, and J. Clune, “Automated design of agentic systems,” in *International Conference on Learning Representations*, 2025.
- [67] J. Zhang, J. Xiang, Z. Yu, F. Teng, X.-H. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, “AFlow: Automating agentic workflow generation,” in *International Conference on Learning Representations*, 2025.
- [68] OpenAI, “Research engineer / research scientist / ai systems engineer, rsi,” <https://openai.com/careers/research-engineer-research-scientist-ai-systems-engineer-rsi-san-francisco/>, 2026, accessed: 2026-09-08.
- [69] Tencent, “Tencent releases and open-sources Tencent Hy4 preview,” Tencent, Aug. 2026, published August 28, 2026; accessed September 5, 2026. [Online]. Available: <https://www.tencent.com/tencent-releases-and-open-sources-tencent-hy4-preview/>
- [70] Anthropic Institute, “When ai builds itself,” <https://www.anthropic.com/institute/recursive-self-improvement>, Jun. 2026, accessed: 2026-09-08.
- [71] T. Wu, L. Luo, Y.-F. Li, S. Pan, T.-T. Vu, and G. Haffari, “Continual Learning for Large Language Models: A Survey,” 2024, arXiv version 2, revised 2024-02-07. [Online]. Available: <https://arxiv.org/abs/2402.01364>
- [72] J. Zheng, S. Qiu, C. Shi, and Q. Ma, “Towards lifelong learning of large language models: A survey,” *ACM Comput. Surv.*, vol. 57, no. 8, Mar. 2025. [Online]. Available: <https://doi.org/10.1145/3716629>
- [73] P. Trirat, W. Jeong, and S. J. Hwang, “Automl-agent: A multi-agent LLM framework for full-pipeline automl,” in *ICML*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR / OpenReview.net, 2025.
- [74] Y. Shang, Y. Li, K. Zhao, L. Ma, J. Liu, F. Xu, and Y. Li, “AgentSquare: Automatic LLM agent search in modular design space,” in *International Conference on Learning Representations*, 2025.
- [75] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, L. Weng, and A. Mađry, “MLE-bench: Evaluating machine learning agents on machine learning engineering,” in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=6s5uXNWGIh>
- [76] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha, “The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.08066>
- [77] G. Starace, O. Jaffe, D. Sherburn, J. Aung, C. J. Shern, L. Maksin, R. Dias, E. Mays, B. Kinsella, W. Thompson, J. Heidecke, A. Glaese, and T. Patwardhan, “Paperbench: evaluating ai’s ability to replicate ai research,” in *Proceedings of the 42nd International Conference on Machine Learning*, ser. ICML’25. JMLR.org, 2025.
- [78] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, “Promptbreeder: self-referential

- self-improvement via prompt evolution,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML’24. JMLR.org, 2024.
- [79] T. Li, Y. Wang, Z. Chen, Z. Wang, L. Ma, and G.-J. Qi, “C-evolve: Consensus-based evolution for prompt groups,” in *International Conference on Learning Representations*, C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust, Eds., vol. 2026, 2026, pp. 37 510–37 586. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2026/file/3f1351a10a3f904e6f3845a94275a909-Paper-Conference.pdf
 - [80] J. Chen, Z. Song, J. Liu, S. Zhou, H. Wu, H. Shi, C. Zhou, H. Li, X. Yang, D. Zhu *et al.*, “Decoevo: Score-decoupled co-evolution of solver and rubric-generator skills in text space,” *arXiv preprint arXiv:2607.25675*, 2026.
 - [81] Z. Wu, L. Zhang, T. Wang, R. Zhao, P. Andrews, C. Aloisi, and Y. He, “EDIT: Evidence-diagnosed intervention training for rule-faithful llm grading,” *arXiv preprint arXiv:2606.06350*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.06350>
 - [82] L. A. Agrawal, S. Tan, D. Soylu, N. Ziemis, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, “GEPA: Reflective prompt evolution can outperform reinforcement learning,” in *International Conference on Learning Representations*, 2026.
 - [83] Y. Choi, D. Kim, J. Baek, and S. J. Hwang, “Multimodal prompt optimization: Why not leverage multiple modalities for mllms,” 2026. [Online]. Available: <https://arxiv.org/abs/2510.09201>
 - [84] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.16291>
 - [85] S. Ouyang, J. Yan, I. Hsu, Y. Chen, K. Jiang, Z. Wang, R. Han, L. Le, S. Daruki, X. Tang *et al.*, “Reasoningbank: Scaling agent self-evolving with reasoning memory,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 94 327–94 354.
 - [86] J. Ni, Y. Liu, X. Liu, Y. Sun, M. Zhou, P. Cheng, D. Wang, E. Zhao, X. Jiang, and G. Jiang, “Trace2skill: Distill trajectory-local lessons into transferable agent skills,” *arXiv preprint arXiv:2603.25158*, 2026.
 - [87] H. Lu and C. Xia, “Agent-agnostic end-to-end c/c++ application performance optimization,” in *Proceedings of the 2026 International Conference on Supercomputing Workshops*. Association for Computing Machinery, 2026, pp. 65–69. [Online]. Available: <https://doi.org/10.1145/3774895.3812199>
 - [88] OpenAI, “Harness engineering: Leveraging codex in an agent-first world,” <https://openai.com/index/harness-engineering/>, Feb. 2026, published February 11, 2026; accessed September 9, 2026.
 - [89] J. Su, Z. Wu, S. Huang, and W. Feng, “AIPC: Agent-based automation for ai model deployment with qualcomm ai runtime,” *arXiv preprint arXiv:2604.14661*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.14661>
 - [90] J. Jaber and O. Jaber, “Autokernel: Autonomous gpu kernel optimization via iterative agent-driven search,” *arXiv preprint arXiv:2603.21331*, 2026.
 - [91] Y. Wu, J. Zhang, J. Shi, X. Lei, Q. Gu, Y. Zhang, Z. Wang, C. He, C. Huang, M. Song *et al.*, “Harnessdev: Can llms create and evolve their own agent harness?” *arXiv preprint arXiv:2609.01437*, 2026.
 - [92] OpenAI and Thrive Holdings, “Building self-improving tax agents with Codex,” Engineering case study, 2026. [Online]. Available: <https://openai.com/index/building-self-improving-tax-agents-with-codex/>

- [93] J. Zhang, B. Zhao, W. Yang, J. Foerster, J. Clune, M. Jiang, S. Devlin, and T. Shavrina, “Hyperagents,” *arXiv preprint arXiv:2603.19461*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.19461>
- [94] Weco Team, “AIDE²: The first evidence of recursive self-improvement,” *Weco AI Blog*, Jul. 2026, industrial research disclosure; technical report and artifact release announced as forthcoming at the time of writing. [Online]. Available: <https://www.weco.ai/blog/first-evidence-of-recursive-self-improvement>
- [95] Microsoft Research, “SynthLLM: Breaking the ai data wall with scalable synthetic data,” <https://www.microsoft.com/en-us/research/articles/synthllm-breaking-the-ai-data-wall-with-scalable-synthetic-data/>, 2025, accessed September 9, 2026.
- [96] Z. Wang, Y. Liang, X. Zhang, Q. Wu, S. Han, A. Bastos, R. Wang, C. Bansal, B. Peng, J. Gao, S. Rajmohan, and H. Yao, “SynthAgent: Adapting web agents with synthetic supervision,” in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026, pp. 15 730–15 752. [Online]. Available: <https://aclanthology.org/2026.acl-long.716/>
- [97] NVIDIA, “Nemotron-4 340b technical report,” *arXiv preprint arXiv:2406.11704*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.11704>
- [98] H. Lu, Y. Wen, P. Cheng, R. Ding, J. Guo, H. Xu, C. Wang, H. Chen, xiaoxi jiang, and guanjunjiang, “Search self-play: Pushing the frontier of agent capability without supervision,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=ZmGirmNJqE>
- [99] A. Zhao, Y. Wu, T. Wu, Q. Xu, Y. Yue, M. Lin, S. Wang, Q. Wu, Z. Zheng, and G. Huang, “Absolute zero: Reinforced self-play reasoning with zero data,” in *NeurIPS*, 2025.
- [100] K. Dong and T. Ma, “STP: self-play LLM theorem provers with iterative conjecturing and proving,” in *ICML*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR / OpenReview.net, 2025.
- [101] A. Wilf, P. Aggarwal, B. Parno, D. Fried, L.-P. Morency, P. P. Liang, and S. Welleck, “Propose, solve, verify: Self-play through formal verification,” in *Forty-third International Conference on Machine Learning*, 2026. [Online]. Available: <https://openreview.net/forum?id=Vuaq1qXCki>
- [102] X. Zeng, Y. Chen, L. Liu, C. Luo, Y. Chen, and Zhuangzhuoran, “Teaching llm to be persuasive: Reward-enhanced policy optimization for alignment from heterogeneous rewards,” in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: Industry Track*, 2026. [Online]. Available: <https://aclanthology.org/2026.acl-industry.34/>
- [103] C. Huang, W. Yu, X. Wang, H. Zhang, Z. Li, R. Li, J. Huang, H. Mi, and D. Yu, “R-zero: Self-evolving reasoning LLM from zero data,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=96apU6YzSO>
- [104] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li *et al.*, “Agentic context engineering: Evolving contexts for self-improving language models,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 86 069–86 100.
- [105] W. Zhang, X. Zhang, C. Zhang, L. Yang, J. Shang, Z. Wei, H. P. Zou, Z. Huang, Z. Wang, Y. Gao *et al.*, “Personaagent: Bridging memory and action for personalized llm agents,” in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 26 421–26 439.
- [106] F. Shang and Y. Yang, “Hypothesis-driven skill optimization for llm agents,” *arXiv preprint arXiv:2606.22330*, 2026.

- [107] R. K. Arora, J. Wei, R. Soskin Hicks, P. Bowman, J. Quiñonero-Candela, F. Tsimpourlas, M. Sharman, M. Shah, A. Vallone, A. Beutel, J. Heidecke, and K. Singhal, “Healthbench: Evaluating large language models towards improved human health,” *arXiv preprint arXiv:2505.08775*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.08775>
- [108] A. Karpathy, “autoresearch: Ai agents running research on single-gpu nanochat training automatically,” GitHub repository, 2026. [Online]. Available: <https://github.com/karpathy/autoresearch>
- [109] Y. Wu, J. Zhang, J. Shi, Y. Zhang, X. Lei, J. Zhou, Z. Wang, Y. Wu, H. Zhou, D. Wang *et al.*, “Aspire: Can models self-evolve from vague goals?” *arXiv preprint arXiv:2608.31111*, 2026.
- [110] M. Suzgun, M. Yuksekgonul, F. Bianchi, D. Jurafsky, and J. Zou, “Dynamic cheatsheet: Test-time learning with adaptive memory,” in *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 7080–7106.
- [111] P. Wang, Z. Ma, Y. Chang, X. Luo, X. Yang, S. Feng, Y. Yang, and D. Li, “Self-evolving embodied agents via skill-harness evolution,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.11350>
- [112] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang *et al.*, “Self-refine: Iterative refinement with self-feedback,” *Advances in neural information processing systems*, vol. 36, pp. 46 534–46 594, 2023.
- [113] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.11366>
- [114] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *NeurIPS*, 2023.
- [115] P. Banerjee, M. Moshtaghi, and A. Chadha, “APEX-EM: Non-parametric online learning for autonomous agents via structured procedural-episodic experience replay,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.29093>
- [116] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, “ExpeL: LLM agents are experiential learners,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 632–19 642. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/29936>
- [117] K. Tsui, “Self-correction bench: Uncovering and addressing the self-correction blind spot in large language models,” 2026. [Online]. Available: <https://arxiv.org/abs/2507.02778>
- [118] G. Tyen, H. Mansoor, V. Carbune, P. Chen, and T. Mak, “LLMs cannot find reasoning errors, but can correct them given the error location,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 13 894–13 908. [Online]. Available: <https://aclanthology.org/2024.findings-acl.826/>
- [119] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large language models cannot self-correct reasoning yet,” in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=Ikmd3fKBPQ>
- [120] Y. Li, “Decomposing LLM self-correction: The accuracy-correction paradox and error depth hypothesis,” *arXiv preprint arXiv:2601.00828*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.00828>
- [121] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, “Is self-repair a silver bullet for code generation?” in *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, Eds., vol. 2024, 2024, pp. 36 545–36 593. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/9ddc141bdbf9d1db510ceff56c586ad-Paper-Conference.pdf

- [122] Y. Verma, “Try again, don’t look back: Blind resampling outperforms self-repair in small code models,” *arXiv preprint arXiv:2607.26117*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.26117>
- [123] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 21 558–21 572, 2023. [Online]. Available: <https://arxiv.org/abs/2305.01210>
- [124] Google Research, “Achieving 10,000x training data reduction with high-fidelity labels,” <https://research.google/blog/achieving-10000x-training-data-reduction-with-high-fidelity-labels/>, Aug. 2025, published August 7, 2025; accessed September 9, 2026.
- [125] M. Abdin, S. Agarwal, A. Awadallah, V. Balachandran, H. Behl, L. Chen, G. de Rosa, S. Gunasekar, M. Javaheripi, N. Joshi, P. Kauffmann, Y. Lara, C. C. T. Mendes, A. Mitra, B. Nushi, D. Papailiopoulos, O. Saarikivi, S. Shah, V. Shrivastava, V. Vineet, Y. Wu, S. Yousefi, and G. Zheng, “Phi-4-reasoning technical report,” *arXiv preprint arXiv:2504.21318*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.21318>
- [126] NVIDIA, “NVIDIA NeMo Curator: Data curation for generative ai,” <https://docs.nvidia.com/nemo/curator/>, 2025, accessed September 9, 2026.
- [127] D. Chen, Y. Huang, Z. Ma, H. Chen, X. Pan, C. Ge, D. Gao, Y. Xie, Z. Liu, J. Gao, Y. Li, B. Ding, and J. Zhou, “Data-juicer: A one-stop data processing system for large language models,” in *Proceedings of the 2024 International Conference on Management of Data*, 2024. [Online]. Available: <https://doi.org/10.1145/3626246.3653385>
- [128] P. Liu, J. Wu, and U. Thakore, “Leveraging agents to debug nccl watchdog timeouts: A technical, repeatable workflow,” @Scale: Systems & Reliability, Jun. 2026, published June 25, 2026.
- [129] Amazon Web Services, “Agent toolkit for aws,” <https://aws.amazon.com/products/developer-tools/agent-toolkit-for-aws/>, May 2026, announced May 6, 2026; accessed September 9, 2026.
- [130] LinkedIn Engineering, “Contextual agent playbooks and tools: How linkedin gave ai coding agents organizational context,” LinkedIn Engineering Blog, Jan. 2026, published January 27, 2026.
- [131] OpenAI, “Research acceleration: The view inside OpenAI,” OpenAI, Sep. 2026. [Online]. Available: <https://openai.com/index/research-acceleration-view-inside-openai/>
- [132] E. Wang and D. Meyerzon, “How we optimized Dash’s relevance judge with DSPy,” Dropbox.Tech, Mar. 2026, published March 17, 2026; accessed September 5, 2026. [Online]. Available: <https://dropbox.tech/machine-learning/optimizing-dropbox-dash-relevance-judge-with-dspy>
- [133] L. Quintanilla and V. Dibia, “Introducing agent optimizer in Foundry Agent Service,” Microsoft Foundry Blog, Jun. 2026, published June 3, 2026; accessed September 5, 2026. [Online]. Available: <https://devblogs.microsoft.com/foundry/agent-optimizer-build2026/>
- [134] D. Rodge, “Post-train NVIDIA Cosmos 3 in one day using agent skills,” NVIDIA Technical Blog, Jul. 2026. [Online]. Available: <https://developer.nvidia.com/blog/post-train-nvidia-cosmos-3-in-one-day-using-agent-skills/>
- [135] OpenAI, “GPT-6 Astra system card,” OpenAI Deployment Safety Hub, Sep. 2026. [Online]. Available: <https://deploymentsafety.openai.com/gpt-6-astra/vision>
- [136] J. Ning, X. Li, J. Zeng, H. Kang, and C. Xiong, “Auto research with specialist agents develops effective and non-trivial training recipes,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.05724>

- [137] L. Soldaini, R. Kinney, A. Bhagia, D. Schwenk, D. Atkinson, R. Authur, B. Bogin, K. R. Chandu, J. Dumas, Y. Elazar, V. Hofmann, A. H. Jha, S. Kumar, L. Lucy, X. Lyu, N. Lambert, I. Magnusson, J. Morrison, N. Muennighoff, A. Naik, C. Nam, M. E. Peters, A. Ravichander, K. Richardson, Z. Shen, E. Strubell, N. Subramani, O. Tafjord, P. Walsh, L. Zettlemoyer, N. A. Smith, H. Hajishirzi, I. Beltagy, D. Groeneveld, J. Dodge, and K. Lo, “Dolma: an open corpus of three trillion tokens for language model pretraining research,” in *ACL (1)*. Association for Computational Linguistics, 2024, pp. 15 725–15 788.
- [138] . AI, :, A. Young, B. Chen, C. Li, C. Huang, G. Zhang, G. Zhang, G. Wang, H. Li, J. Zhu, J. Chen, J. Chang, K. Yu, P. Liu, Q. Liu, S. Yue, S. Yang, S. Yang, W. Xie, W. Huang, X. Hu, X. Ren, X. Niu, P. Nie, Y. Li, Y. Xu, Y. Liu, Y. Wang, Y. Cai, Z. Gu, Z. Liu, and Z. Dai, “Yi: Open foundation models by 01.ai,” 2025. [Online]. Available: <https://arxiv.org/abs/2403.04652>
- [139] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, “Qwen3 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [140] M. Abdin, J. Aneja, H. Behl, S. Bubeck, R. Eldan, S. Gunasekar, M. Harrison, R. J. Hewett, M. Javaheripi, P. Kauffmann, J. R. Lee, Y. T. Lee, Y. Li, W. Liu, C. C. T. Mendes, A. Nguyen, E. Price, G. de Rosa, O. Saarikivi, A. Salim, S. Shah, X. Wang, R. Ward, Y. Wu, D. Yu, C. Zhang, and Y. Zhang, “Phi-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.08905>
- [141] J. Li, A. Fang, G. Smyrnis, M. Ivgi, M. Jordan, S. Y. Gadre, H. Bansal, E. K. Guha, S. S. Keh, K. Arora, S. Garg, R. Xin, N. Muennighoff, R. Heckel, J. Mercat, M. F. Chen, S. Gururangan, M. Wortsman, A. Albalak, Y. Bitton, M. Nezhurina, A. Abbas, C. Hsieh, D. Ghosh, J. Gardner, M. Kilian, H. Zhang, R. Shao, S. M. Pratt, S. Sanyal, G. Ilharco, G. Daras, K. Marathe, A. Gokaslan, J. Zhang, K. R. Chandu, T. Nguyen, I. Vasiljevic, S. M. Kakade, S. Song, S. Sanghavi, F. Faghri, S. Oh, L. Zettlemoyer, K. Lo, A. El-Nouby, H. Pouransari, A. Toshev, S. Wang, D. Groeneveld, L. Soldaini, P. W. Koh, J. Jitsev, T. Kollar, A. Dimakis, Y. Carmon, A. Dave, L. Schmidt, and V. Shankar, “Datacomp-lm: In search of the next generation of training sets for language models,” in *NeurIPS*, 2024.
- [142] Y. He, C. Huang, Z. Li, J. Huang, and Y. Yang, “Visplay: Self-evolving vision-language models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2026, pp. 26 274–26 284.
- [143] Y. Li, J. Yang, Z. Zheng, Z. Hu, Y. Sui, S. Wang, Y. He, and B. Hooi, “Apex: Autonomous policy exploration for self-evolving llm agents,” *arXiv preprint arXiv:2605.21240*, 2026.
- [144] K. Liang, J. Kruk, S. Qian, X. Yang, S. Bi, Y. Yao, S. Nie, M. Zhang, L. Liu, J. F. Fisac *et al.*, “Learning personalized agents from human feedback,” *arXiv preprint arXiv:2602.16173*, 2026.
- [145] S. A. Er, D. Ribeiro, Y. Virkar, S. Lakew, A. Kalyanpur, J. Gung, T. Delteil, and A. Gupta, “Memtoolagent: Leveraging memory for tool using agents based on environment and user feedback,” *arXiv preprint arXiv:2606.07909*, 2026.
- [146] Z. Bai, S. Li, T. Huang, and B. F. Karlsson, “Practice: From experience to expertise in self-evolving embodied agents,” *arXiv preprint arXiv:2608.30760*, 2026.
- [147] Y. Xiao, Y. Sun, H. Wu, W. Hui, W. Da, Z. Luo, M. Chuan, Y. Hu, W. Li, and C. Jiang, “Pilot in the loop: Live self-improvement for long-horizon agents,” *arXiv preprint arXiv:2608.26530*, 2026.

- [148] T. Wei, Z. Shi, M. Lin, B. He, Z. Liu, Y. Sang, Y. Bei, X. Ning, J. Zou, T.-W. Li *et al.*, “Evo-harness: Context-to-harness skill compilation for self-evolving agents,” *arXiv preprint arXiv:2608.15071*, 2026.
- [149] T. Wei, N. Sachdeva, B. Coleman, Z. He, Y. Bei, X. Ning, M. Ai, Y. Li, J. He, E. H. Chi *et al.*, “Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory,” *arXiv preprint arXiv:2511.20857*, 2025.
- [150] J. Shi, S. Tao, Y. Wu, Z. Wang, J. Zhang, J. Liu, X. Lei, X. Zhang, S. Fang, Z. Tan *et al.*, “S3gym: Can llms turn self-testing and self-judging into self-improvement?” *arXiv preprint arXiv:2608.31100*, 2026.
- [151] M. Lin, J. Wu, Z. Wang, Z. Shi, Y. Sang, B. He, Z. Liu, T. Wei, Z. Wu, Z. Zhang *et al.*, “Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving llm agents,” *arXiv preprint arXiv:2605.30621*, 2026.
- [152] X. Zhang, Y. Cui, G. Wang, Z. Li, W. Qiu, B. Zhu, and P. He, “Library drift: Diagnosing and fixing a silent failure mode in self-evolving llm skill libraries,” *arXiv preprint arXiv:2605.19576*, 2026.
- [153] Y. Liu, Z. Su, Y. Zhang, J. Guo, Z. Xie, H. Jing, L. Xie, Q. Zong, Y. Yim, Z. Zhang *et al.*, “Rethinking self-evolving agent skills: Feedback dynamics over multiple rounds,” *arXiv preprint arXiv:2608.02636*, 2026.
- [154] K. Hambarzumyan, N. Baldwin, E. Toledo, R. Hazra, M. Kuchnik, B. A. Omari, T. S. Foster, A. Protopopov, J.-C. Gagnon-Audet, I. Mediratta, K. Niu, M. Shvartsman, A. Lupidi, A. Audran-Reiss, P. Pathak, T. Shavrina, D. Magka, H. Momand, D. Dunfield, N. Cancedda, P. Stenetorp, C.-J. Wu, J. N. Foerster, Y. Bachrach, and M. Josifoski, “AIRA₂: Overcoming bottlenecks in AI research agents,” *arXiv preprint arXiv:2603.26499*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.26499>
- [155] Y.-H. Chen, J. Wen, and J. H. Kirchner, “Automated researchers can mitigate well-characterized alignment failures,” Anthropic Alignment Science Blog, Aug. 2026, research report; accessed September 6, 2026. [Online]. Available: <https://alignment.anthropic.com/2026/automated-alignment-researchers/>
- [156] S. Jiang, L. Ma, Z. Hong, K. Wang, Z. Lu, T. Wang, S. Chen, J. Zhang, T. Pan, W. Li *et al.*, “Sea-eval: A benchmark for evaluating self-evolving agents beyond episodic assessment,” *arXiv preprint arXiv:2604.08988*, 2026.
- [157] C. Zheng, C. Xue, B. Liang, J. Yang, and C. Zhang, “Seagym: An evaluation environment for self-evolving llm agents,” *arXiv preprint arXiv:2606.17546*, 2026.
- [158] Z. Qian, J. Lei, Y. Wang, and N. Cao, “Hypoforge: A self-improving multi-agent framework for automated hypothesis generation and testing via scientific skill learning,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.25770>
- [159] Y. Lyu, X. Zhang, X. Yi, Y. Zhao, S. Guo, W. Hu, J. Piotrowski, J. Kaliski, J. Urbani, Z. Meng, L. Zhou, and X. Yan, “Evoscientist: Towards multi-agent evolving ai scientists for end-to-end scientific discovery,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.08127>
- [160] M. Yuksekgonul, D. Kocejka, X. Li, F. Bianchi, J. McCaleb, X. Wang, J. Kautz, Y. Choi, J. Zou, C. Guestrin, and Y. Sun, “Learning to discover at test time,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.16175>
- [161] A. Khajeh, X. Lei, W. Ye, Z. Yang, L. Hung, D. Schweigert, and H.-K. Kwon, “A materials discovery framework based on conditional generative models applied to the design of polymer electrolytes,” *Digital Discovery*, vol. 4, no. 1, p. 11–20, 2025. [Online]. Available: <http://dx.doi.org/10.1039/D4DD00293H>
- [162] J. Lu, Z. Kong, Y. Wang, R. Fu, H. Wan, C. Yang, W. Lou, H. Sun, L. Wang, Y. Jiang, X. Wang, X. Sun, and D. Zhou, “Beyond static tools: Test-time tool evolution for scientific reasoning,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.07641>

- [163] X. Huang, J. Chen, Y. Fei, Z. Li, P. Schwaller, and G. Ceder, “Cascade: Cumulative agentic skill creation through autonomous development and evolution,” 2026. [Online]. Available: <https://arxiv.org/abs/2512.23880>
- [164] N. Team, “S1-nexusagent: a self-evolving agent framework for multidisciplinary scientific research,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.01550>
- [165] S. Shen, W. Cheng, M. Ma, A. Turcan, M. J. Zhang, and J. Ma, “Skillfoundry: Building self-evolving agent skill libraries from heterogeneous scientific resources,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.03964>
- [166] Y. Zhang, X. Cheng, T. Liu, Y. Du, and W. Jin, “Drugsage:self-evolving agent experience for efficient state-of-the-art drug discovery,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.15461>
- [167] R. Jin, Z. Zhang, M. Wang, and L. Cong, “Stella: Self-evolving llm agent for biomedical research,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.02004>
- [168] Z. Guo, J. Wang, F. Ling, W. Wei, X. Yue, Z. Jiang, W. Xu, J.-J. Luo, L. Cheng, Y.-G. Ham, F. Song, P. Gentine, T. Yamagata, B. Fei, W. Zhang, X. Gu, C. Li, Y. Wang, T. Chen, W. Ouyang, B. Zhou, and L. Bai, “A self-evolving ai agent system for climate science,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.17311>
- [169] Z. Zhang, Z. Qiu, Y. Wu, S. Li, D. Wang, Y. Liu, Z. Zhou, Y. Hu, Y. Chen, D. An, Y. Wang, Y. Li, Z. Zhong, C. Ou, Z. Wang, F. Tang, J. X. Chen, R. Ma, J. Li, X. Wang, W. Lu, H. Xue, W. Zhang, Z. Wei, R. Ma, Z. Shi, K. Wang, Q. Liu, B. Dong, Y. He, T. Liu, J. Gu, S. Song, Q. Feng, J. Zhang, B. Zhang, L. Tian, L. Bai, Q. Gao, S. Sun, and S. Zheng, “Origene: A self-evolving virtual disease biologist automating therapeutic target discovery,” *bioRxiv*, 2026. [Online]. Available: <https://www.biorxiv.org/content/early/2026/02/25/2025.06.03.657658>
- [170] B. Sun, W. Guo, Z. Yu, and L. Yang, “Self-evolving scientific agent designs physically-reasoned whitebox fluid control,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.08405>
- [171] P. Hebbar, Y. Manawat, S. Verboomen, A. Ivanova, S. Palanimalai, K. Bhatia, and V. Baskaran, “SIA: Self improving ai with harness and weight updates,” *arXiv preprint arXiv:2605.27276*, 2026. [Online]. Available: <https://arxiv.org/abs/2605.27276>
- [172] A. Qu, H. Zheng, Z. Zhou, Y. Yan, Y. Tang, S. Y. Ong, F. Hong, K. Zhou, C. Jiang, M. Kong, J. Zhu, X. Jiang, S. Li, C. Wu, B. K. H. Low, J. Zhao, and P. P. Liang, “Coral: Towards autonomous multi-agent evolution for open-ended discovery,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.01658>
- [173] Y. Du, B. Yu, T. Liu, T. Shen, J. Chen, J. G. Rittig, K. Sun, Y. Zhang, A. Krishnan, Y. Zhang, D. Rosen, R. Pirone, Z. Song, B. Zhou, C. Masschelein, Y. Wang, H. Wang, H. Jia, C. Zhang, H. Zhao, M. Ester, N. Hacohen, T. Head-Gordon, C. P. Gomes, H. Sun, C. Duan, P. Schwaller, and W. Jin, “Accelerating scientific discovery with autonomous goal-evolving agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2512.21782>
- [174] A. Zala, J. Cho, H. Lin, J. Yoon, and M. Bansal, “Envgen: Generating and adapting environments via llms for training embodied agents,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.12014>
- [175] J. Guo, L. Yang, P. Chen, Q. Xiao, Y. Wang, X. Juan, J. Qiu, K. Shen, and M. Wang, “Genenv: Difficulty-aligned co-evolution between llm agents and environment simulators,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.19682>
- [176] M. Faldor, J. Zhang, A. Cully, and J. Clune, “Omni-epic: Open-endedness via models of human notions of interestingness with environments programmed in code,” 2025. [Online]. Available: <https://arxiv.org/abs/2405.15568>

- [177] H. Kang, X. Ye, Y. Liu, S. H. Mantri, L. Mao, J. Fleming, D. Regmi, and L. Qin, “Simworld studio: Automatic environment generation with evolving coding agent for embodied agent learning,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.09423>
- [178] G. Tzifas and H. Kasaei, “Lifelong robot library learning: Bootstrapping composable and generalizable skills for embodied control with language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.18746>
- [179] R. Ju, X. Wang, X. Ding, Y. Yang, H. Wu, S. Jiang, Q. Zhang, H. Wen, X. Li, W. Wang, K. Li, Y. Liu, H. Dai, W. Wang, and T. Cao, “Embodiskill: Skill-aware reflection for self-evolving embodied agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.10332>
- [180] R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, M. Chowdhury, Y. Zhu, L. J. Fan, and G. Wang, “Aspire: Agentic /skills discovery for robotics,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.00272>
- [181] W. Xiao, J. Xie, T. Zhang, H. Lin, L. M. Fu, H. Xue, J. Lu, Y. Yang, C. Dai, Z. Wang, J. Wu, G. Wang, S. S. Sastry, K. Goldberg, L. J. Fan, Y. Zhu, and G. Shi, “Empire: Agentic robot policy self-improvement in the real world,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.19980>
- [182] S. K. S. Ghasemipour, A. Wahid, J. Tompson, P. Sanketi, and I. Mordatch, “Self-improving embodied foundation models,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.15155>
- [183] A. Sharma, A. M. Ahmed, R. Ahmad, and C. Finn, “Self-improving robots: End-to-end autonomous visuomotor reinforcement learning,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.01488>
- [184] V. Giridhar, A. Khandelwal, J. A. Collins, I. Georgiev, and A. Garg, “Beyond imitation: Self-improving robot policies via off-policy q-planning,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.21204>
- [185] G. Li, R. Han, C. Li, H. Li, S. Wang, W. Ding, H. Zhang, and C. Xu, “Agentic self-evolutionary replanning for embodied navigation,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.02772>
- [186] Y. Guo, T. Lee, L. X. Shi, J. Chen, P. Liang, and C. Finn, “Vlaw: Iterative co-improvement of vision-language-action policy and world model,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.12063>
- [187] X. Liu, Z. Bai, H. Ci, K. Y. Ma, and M. Z. Shou, “World-vla-loop: Closed-loop learning of video world model and vla policy,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.06508>
- [188] H. Bi, Z. Zhou, Y. Tang, J. Pang, S. Huang, H. Liu, R. Wang, S. Huang, Y. Wang, Y. Cheng, R. Zhao, Z. Li, H. Tan, X. Liu, J. Wan, J. Liu, M. Zhao, F. Bao, and J. Zhu, “Motus2: A self-evolving general world model for dexterous manipulation,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.30237>
- [189] C. Luo, Z. Zeng, M. Jia, Y. Du, and C. Sun, “Self-improving loops for visual robotic planning,” 2026. [Online]. Available: <https://arxiv.org/abs/2506.06658>
- [190] M. Robeyns, M. Szummer, and L. Aitchison, “A self-improving coding agent,” in *ICLR Workshop on Scaling Self-Improving Foundation Models*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.15228>
- [191] Y. Li, Y. Zhang, X. Zhang, X. Liu, and Y. Liu, “Codeskill: Learning self-evolving skills for coding agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.25430>
- [192] Y. Hu, Y. Cai, Y. Du, X. Zhu, X. Liu, Z. Yu, Y. Hou, S. Tang, and S. Chen, “Self-evolving multi-agent collaboration networks for software development,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.16946>
- [193] Z. Weng, A. Antoniadis, D. Nathani, Z. Zhang, X. Pu, and X. E. Wang, “Group-evolving agents: Open-ended self-improvement via experience sharing,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.04837>

- [194] Y. Zhang, Y. Dai, J. Tan, L. Yang, R. Mullur, T. Hoang, Z. Hu, J. Zhu, P. Mui, S. Savarese, R. Xu, and Z. Chen, “Darwinx: Evolving agent harnesses through natural selection,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.07545>
- [195] T. Fan and C. Huang, “Helix: Model-harness co-evolution for recursive self-improvement,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.13951>
- [196] Y. Wei, Z. Sun, E. McMilin, J. Gehring, D. W. Zhang, G. Synnaeve, D. Fried, L. ZHANG, and S. Wang, “Toward training superintelligent software agents through self-play SWE-RL,” in *Forty-third International Conference on Machine Learning*, 2026. [Online]. Available: <https://openreview.net/forum?id=0ophJB76sC>
- [197] C. Xiao, Z. Jiao, S. Wang, W. Wang, B. Zhao, H. Wei, L. Zhang, and L. Qu, “Socratic-swe: Self-evolving coding agents via trace-derived agent skills,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.07412>
- [198] J. Li, Y. Lai, W. Li, J. Ren, M. Zhang, X. Kang, S. Wang, P. Li, Y.-Q. Zhang, W. Ma, and Y. Liu, “Agent hospital: A simulacrum of hospital with evolvable medical agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2405.02957>
- [199] K. Lan, B. Jin, Z. Zhu, S. Chen, S. Zhang, K. Q. Zhu, and M. Wu, “Depression diagnosis dialogue simulation: Self-improving psychiatrist with tertiary memory,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.15084>
- [200] M. Almansoori, K. Kumar, and H. Cholakkal, “MedAgentSim: Self-Evolving Multi-Agent Simulations for Realistic Clinical Interactions,” in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2025*, ser. Lecture Notes in Computer Science, vol. 15968. Springer Nature Switzerland, 2025, pp. 362–372.
- [201] R. Ren, Y. Wang, Y. Liang, L. Luo, J. Liu, H. Wang, C. Feng, Y. Zhang, C. Miao, J.-R. Wen, and W. X. Zhao, “Emulating clinician cognition via self-evolving deep clinical research,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.10677>
- [202] X. Han, Y. Fan, S. Zhao, H. Wang, and B. Qin, “Gsem: Graph-based self-evolving memory for experience augmented clinical reasoning,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.22096>
- [203] W. Shen, B. Jian, J. Li, C. Liu, J. Moll, X. Hu, D. Rueckert, H. B. Li, and J. Pan, “Evo-medagent: Beyond one-shot diagnosis with agents that remember, reflect, and improve,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.14475>
- [204] Y. He, J. Liu, Z. Hu, Y. Chen, Y. Liu, Y. Sui, Y. Li, N. Chen, J. Hu, B. Hooi, X. Xu, and J. Bian, “Evoclinician: A self-evolving agent for multi-turn medical diagnosis via test-time evolutionary learning,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.22964>
- [205] Q. Liu, Z. Hu, T. Huang, Y. Niu, X. Zhang, S. Ma, C. Lin, G. K. Huat, H. E. Kwon, F. Gao, X. Sun, Z. Ying, and G. Qiang, “EvoMDT: A Self-Evolving Multi-Agent System for Structured Clinical Decision-Making in Multi-Cancer,” *npj Digital Medicine*, vol. 9, no. 1, p. 124, Jan. 2026. [Online]. Available: <https://doi.org/10.1038/s41746-025-02304-8>
- [206] Y. Yang, J. Li, Q. Pan, J. Zhou, K. Chen, Q. Chen, J. Zhao, N. Zhou, X. Li, and L. He, “Psychagent: An experience-driven lifelong learning agent for self-evolving psychological counselor,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.00931>
- [207] L. Fan, P. Dai, Z. Deng, H. Wang, X. Gong, Y. Zheng, and Y. Ou, “Evolving medical imaging agents via experience-driven self-skill discovery,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.05860>
- [208] H. Sun, W. Li, Y. Zhang, Z. Lin, F. Zhang, K. Chen, X. He, Y. Li, M. Liu, L. Liu, and Y. Jiang, “Experience

- makes skillful: Enabling generalizable medical agent reasoning via self-evolving skill memory,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.09365>
- [209] S. Li, J. Xu, T. Bao, Y. Liu, Y. Liu, Y. Liu, L. Wang, W. Lei, S. Wang, Y. Xu, Y. Cui, J. Yao, S. Koga, and Z. Huang, “A co-evolving agentic ai system for medical imaging analysis,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.20279>
- [210] Y. Zhu, Z. Wang, Y. Qi, L. Gu, D. Sui, H. Hu, X. Zhang, Z. He, Y. Wang, J. He, L. Ma, and L. Yu, “HealthFlow: Automating Electronic Health Record Analysis via a Strategically Self-Evolving Multi-Agent Framework,” *npj Digital Medicine*, vol. 9, no. 1, p. 660, Aug. 2026. [Online]. Available: <https://doi.org/10.1038/s41746-026-03097-0>
- [211] Z. Du, L. Zheng, R. Hu, Y. Xu, X. Li, Y. Sun, W. Chen, J. Wu, H. Cai, and H. Ying, “Llms can simulate standardized patients via agent coevolution,” in *ACL (1)*. Association for Computational Linguistics, 2025, pp. 17 278–17 306.
- [212] Z. Tang *et al.*, “Workspace-bench 1.0: Benchmarking ai agents on workspace tasks with large-scale file dependencies,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.03596>
- [213] Z. Zhu, Q. He, S. Li, Y. Chen, H. Sun, Z. Li, Y. Li, and Z. Liu, “ForgeTrain: Forging production-grade training frameworks via harness-driven AI development,” <https://github.com/OpenBMB/ForgeTrain>, 2026, technical report and GitHub repository. [Online]. Available: <https://github.com/OpenBMB/ForgeTrain>
- [214] The ForgeStencil Authors, “ForgeStencil: Autonomous agents for stencil optimization and deployment,” <https://github.com/OpenBMB/ForgeStencil>, 2026, gitHub repository.
- [215] A. Karpathy, “autoresearch: Autonomous ai research on small language models,” <https://github.com/karpathy/autoresearch>, 2026, gitHub repository.
- [216] K. Jordan, J. Bernstein, B. Rappazzo, @fernbear.bsky.social, B. Vlado, Y. Jiacheng, F. Cesista, B. Koszarsky, and @Grad62304977, “modded-nanogpt: Speedrunning the nanogpt baseline,” <https://github.com/KellerJordan/modded-nanogpt>, 2024, gitHub repository. [Online]. Available: <https://github.com/KellerJordan/modded-nanogpt>
- [217] E. Lin, S. Modi, S. K. S. Hari, Q. Huang, Z. Ye, N. Qin, F. Zhou, Y. Zhang, J. Wang, S. Damani, D. Peri, O. Xie, A. Kane, M. Maor, M. Behar, T. Cao, R. Mehta, V. Singh, V. S. Mailthody, T. Chen, Z. Ye, H. Chen, T. Chen, V. Grover, W. Chen, W. Liu, E. Chung, L. Ceze, R. Bringmann, C. Zeller, M. Lightstone, C. Kozyrakis, and H. Shi, “Sol-execbench: Speed-of-light benchmarking for real-world gpu kernels against hardware limits,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.19173>

Appendix A: RSI Landscape

图 16 按自主级别和改进目标总结了所调查的论文。

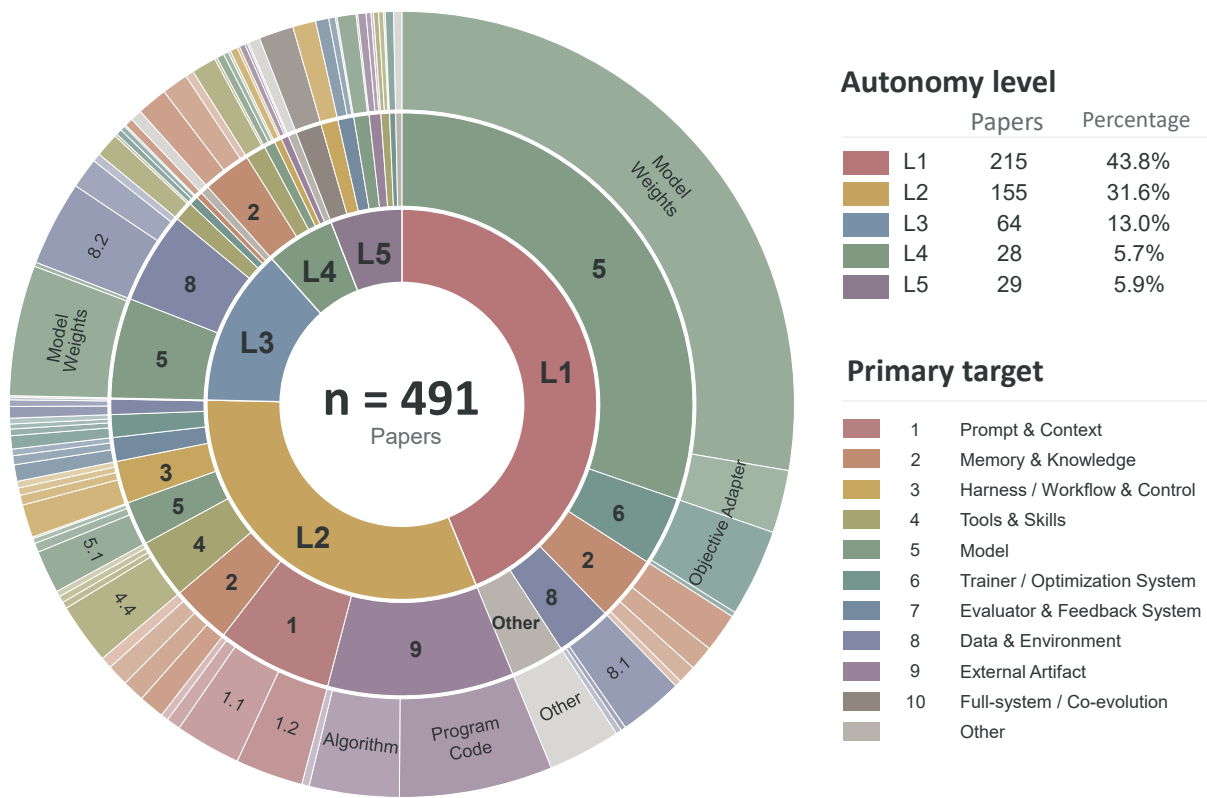


表 11 详细列出了图 16 中所表示的主要目标和子目标。

Table 11 递归自我改进系统中改进目标的分类。该分类将图 16 中所表示的改进目标分解为主目标和子目标。对于每个子目标，表格指定了需要修改的具体系统组件。

Primary Target	Sub-target	Modified Component
1. Prompt & Context	1.1 Instruction	System prompts, role specifications, behavioral rules, constraints, and high-level instructions
	1.2 Task Prompt / Template	Task descriptions, problem formulations, prompt templates, and reusable task specifications
	1.3 Reasoning Prompt / Protocol	Explicit reasoning instructions and protocols, including chain-of-thought, reflection, critique, and debate
	1.4 Demonstrations / Examples	Few-shot examples, demonstrations, successful trajectories, and worked solutions included in context
	1.5 Context Composition	Context selection, ordering, compression, summarization, and allocation of the available context budget
2. Memory & Knowledge	2.1 Episodic / Experience Memory	Records of prior interactions, execution trajectories, successful and failed attempts, and task-specific reflections
	2.2 Semantic / Knowledge Memory	Persistent factual, conceptual, and domain knowledge, including structured or external knowledge stores
	2.3 Procedural Memory	Reusable strategies, experience-derived rules, heuristics, procedures, and playbooks
	2.4 Memory Representation / Organization	Memory schemas, hierarchical structures, summaries, graphs, indexes, and vector representations
	2.5 Memory Operations	Policies governing memory writing, retrieval, updating, consolidation, prioritization, and forgetting
3. Harness / Workflow & Control	3.1 Workflow / Graph	Agent graphs, pipelines, nodes, edges, and execution dependencies between computational stages
	3.2 Planning / Decomposition	Task decomposition, planning modules, subgoal structures, and execution order
	3.3 Routing / Scheduling	Model routing, agent routing, tool routing, execution scheduling, and computational resource allocation
	3.4 Verification / Reflection Loop	Control loops for checking and revising intermediate or final outputs, including critic-revise, verify-retry, and reflection procedures
	3.5 Multi-agent Structure / Protocol	Agent population, role assignment, interaction topology, coordination mechanisms, and communication protocols
	3.6 Harness Implementation / Scaffold Code	Source code implementing the agent scaffold, orchestration logic, workflow controller, and runtime coordination mechanisms

Continued on next page

Table 11 continued

Primary Target	Sub-target	Modified Component
4. Tools & Skills	4.1 Tool Set / Inventory	The collection of tools, APIs, external services, and callable capabilities available to the system
	4.2 Tool Definition / Interface	Tool descriptions, function signatures, schemas, API wrappers, and input-output specifications
	4.3 Tool Implementation	Executable code and internal logic implementing tools or callable external functions
	4.4 Skill / Macro Library	Reusable skills, macros, subroutines, procedures, and higher-level behavioral modules
	4.5 Skill Composition	Rules and structures for combining lower-level skills into higher-level procedures or capabilities
5. Model	5.1 Model Weights	Trainable parameters of the backbone model
	5.2 Adapter / Auxiliary Module	Parameters of LoRA modules, adapters, memory modules, and other trainable auxiliary components
	5.3 Architecture	Network architecture, module composition, connectivity patterns, and computational structure
	5.4 Inference Policy / Configuration	Persistent decoding strategies, test-time policies, inference configurations, and runtime model settings
6. Trainer / Optimization System	6.1 Training Objective	Loss functions, training rewards, learning objectives, and optimization criteria
	6.2 Optimizer / Update Rule	Optimizers, parameter-update algorithms, learning-rate policies, and associated hyperparameters
	6.3 Training Schedule / Pipeline	Ordering, configuration, and interaction of training stages such as fine-tuning, reinforcement learning, and distillation
	6.4 Data Selection / Curriculum	Training-sample selection rules, data mixtures, difficulty schedules, and curriculum policies
	6.5 Search / Meta-optimization Procedure	Evolutionary, search, selection, and meta-optimization procedures used to generate and select candidate improvements
7. Evaluator & Feedback System	7.1 Evaluator / Judge	Evaluators, critics, graders, judge models, and related components for assessing candidate outputs or system variants
	7.2 Reward / Fitness Function	Reward models, fitness functions, utility functions, preference models, and scoring rules
	7.3 Verifier	Correctness verifiers, test generators, proof checkers, consistency checks, and validation mechanisms
	7.4 Feedback / Credit Assignment	Mechanisms that transform observed outcomes into localized, aggregated, or temporally assigned improvement signals

Continued on next page

Table 11 continued

Primary Target	Sub-target	Modified Component
8. Data & Environment	8.1 Training / Experience Data	Training datasets, replay buffers, experience pools, interaction trajectories, and other data used for subsequent learning
	8.2 Task / Curriculum Generator	Components that generate tasks, problems, challenges, or training episodes for subsequent improvement cycles
	8.3 Environment / Simulator	Environment dynamics, simulators, interaction rules, task worlds, and structures governing system–environment interaction
	8.4 World Model	Learned models used to represent, predict, generate, or simulate environmental states and dynamics
9. External Artifact	9.1 Program / Solution Code	Task-level programs, patches, or solution code produced by the system, excluding code implementing the agent itself
	9.2 Algorithm	Algorithms, kernels, mathematical procedures, symbolic methods, and computational techniques
	9.3 Scientific / Design Artifact	Scientific hypotheses, experimental designs, architectures, circuits, engineering designs, and related research artifacts
10. Full-system / Co-evolution	10.1 Multi-target Harness	Multiple harness-level components jointly modified within the same improvement process, such as prompts, memory, workflows, and tools
	10.2 Model–Harness Co-evolution	Model parameters and surrounding harness or scaffold components jointly modified across improvement cycles
	10.3 Improvement-loop / Meta-RSI	The mechanism that generates, evaluates, selects, and applies candidate changes across successive improvement cycles

Appendix B: Industry Landscape

表 12 按公司范式和改进目标对所调查的工业系统进行了分类。

新兴 RSI/自我提升领域中的企业原型

每行一个产品或代表性作品；共 72 家不同的公司/团队；开源快照时间：2026 年 9 月

组织原则：公司原型，而非地理区域。*RSI* 标签是对 *B0–L5* 方案的紧凑映射，并不表示该公司本身使用该标签。🌐 = 可点击的原始来源。

Table 12 按公司原型划分的瑞斯（RSI）/自我提升企业格局。

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	🌐
A. Frontier foundation-model & general-agent labs Broad model/platform labs; RSI appears as one capability frontier rather than the sole company thesis.						
OpenAI	Research acceleration / automated research intern	AI-for-AI research	research code; experiments; evals	code; experiments; candidate inte-gration	L2	🌐
	GPT-Red	self-play robustness	red-team policy; adversarial data	attack; defend; train; evaluate	L2	🌐
	Self-improving tax agents	production adaptation	agent code; prompts; eval set	mine failures; patch; evaluate	L4 cand.	🌐
	Harness Engineering	agent-first software R&D	repo; CI; agent instructions	code; test; PR; repair	L1-L2 adj.	🌐
	Symphony	agent orchestration	task state; repo; workflows	schedule; execute; handoff	L1 adj.	🌐
Anthropic	AgentKit / prompt optimizer / RFT	agent optimization stack	prompts; graders; weights	optimize; grade; fine-tune	L1-L2	🌐
	Deep Research	autonomous research	task-local evidence	search; browse; synthesize	B0	🌐
	When AI builds itself	RSI roadmap	AI-development process	code; experiments; future successor design	L5 target	🌐
	Automated Weak-to-Strong Re-searcher	automated alignment research	hypotheses; code; experiment logs	propose; train; evaluate; share	L2	🌐
	Tool optimization with Claude	tool self-optimization	tool specs; implementations	analyze traces; rewrite; evaluate	L2	🌐
	Harness design for long-running apps	autonomous software R&D	harness; evaluator; app code	plan; generate; evaluate; iterate	L2 adj.	🌐
	Effective long-running agent har-nesses	cross-context persistence	progress files; git state	initialize; code; handoff	L1 adj.	🌐
	Agent Skills	persistent skill substrate	skills; scripts; resources	discover; load; reuse	L1 enabler	🌐
	Multi-agent Research	autonomous research	task-local findings	plan; spawn; search; synthesize	B0	🌐
	Managed Agents	long-horizon agent in-frastructure	stable interface; harness	run; resume; supervise	L1 enabler	🌐
	Parallel Claude compiler project	autonomous software engineering	shared codebase; tests	decompose; code; test; coordinate	B0-L1 adj.	🌐
Google DeepMind	AlphaEvolve	task-specific program search	algorithms; kernels; system code	generate; mutate; evaluate; select	B0	🌐

continued on next page

Table 12 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
NVIDIA	AI Co-Scientist	scientific hypothesis search	hypotheses; research proposals	generate; debate; rank; refine	L2 adj.	
	AlphaChip	AI-for-hardware co-design	chip layouts; design policy	place; score; learn; transfer	L2 adj.	
	Eureka	reward / policy design	reward code; robot policies	reward synthesis; simulation; policy training	L2	
	ASPIRE	reward / policy design	reward code; robot policies	reward synthesis; simulation; policy training	L2	
Microsoft	Agent Lightning	agent reinforcement learning	policy weights; experience traces	collect; credit; train; evaluate	L2	
	Agent Lightning v1.0	harnessed agentic RL	harness traces; policy weights	interact; retokenize; train; benchmark	L2	
Meta	SkillOpt	skill optimization	skill files; instructions	edit; evaluate; optimize	L2	
	ReVeal	self-verifying code agents	code; tests; verifier policy	generate; verify; revise; scale	L2	
	Universal Verifier / auto-research Self-Taught Evaluator	agent verification R&D evaluator self-training	rubrics; verifier; eval pipeline	design; test; compare; refine	L2 adj.	
			judge model; synthetic preferences	generate; judge; train; iterate	L2	
	HyperAgents	meta-agent self-modification	task agent; meta agent; program	solve; self-edit; evaluate; archive	L5 cand.	
Alibaba / Qwen	Qwen-Agent	agent training / synthetic environments	training data; environments; post-training	simulation; data synthesis; tool use; RL	L1-L2	
	AgentWorld	agent training / synthetic environments	training data; environments; post-training	simulation; data synthesis; tool use; RL	L1-L2	
	Qwen-Scope	agent training / synthetic environments	training data; environments; post-training	simulation; data synthesis; tool use; RL	L1-L2	
DeepSeek	R1	reasoning self-bootstrapping	reasoning policy; verifier	self-generated reasoning; RL; verifier iteration	L2	
	Math-V2	reasoning self-bootstrapping	reasoning policy; verifier	self-generated reasoning; RL; verifier iteration	L2	
Tencent AI Lab	R-Zero	self-play reasoning	tasks; pseudo-labels; policy weights	challenge generation; solve; vote; RL	L2-L3	
ByteDance Seed	Seed-Thinking	model / agent post-training	data; reward; policy weights	data filtering; reward verification; RL	L1-L2	
	Seed1.5-VL	model / agent post-training	data; reward; policy weights	data filtering; reward verification; RL	L1-L2	
MiniMax	M2	persistent cloud agents	memory; skills; agent policy	tool use; long-run execution; skill reuse	L1-L2	

continued on next page

Table 12 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
Moonshot AI	MaxHermes	persistent cloud agents	memory; skills; agent policy	tool use; long-run execution; skill reuse	L1-L2	
	MaxClaw	persistent cloud agents	memory; skills; agent policy	tool use; long-run execution; skill reuse	L1-L2	
	Kimi K3	long-horizon agents	agent orchestration; tool policy	planning; tool use; swarm coordination	L1-L2 adj.	
	Agent Swarm	long-horizon agents	agent orchestration; tool policy	planning; tool use; swarm coordination	L1-L2 adj.	
Zhipu AI / Z.AI	AutoGLM	computer-use agent training	policy weights; virtual-phone environments	perception; planning; action; RL	L2	
	AgentRL	computer-use agent training	policy weights; virtual-phone environments	perception; planning; action; RL	L2	
Deep Cogito	Cogito v2	reasoning post-training	reasoning policy; weights	search; distill; iterative alignment	L1-L2	
	IDA	reasoning post-training	reasoning policy; weights	search; distill; iterative alignment	L1-L2	
Poolside	Model Factory	automated model R&D	synthetic data; RL; architecture	eval; code-exec RL; ablations; data mix	L1-L2	
Thinking Machines Lab	Tinker Agent RL	model customization / agent RL	policy weights; tool-use policy	RL; LLM-judge grading; tool discovery	L1-L2	
	Inkling	model customization / agent RL	policy weights; tool-use policy	RL; LLM-judge grading; tool discovery	L1-L2	
Nous Research	Hermes Agent	persistent agents	skills; memory; tool gateway	memory; skill reuse; tool orchestration	L1-L2	
	skills / memory	persistent agents	skills; memory; tool gateway	memory; skill reuse; tool orchestration	L1-L2	
B. RSI-native / AI4AI companies Self-improvement, AI-for-AI, self-evolution, or recursive improvement is central to the company/research thesis.						
Ricursive Intelligence	AI-chip co-design	AI systems / chips	EDA designs; compute stack	design search; verify; iterate	L2; L5 vision	
Recursive Imbue	Automated AI Research	model-training research	training recipes; kernels; code	ideas; experiments; branch merge	L2	
	Catalyst	research search	recipes; code; hypotheses	population search; experiments; interpretation	L2-L3	
	Darwinian Evolver	research search	recipes; code; hypotheses	population search; experiments; interpretation	L2-L3	
Weco AI	AIDE2	meta-improvement of researcher	research harness; improver code	rewrite improver; eval; inheritance	L5 cand.	
Sakana AI	Darwin Godel Machine	self-editing agents / AI research	agent source; research pipeline	self-modify; benchmark; archive; experiments	L5 cand.	
	AI Scientist	self-editing agents / AI research	agent source; research pipeline	self-modify; benchmark; archive; experiments	L5 cand.	

continued on next page

Table 12 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
Evolvent AI	Org self-evolving agents	software-agent evolution	skills; memory; code; environments	tasks; feedback; refactor; skill updates	L2-L3	
	RSIBench	software-agent evolution	skills; memory; code; environments	tasks; feedback; refactor; skill updates	L2-L3	
	Terrarium	software-agent evolution	skills; memory; code; environments	tasks; feedback; refactor; skill updates	L2-L3	
MetaCircle	ComfyResearch	AI4AI / autoresearch	training workflows; research hypotheses	experiment compose; code; hypothesis; scoring	L2; L5 vision	
	OPHIS	AI4AI / autoresearch	training workflows; research hypotheses	experiment compose; code; hypothesis; scoring	L2; L5 vision	
Frontis AI / Xianyuan	OpenRSI	AI4AI / self-improving agents	skills; memory; harness; weights	experience; update search; eval; post-training	L2-L3	
	Frontis-MA1	AI4AI / self-improving agents	skills; memory; harness; weights	experience; update search; eval; post-training	L2-L3	
EvoMap	Evolver	shared code evolution	genes / capsules; code assets	generate; test; publish; reuse; adapt	L2-L3	
	GEP	shared code evolution	genes / capsules; code assets	generate; test; publish; reuse; adapt	L2-L3	
	GeneBench	shared code evolution	genes / capsules; code assets	generate; test; publish; reuse; adapt	L2-L3	
Endless Frontier	BigBang-v1	AI-research data generation	synthetic programs; training data; critic	generate; execute; critique; meta-critique	L2-L3	
Mirendil	Self-accelerating AI	AI R&D automation	model code; experiments; research stack	experiment generation; execution; iteration	L2	
	AI Scientist	AI R&D automation	model code; experiments; research stack	experiment generation; execution; iteration	L2	
Chaoyan Intelligence*	TUMIX	self-evolving research models	research policy; tool-use policy	questioning; experiments; code; self-check	L2-L3	
	R1-Code-Interpreter	self-evolving research models	research policy; tool-use policy	questioning; experiments; code; self-check	L2-L3	
	AI Scientist	self-evolving research models	research policy; tool-use policy	questioning; experiments; code; self-check	L2-L3	
Theseus	Argus	long-horizon self-evolving agents	experience; memory; environment setup	plan; code; review; experience writeback	L2-L3	
	SetupX	long-horizon self-evolving agents	experience; memory; environment setup	plan; code; review; experience writeback	L3	
Adaption Labs	AutoScientist	AI-research automation	training recipes; datasets; code	research plan; experiments; selection	L2	

continued on next page

Table 12 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
	Forge	AI-research automation	training recipes; datasets; code	research plan; experiments; selection	L2	
C. Autonomous R&D & scientific-discovery companies Primary product is automated research/discovery; RSI relevance comes from automating experiment and knowledge-production loops.						
Periodic Labs	Autonomous laboratory	AI for science	hypotheses; experiment data; models	experiment design; lab run; learning	L3 cand.	
FutureHouse	BixBench	AI-scientist evaluation	research tasks; benchmark frontier	bioinformatics workflows; open-ended eval	B0-L2 adj.	
Edison Scientific	Kosmos	autonomous science	hypotheses; code; scientific artifacts	literature; experiments; synthesis	L2-L3	
Axiom Math	Putnam 2025	formal mathematics	proofs; verifier traces	conjecture; proof search; formal verification	L2	
	IMO 2026	formal mathematics	proofs; verifier traces	conjecture; proof search; formal verification	L2	
Harmonic	Aristotle	theorem proving	formal proofs	translate; prove; verify	L1-L2	
Core Automation	AI systems-research stack	automated systems research	systems code; research hypotheses	design; code; benchmark; iterate	L2 cand.	
Discovery Loop	Autonomous discovery loop	automated experimentation	protocols; findings	hypothesis; experiment; analysis; next-step	L3 cand.	
Lila Sciences	Autonomous Science platform	scientific discovery	hypotheses; experiments; post-training data	design; lab execution; real-time learning	L3 cand.	
Karpathy / autoresearch	autoresearch	narrow automated research	training code; configs	edit; train; measure; keep	L2	
Prime Intellect	Autonomous research	model R&D	training recipe; optimizer; code	hypothesis; GPU runs; ablation	L2	
	Speedrun Frontier	model R&D	training recipe; optimizer; code	hypothesis; GPU runs; ablation	L2	
Analemma	FARS	autonomous research	proposals; code; logs; papers	topic; experiment; analysis; writing	L2-L3	
Novix	AutoAgent	AI research agent	research workflow; eval harness	idea; tool use; experiments; reporting	L2	
	OpenHarness	AI research agent	research workflow; eval harness	idea; tool use; experiments; reporting	L2	
	AI-Researcher	AI research agent	research workflow; eval harness	idea; tool use; experiments; reporting	L2	
UniPat	UniScientist	research / evaluation agents	research traces; benchmarks	experiments; coding; multi-agent eval	L1-L2	
	UniSwarm	research / evaluation agents	research traces; benchmarks	experiments; coding; multi-agent eval	L1-L2	
	UniMath	research / evaluation agents	research traces; benchmarks	experiments; coding; multi-agent eval	L1-L2	
Kai Chen / venture TBD*	Intern-S1	AI for science models	scientific model; training data	model training; scientific reasoning	adj.; entity TBD	

continued on next page

Table 12 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
D. Agent optimization, evaluation & learning infrastructure						
Infrastructure for feedback, skills, RL, evaluation, simulators, training data, or production-agent improvement.						
Warp	Skill optimization loop	coding-agent skills	skills; instructions; examples	feedback mining; skill rewrite; eval	L2	
Factory	Signals	production coding agents	agent behavior; product code	session mining; failure clusters; patch PRs	L4	
	Software Factory	production coding agents	agent behavior; product code	session mining; failure clusters; patch PRs	L4	
LangChain	LangSmith self-improving evaluators	evaluator alignment	judge prompts; evaluator model	feedback ingest; judge update; eval	L1-L2	
Replit	Agent 3	software engineering	application code; tests	build; test; repair; long runs	L1-L2	
MorphMind	Caliper	agent calibration / org memory	agent skills; shared experience	measure; route; reuse experience	L1-L2 adj.	
DatologyAI	BeyondWeb	data optimization	training corpus; synthetic data	curate; filter; synthesize; benchmark	L1-L2	
	DatBench	data optimization	training corpus; synthetic data	curate; filter; synthesize; benchmark	L1-L2	
Ineffable Intelligence	RL infrastructure	RL / experience infrastructure	training environments; trajectories	rollouts; reward; distributed RL	L1-L3 enabler	
Goodfire	Ember	interpretability-driven training	feature activations; reward signals	feature discovery; feedback; RL	L1-L2	
	RLFR	interpretability-driven training	feature activations; reward signals	feature discovery; feedback; RL	L1-L2	
Patronus AI	Generative Simulators	evaluation / simulation	simulators; eval suites	scenario generation; grading; failure analysis	L2-L3 enabler	
	Percival	evaluation / simulation	simulators; eval suites	scenario generation; grading; failure analysis	L2-L3 enabler	
Braintrust	Loop	production feedback / eval	prompts; evals; datasets	trace ingest; scoring; prompt optimization	L1-L2	
	Autoevals	production feedback / eval	prompts; evals; datasets	trace ingest; scoring; prompt optimization	L1-L2	
Mechanize	RL environments	experience / eval infrastructure	RL tasks; graders; environments	task design; grading; training signal	L3 enabler	
	GBA Eval	experience / eval infrastructure	RL tasks; graders; environments	task design; grading; training signal	L3 enabler	
Kando AI	Early company signal	undisclosed / early-stage	undisclosed	undisclosed	unverified	
Entropy Order*	Data-expert platform	high-quality training data	expert data; eval assets	data production; benchmarking	L1-L3 enabler	
Compounding Intelligence / CORAL*	CORAL	multi-agent research	shared notes; skills; logs	parallel experiments; knowledge sharing; eval	L2-L3	

continued on next page

Table 12 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
Naive.ai*	Research-lab signal	early RSI lab	undisclosed	undisclosed	RSI claim; unverified	
E. Embodied, world-model & continual-adaptation companies Persistent adaptation is tied to world models, robotics, environments, or test-time/continual learning.						
AI2 Robotics	FiS-VLA	embodied policy learning	VLA policy; action data	data; training; planning; control	L2-L3 adj.	
	Video2Act	embodied policy learning	VLA policy; action data	data; training; planning; control	L2-L3 adj.	
X Square Robot	WALL	world models / skill acquisition	world model; skills; action policy	video skill capture; world prediction; control	L2-L3	
	HOST	world models / skill acquisition	world model; skills; action policy	video skill capture; world prediction; control	L2-L3	
Galaxea AI	G0 / G0.5	VLA R&D platform	VLA model; data; deployment stack	training; eval; real-robot deployment	L2 adj.	
	GForge	VLA R&D platform	VLA model; data; deployment stack	training; eval; real-robot deployment	L2 adj.	
TARS Robotics	AWE	tactile world models	tactile world model; manipulation policy	multimodal sensing; prediction; control	L2-L3 adj.	
	TacForeSight	tactile world models	tactile world model; manipulation policy	multimodal sensing; prediction; control	L2-L3 adj.	
Synapx Dynamics	SYNWorld	embodied data / model stack	data; world model; post-training	data synthesis; training; eval; control	L2-L3	
	OctoMind	embodied data / model stack	data; world model; post-training	data synthesis; training; eval; control	L2-L3	
	OctoSense	embodied data / model stack	data; world model; post-training	data synthesis; training; eval; control	L2-L3	
MirrOS	Code as Worlds	world models / test-time adaptation	world code; fast weights	environment modeling; eval; test-time update	L1-L2	
	Spatial-TTT	world models / test-time adaptation	world code; fast weights	environment modeling; eval; test-time update	L1-L2	
Wuya Zhiyuan*	SHINE	continual / test-time learning	LoRA; skill vectors; weights	context adaptation; skill transfer	L1-L2	
	LIFT	continual / test-time learning	LoRA; skill vectors; weights	context adaptation; skill transfer	L1-L2	
	PaST	continual / test-time learning	LoRA; skill vectors; weights	context adaptation; skill transfer	L1-L2	
Singularity Escape / Nexus*	Nexus	collaborative self-evolving agents	shared state; org knowledge; harness	collaboration; experience capture; harness adaptation	L3-L4 cand.	
F. Persistent-memory & personal-AI companies Cross-task memory, identity, or reusable skills are the main substrate of adaptation.						

continued on next page

Table 12 – continued

Company	Product / representative work	Sub-scenario	Improvement target / artifact	AI-controlled part	RSI relation	
Engram	Knowledge Cartridges	persistent enterprise memory	knowledge modules; memory	extract; store; retrieve; adapt	L1-L2 adj.	
Lemon AI / Hexdo	LemonAI Evolving	local persistent agents	workspace; memory; experience library	web / files / code; persistence; reuse	L1-L2	
EverMind	EverOS	agent memory / OS	long-term memory; skills	retrieve; skill forge; active tasks; eval	L2-L3	
	Raven	agent memory / OS	long-term memory; skills	retrieve; skill forge; active tasks; eval	L2-L3	
	EverMemOS	agent memory / OS	long-term memory; skills	retrieve; skill forge; active tasks; eval	L2-L3	
Mindverse	Second Me	personal memory / adaptation	identity model; memory; LoRA	memory modeling; personalization; retrieval	L1-L2	
	Me.bot	personal memory / adaptation	identity model; memory; LoRA	memory modeling; personalization; retrieval	L1-L2	

阅读指南。B0 仅表示任务级自主性。L1–L5 按照综述中逐步增强的自主性等级排列。“adj.” 标记与 RSI 相邻的基础设施或行为；“cand.” 标记一种可能但尚未完全证实的等级；“target” 标记一个明确的未来目标，而非当前已证实的能力。星号标记的条目其公司身份或公开技术边界仍需更强的验证。